

Lời nói đầu

Cuốn **Giáo trình C++ — Sách giáo viên — Quyển 2** là tập tiếp theo trong bộ tài liệu đào tạo lập trình thuật toán C++ nâng cao iKHEDU, dành riêng cho quý thầy cô giáo và các huấn luyện viên chuyên trách đội tuyển tin học. Tài liệu này cung cấp trọn bộ lời giải chi tiết, mã nguồn C++ tối ưu và hệ thống **phương pháp luận giảng dạy thực chiến chuyên sâu** (ý tưởng và phân tích thuật toán, bảng mô phỏng chạy tay từng bước trên số liệu mẫu thực tế, lưu ý và các bẫy lỗi kinh điển thường gặp) cho toàn bộ **110 bài toán thực hành** từ Bài 08 đến Bài 14 thuộc các chủ đề thuật toán trọng tâm: Đồng dư thức & lũy thừa nhị phân, Xử lý số nguyên lớn (BigInt), Thuật toán đệ quy & cây gọi hàm, Kỹ thuật chia để trị, Thuật toán quay lui & nhánh cận, Quy hoạch động 1D & dãy con tăng dài nhất, và Quy hoạch động 2D & bài toán cái túi (Knapsack).

Mỗi bài toán trong sách giáo viên được cấu trúc thống nhất và tinh gọn qua 4 mục trọng tâm: (1) Ý tưởng & Phân tích thuật toán gắn liền với bản chất toán học; (2) Bảng chạy tay trực quan (Dry Run Table) trên các giá trị số thực tế của mẫu thử (Sample); (3) Lưu ý & các bẫy lỗi kinh điển học sinh thường gặp trong môi trường lập trình; và (4) Mã nguồn C++ tham khảo tối ưu nhất (Fast I/O, Safe Input, 0 std:::). Tài liệu là cẩm nang đặc lực hỗ trợ thầy cô tổ chức các giờ dạy chất lượng cao, phân tích sai sót của học sinh và đồng hành hiệu quả trong từng bài học.

CHƯƠNG 03: MẢNG TIỀN TỔ & TÌM KIẾM NHỊ PHÂN

Bài 09: Phép toán BIT & biểu diễn trạng thái

Bài 01 [cppb_bit_01_bat_tat_kiem_tra_bit]: Bật, Tắt Và Kiểm Tra Bit Thứ K

Bối cảnh: Trong kiến trúc hệ điều hành nhúng, thanh ghi trạng thái phần cứng của vi điều khiển được biểu diễn dưới dạng số nguyên không âm N (64-bit). Mỗi bit thứ k đại diện cho trạng thái của một cổng kết nối cảm biến ngoại vi (1 là đang bật, 0 là đang tắt). Người lập trình viên nhận Q lệnh điều khiển từ xa để thao tác trực tiếp trên các bit của thanh ghi.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho số nguyên không âm N và Q thao tác: loại 1 (bật bit thứ k), loại 2 (tắt bit thứ k), loại 3 (kiểm tra trạng thái bit thứ k). Với thao tác loại 3, in ra 1 nếu bit đang bật, ngược lại in 0.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($0 \leq N \leq 10^{18}$, $1 \leq Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng gồm 2 số nguyên thể hiện loại thao tác và chỉ số bit k ($0 \leq k \leq 62$).

Đầu ra (Output):

- Với mỗi thao tác loại 3, in ra kết quả trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 4 3 0 3 1 1 1 3 1	1 0 1

Giải thích: $N = 5$ có biểu diễn nhị phân là 101_2 :

- Thao tác 3 0: Bit thứ 0 có giá trị 1 $\rightarrow$ in 1.
- Thao tác 3 1: Bit thứ 1 có giá trị 0 $\rightarrow$ in 0.
- Thao tác 1 1: Bật bit thứ 1 lên 1 $\rightarrow N$ trở thành $111_2 = 7$.
- Thao tác 3 1: Bit thứ 1 hiện tại là 1 $\rightarrow$ in 1.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên không âm N và Q thao tác: loại 1 (bật bit thứ k), loại 2 (tắt bit thứ k), loại 3 (kiểm tra trạng thái bit thứ k). Với thao tác loại 3, in ra 1 nếu bit đang bật, ngược lại in 0.
- **Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 4 3 0 3 1 1 1 3 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $N = 5$ có biểu diễn nhị phân là 101_2 : - Thao tác 3 0: Bit thứ 0 có giá trị 1 $\rightarrow$ in 1. - Thao tác 3 1: Bit thứ 1 có giá trị 0 $\rightarrow$ in 0. - Thao tác 1 1: Bật bit thứ 1 lên 1 $\rightarrow N$ trở thành $111_2 = 7$. - Thao tác 3 1: Bit thứ 1 hiện tại là 1 $\rightarrow$ in 1. | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 0 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $N = 5$ có biểu diễn nhị phân là 101_2 :

- Thao tác 3 0: Bit thứ 0 có giá trị 1 $\rightarrow$ in 1.
- Thao tác 3 1: Bit thứ 1 có giá trị 0 $\rightarrow$ in 0.
- Thao tác 1 1: Bật bit thứ 1 lên 1 $\rightarrow N$ trở thành $111_2 = 7$.
- Thao tác 3 1: Bit thứ 1 hiện tại là 1 $\rightarrow$ in 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    unsigned long long n;
    int q;
    if (!(cin >> n >> q)) return 0;

    while (q--) {
        int type, k;
        cin >> type >> k;
        if (type == 1) {
            n |= (1ULL << k);
        } else if (type == 2) {
            n &= ~(1ULL << k);
        }
    }
}
```

```

    } else if (type == 3) {
        cout << ((n >> k) & 1ULL) << "\n";
    }
}

return 0;
}

```

Bài 02 [cppb_bit_02_dem_so_bit_mot]: Đếm Số Lượng Bit 1 (Popcount)

Bối cảnh: Một giao thức mã hóa dữ liệu mạng kiểm tra tính toàn vẹn của gói tin bằng cách đếm trọng số Hamming — tức số lượng bit 1 xuất hiện trong mã khóa nhị phân 64-bit của số nguyên N . Hãy đếm chính xác số lượng bit 1 trong biểu diễn nhị phân của N .



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho số nguyên không âm N ($0 \leq N \leq 10^{18}$). Hãy đếm và in ra số lượng bit có giá trị bằng 1 trong biểu diễn nhị phân của N .

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên không âm N ($0 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng bit 1 của N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
13	3

Giải thích: 13 biểu diễn nhị phân là 1101₂, có tổng cộng 3 bit 1 (tại các vị trí bit 0, 2, 3). Kết quả in ra: 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên không âm N ($0 \leq N \leq 10^{18}$). Hãy đếm và in ra số lượng bit có giá trị bằng 1 trong biểu diễn nhị phân của N .
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.

- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 13 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | 13 biểu diễn nhị phân là 1101₂, có tổng cộng 3 bit 1 (tại các vị trí bit 0, 2, 3). Kết quả in ra: 3.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: 13 biểu diễn nhị phân là 1101₂, có tổng cộng 3 bit 1 (tại các vị trí bit 0, 2, 3). Kết quả in ra: 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

void solve() {
    unsigned long long n;
    cin >> n;
    cout << __builtin_popcountll(n) << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        solve();
    }
    return 0;
}
```

Bài 03 [cppb_bit_03_kiem_tra_luy_thua_cua_hai]: Kiểm Tra Số Có Phải Lũy Thừa Của 2

Bối cảnh: Trong kỹ thuật cấu hình bộ nhớ đệm máy tính, dung lượng bộ nhớ hợp chuẩn bắt buộc phải là một lũy thừa của 2 (nghĩa là có dạng 2^k với $k \geq 0$). Hệ thống kiểm tra hợp lệ cần xác định nhanh xem số nguyên dương N có phải là một lũy thừa của 2 hay không bằng phép toán bit $\mathcal{O}(1)$.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Kiểm tra N có phải là lũy thừa của 2 không. In YES nếu đúng, ngược lại in NO.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
16	YES

Giải thích: $16 = 2^4$ là một lũy thừa của 2. Phép toán bit: $(16 \& 15) = 0$. Do đó in YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Kiểm tra N có phải là lũy thừa của 2 không. In YES nếu đúng, ngược lại in NO.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 16 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $16 = 2^4$ là một lũy thừa của 2. Phép toán bit: $(16 \& 15) = 0$. Do đó in YES.... |
 Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $16 = 2^4$ là một lũy thừa của 2. Phép toán bit: $(16 \& 15) = 0$. Do đó in YES.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

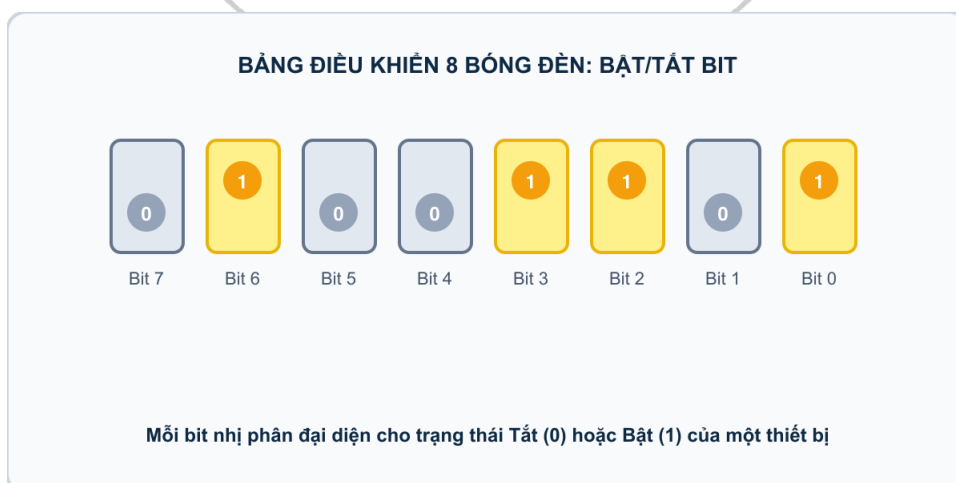
void solve() {
    unsigned long long n;
    cin >> n;
    if (n > 0 && (n & (n - 1)) == 0) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        solve();
    }
    return 0;
}
```

Bài 04 [cppb_bit_04_tìm_phan_tu_don_le]: Tìm Phần Tử Xuất Hiện 1 Lần Duy Nhất

Bối cảnh: Trong một trò chơi ghép thẻ bài đôi gồm $2N + 1$ lá bài mang các con số nguyên, toàn bộ các lá bài đều có cặp trùng khớp ngoại trừ đúng một lá bài duy nhất không có đối tác ghép cặp. Hãy tìm giá trị của lá bài cô độc đó trong thời gian $O(N)$ và bộ nhớ $O(1)$.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho mảng gồm $2N + 1$ số nguyên, trong đó mọi phần tử đều xuất hiện đúng 2 lần trừ 1 phần tử xuất hiện đúng 1 lần. Hãy tìm phần tử duy nhất đó.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa $2N + 1$ số nguyên $A_1, A_2, \dots, A_{2N+1}$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị phần tử xuất hiện một lần duy nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 4 1 2 1 2	4

Giải thích: Các số 1 và 2 đều xuất hiện 2 lần. Số 4 chỉ xuất hiện 1 lần duy nhất. Phép XOR toàn bộ mảng triệt tiêu các cặp giống nhau và giữ lại đúng số 4. Kết quả in ra: 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm $2N + 1$ số nguyên, trong đó mọi phần tử đều xuất hiện đúng 2 lần trừ 1 phần tử xuất hiện đúng 1 lần. Hãy tìm phần tử duy nhất đó.
- **Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 4 1 2 1 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các số 1 và 2 đều xuất hiện 2 lần. Số 4 chỉ xuất hiện 1 lần duy nhất. Phép XOR toàn bộ mảng triệt tiêu các cặp giống nhau... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các số 1 và 2 đều xuất hiện 2 lần. Số 4 chỉ xuất hiện 1 lần duy nhất. Phép XOR toàn bộ mảng triệt tiêu các cặp giống nhau và giữ lại đúng số 4. Kết quả in ra: 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n;
if (!(cin >> n)) return 0;

int total_elements = 2 * n + 1;
long long ans = 0;
for (int i = 0; i < total_elements; ++i) {
    long long x;
    cin >> x;
    ans ^= x;
}

cout << ans << "\n";
return 0;
}
```

Bài 05 [cppb_bit_05_tim_hai_so_don_le]: Tìm Hai Số Xuất Hiện 1 Lần Duy Nhất

Bối cảnh: Trong một sự kiện bắt cặp khiêu vũ gồm $2N + 2$ người tham gia mang các số định danh nguyên, sau khi các cặp đôi ghép thành công thì còn lại đúng 2 người chưa tìm được bạn nhảy. Hãy tìm ra số định danh của 2 người đó theo thứ tự tăng dần.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho mảng gồm $2N + 2$ số nguyên, trong đó mọi phần tử xuất hiện 2 lần trừ đúng 2 phần tử X và Y xuất hiện 1 lần duy nhất. Hãy tìm và in ra X, Y theo thứ tự tăng dần.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa $2N + 2$ số nguyên $A_1, A_2, \dots, A_{2N+2}$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 2 số nguyên X, Y ($X < Y$) cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 1 2 3 2 1 4	3 4

Giải thích: Các số 1 và 2 đều xuất hiện 2 lần. Hai số chỉ xuất hiện 1 lần duy nhất là 3 và 4. Kết quả in theo thứ tự tăng dần: 3 4.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng gồm $2N + 2$ số nguyên, trong đó mọi phần tử xuất hiện 2 lần trừ đúng 2 phần tử X và Y xuất hiện 1 lần duy nhất. Hãy tìm và in ra X, Y theo thứ tự tăng dần.
- **Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $O(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 1 2 3 2 1 4 | Khởi tạo cấu trúc dữ liệu ban đầu | |
 2 | Thực thi thuật toán tối ưu | Các số 1 và 2 đều xuất hiện 2 lần. Hai số chỉ xuất hiện 1 lần duy nhất là 3 và 4. Kết quả in theo thứ tự tăng dần: 3 4... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả |
 Output: 3 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các số 1 và 2 đều xuất hiện 2 lần. Hai số chỉ xuất hiện 1 lần duy nhất là 3 và 4. Kết quả in theo thứ tự tăng dần: 3 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    int total_elements = 2 * n + 2;
    vector<long long> a(total_elements);
    long long xor_sum = 0;
    for (int i = 0; i < total_elements; ++i) {
        cin >> a[i];
        xor_sum ^= a[i];
    }

    // Lấy bit 1 phân biệt
    long long diff_bit = xor_sum & (-xor_sum);

    long long num1 = 0, num2 = 0;
    for (long long val : a) {
        if (val & diff_bit) {
            num1 ^= val;
        }
    }
    num2 = xor_sum ^ num1;

    cout << num1 << " " << num2 << endl;
}
```

```

    } else {
        num2 ^= val;
    }
}

if (num1 > num2) swap(num1, num2);
cout << num1 << " " << num2 << "\n";
return 0;
}

```

Bài 06 [cppb_bit_06_dao_bit_gia_tri_bu_mot]: Đảo Bit Và Giá Trị Bù 1

Bối cảnh: Trong thiết kế mạch logic số ALU, phép bù 1 (đảo toàn bộ các bit 0 thành 1 và 1 thành 0) trên biểu diễn nhị phân hiệu dụng của số nguyên dương N được sử dụng để tính giá trị đối trong biểu diễn số bù. Hãy tìm giá trị thập phân của số thu được sau khi đảo toàn bộ bit của N .



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho số nguyên dương N . Hãy đảo toàn bộ các bit từ bit có trọng số lớn nhất đến bit 0 của N và in ra giá trị thập phân của số mới.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra số nguyên sau khi đã đảo bit.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	2

Giải thích: $5 = 101_2$. Đảo toàn bộ 3 bit hiệu dụng: bit 1 thành 0, bit 0 thành 1 $\rightarrow$ ta được $010_2 = 2$. Kết quả in ra: 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy đảo toàn bộ các bit từ bit có trọng số lớn nhất đến bit 0 của N và in ra giá trị thập phân của số mới.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**

- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise &, |, ^, ~, <<, >> để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $5 = 101_2$. Đảo toàn bộ 3 bit hiệu dụng: bit 1 thành 0, bit 0 thành 1 -> ta được $010_2 = 2$. Kết quả in ra: 2.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $5 = 101_2$. Đảo toàn bộ 3 bit hiệu dụng: bit 1 thành 0, bit 0 thành 1 -> ta được $010_2 = 2$. Kết quả in ra: 2.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    unsigned long long n;
    if (!(cin >> n)) return 0;

    int length = 64 - __builtin_clzll(n);
    unsigned long long mask = (1ULL << length) - 1;
    unsigned long long ans = n ^ mask;

    cout << ans << "\n";
    return 0;
}
```

Bài 07 [cppb_bit_07_duyet_tap_con_bitmask]: Duyệt Toàn Bộ 2^N Tập Con Bằng Mặt Nạ Bit

Bối cảnh: Một người đầu tư mạo hiểm cần phân tích toàn bộ 2^N kịch bản danh mục đầu tư từ N dự án khởi nghiệp tiềm năng. Mỗi kịch bản tương ứng với việc chọn hoặc không chọn từng dự án. Hãy sử dụng kỹ thuật mặt nạ bit (Bitmask) để liệt kê tổng giá trị vốn của tất cả 2^N tập con có thể lập ra.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho tập hợp gồm N số nguyên. Hãy in ra tổng các phần tử của tất cả 2^N tập con theo thứ tự mặt nạ bit tăng dần từ 0 đến $2^N - 1$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 16$).
- Dòng 2: N số nguyên $A_0, A_1, \dots, A_{N-1}$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 2^N dòng, mỗi dòng là tổng các phần tử của tập con tương ứng với mặt nạ bit từ 0 đến $2^N - 1$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 3 5	0 3 5 8

Giải thích: - Mask 0 (00_2): tập rỗng $\rightarrow$ tổng 0. - Mask 1 (01_2): tập $\{A[0]\} = \{3\} \rightarrow$ tổng 3. - Mask 2 (10_2): tập $\{A[1]\} = \{5\} \rightarrow$ tổng 5. - Mask 3 (11_2): tập $\{A[0], A[1]\} = \{3, 5\} \rightarrow$ tổng 8.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho tập hợp gồm N số nguyên. Hãy in ra tổng các phần tử của tất cả 2^N tập con theo thứ tự mặt nạ bit tăng dần từ 0 đến $2^N - 1$.
- **Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 3 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - Mask 0 (00_2): tập rỗng $\rightarrow$ tổng 0. - Mask 1 (01_2): tập $\{A[0]\} = \{3\} \rightarrow$ tổng 3. - Mask 2 (10_2): tập $\{A[1]\} = \{5\} \rightarrow$ t... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 0 3 5 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Mask 0 (00_2): tập rỗng $\rightarrow$ tổng 0.

- Mask 1 (01_2): tập $\{A[0]\} = \{3\} \rightarrow$ tổng 3.
- Mask 2 (10_2): tập $\{A[1]\} = \{5\} \rightarrow$ tổng 5.
- Mask 3 (11_2): tập $\{A[0], A[1]\} = \{3, 5\} \rightarrow$ tổng 8.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    int total_masks = (1 << n);
    for (int mask = 0; mask < total_masks; ++mask) {
        bool first = true;
        for (int i = 0; i < n; ++i) {
            if ((mask >> i) & 1) {
                if (!first) cout << " ";
                cout << a[i];
                first = false;
            }
        }
        cout << "\n";
    }

    return 0;
}
```

Bài 08 [cppb_bit_08_tong_tap_con_bang_s]: Bài Toán Tổng Tập Con Bằng S (Subset Sum)

Bối cảnh: Trong két sắt chứa N thỏi vàng có khối lượng lần lượt là $A_1, A_2, \dots, A_n$. Một khách hàng muốn mua đúng lượng vàng có tổng khối lượng bằng S chỉ vàng. Hãy kiểm tra xem có thể chọn ra một tập con các thỏi vàng trong két để có tổng khối lượng đúng bằng S hay không.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho dãy gồm N số nguyên dương và số nguyên dương S . Hãy kiểm tra xem có tồn tại một tập con có tổng đúng bằng S hay không. In YES nếu có, ngược lại in NO.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và S ($1 \leq N \leq 20, 1 \leq S \leq 10^9$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 9 3 34 4 12	NO

Giải thích: Các tập con có thể tạo được từ $\{3, 34, 4, 12\}$ có tổng lần lượt là: 0, 3, 34, 37, 4, 7, 38, 41, 12, 15, 46, 49, 16, 19, 50, 53. Không có tập con nào có tổng bằng 9. Vì vậy in NO.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy gồm N số nguyên dương và số nguyên dương S . Hãy kiểm tra xem có tồn tại một tập con có tổng đúng bằng S hay không. In YES nếu có, ngược lại in NO.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 9 3 34 4 12 | Khởi tạo cấu trúc dữ liệu ban đầu | |
 2 | Thực thi thuật toán tối ưu | Các tập con có thể tạo được từ $\{3, 34, 4, 12\}$ có tổng lần lượt là: 0, 3, 34, 37, 4, 7, 38, 41, 12, 15, 46, 49, 16, 19, 50, 53... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: NO | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các tập con có thể tạo được từ $\{3, 34, 4, 12\}$ có tổng lần lượt là: 0, 3, 34, 37, 4, 7, 38, 41, 12, 15, 46, 49, 16, 19, 50, 53. Không có tập con nào có tổng bằng 9. Vì vậy in NO.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long s;
    if (!(cin >> n >> s)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    int total_masks = (1 << n);
    for (int mask = 0; mask < total_masks; ++mask) {
        long long current_sum = 0;
        for (int i = 0; i < n; ++i) {
            if ((mask >> i) & 1) {
                current_sum += a[i];
            }
        }
        if (current_sum == s) {
            cout << "YES\n";
            return 0;
        }
    }

    cout << "NO\n";
    return 0;
}
```

Bài 09 [cppb_bit_09_chia_tap_hop_chenh_lech_min]: Chia Tập Hợp Thành 2 Phần Có Tổng Chênh Lệch Nhỏ Nhất

Bối cảnh: Có N kiện hàng với khối lượng lần lượt là $P_1, P_2, \dots, P_n$ cần chia cho hai chiếc xe tải cùng loại để vận chuyển đường dài. Hãy tìm cách phân chia toàn bộ N kiện hàng thành hai phần sao cho độ chênh lệch khối lượng hàng hóa giữa hai xe tải là nhỏ nhất có thể.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho N quả táo với khối lượng $P_1, P_2, \dots, P_N$. Hãy chia táo vào 2 rổ sao cho độ chênh lệch tổng khối lượng giữa hai rổ là nhỏ nhất có thể.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 20$).
- Dòng 2: N số nguyên dương $P_1, P_2, \dots, P_N$ ($1 \leq P_i \leq 10^9$).

Đầu ra (Output):

- In ra độ chênh lệch nhỏ nhất giữa hai phần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 2 7 4 1	1

Giải thích: Tổng khối lượng 5 quả táo là $3 + 2 + 7 + 4 + 1 = 17$. Chia thành hai nhóm: nhóm 1 gồm $\{2, 7\}$ có tổng 9; nhóm 2 gồm $\{3, 4, 1\}$ có tổng 8. Độ chênh lệch giữa hai nhóm là $|9 - 8| = 1$. Đây là mức chênh lệch nhỏ nhất.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho N quả táo với khối lượng $P_1, P_2, \dots, P_N$. Hãy chia táo vào 2 rổ sao cho độ chênh lệch tổng khối lượng giữa hai rổ là nhỏ nhất có thể.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&, |, ^, \sim, <<, >>$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 2 7 4 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Tổng khối lượng 5 quả táo là $3 + 2 + 7 + 4 + 1 = 17$. Chia thành hai nhóm: nhóm 1 gồm $\{2, 7\}$ có tổng 9; nhóm 2 gồm $\{3, 4, \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tổng khối lượng 5 quả táo là $3 + 2 + 7 + 4 + 1 = 17$. Chia thành hai nhóm: nhóm 1 gồm $\{2, 7\}$ có tổng 9; nhóm 2 gồm $\{3, 4, 1\}$ có tổng 8. Độ chênh lệch giữa hai nhóm là $|9 - 8| = 1$. Đây là mức chênh lệch nhỏ nhất.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> p(n);
    long long total_sum = 0;
    for (int i = 0; i < n; ++i) {
        cin >> p[i];
        total_sum += p[i];
    }

    long long min_diff = total_sum;
    int total_masks = (1 << n);

    for (int mask = 0; mask < total_masks; ++mask) {
        long long s1 = 0;
        for (int i = 0; i < n; ++i) {
            if ((mask >> i) & 1) {
                s1 += p[i];
            }
        }
        long long s2 = total_sum - s1;
        min_diff = min(min_diff, abs(s1 - s2));
    }

    cout << min_diff << "\n";
    return 0;
}
```

Bài 10 [cppb_bit_10_dem_cap_tich_and_bang_khong]: Đếm Cặp Có Tích Bit AND Bằng 0

Bối cảnh: Trong phân bố kênh truyền vô tuyến số, hai thiết bị được coi là không xung đột tần số nếu các dải tần biểu diễn dưới dạng mặt nạ bit của chúng không có chung bất kỳ kênh nào, nghĩa là tích bit AND của chúng bằng 0 ($A[i] \& A[j] == 0$). Hãy đếm số lượng cặp thiết bị không xung đột.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho dãy gồm N số nguyên không âm. Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn: $A[i] \& A[j] = 0$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 10^5$).
- Dòng 2: N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i < 2^{16}$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng cặp thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 2 4 8	6

Giải thích: Các số 1 (0001₂), 2 (0010₂), 4 (0100₂), 8 (1000₂) đều có các bit 1 ở vị trí hoàn toàn khác nhau. Do đó tích bit AND giữa hai số bất kỳ đều bằng 0. Số cặp là $C(4, 2) = 6$ cặp.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy gồm N số nguyên không âm. Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn: $A[i] \& A[j] = 0$.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 1 2 4 8 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các số 1 (0001₂), 2 (0010₂), 4 (0100₂), 8 (1000₂) đều có các bit 1 ở vị trí hoàn toàn khác nhau. Do đó tích bit AND ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các số 1 (0001₂), 2 (0010₂), 4 (0100₂), 8 (1000₂) đều có các bit 1 ở vị trí hoàn toàn khác nhau. Do đó tích bit AND giữa hai số bất kỳ đều bằng 0. Số cặp là $C(4, 2) = 6$ cặp.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    const int MAX_VAL = 4096;
    vector<long long> cnt(MAX_VAL, 0);

    for (int i = 0; i < n; ++i) {
        int x;
        cin >> x;
        cnt[x]++;
    }

    long long total_pairs = 0;

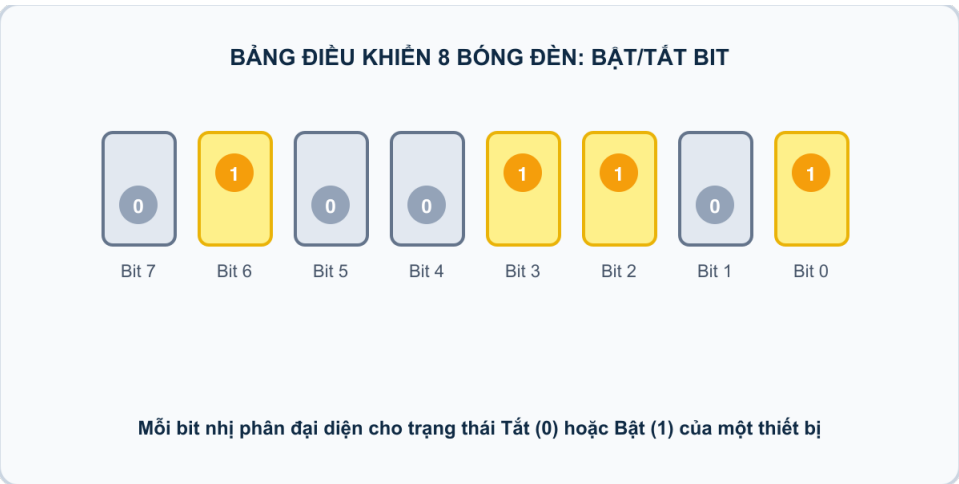
    // Trường hợp u == 0
    total_pairs += cnt[0] * (cnt[0] - 1) / 2;
    for (int v = 1; v < MAX_VAL; ++v) {
        total_pairs += cnt[0] * cnt[v];
    }

    // Trường hợp 1 <= u < v
    for (int u = 1; u < MAX_VAL; ++u) {
        if (cnt[u] == 0) continue;
        for (int v = u + 1; v < MAX_VAL; ++v) {
            if ((u & v) == 0) {
                total_pairs += cnt[u] * cnt[v];
            }
        }
    }

    cout << total_pairs << "\n";
    return 0;
}
```

Bài 11 [cppb_bit_11_cap_xor_lon_nhat]: Tìm Cặp Có XOR Lớn Nhất Trong Mảng

Bối cảnh: Trong thuật toán tạo mã băm ngẫu nhiên, hệ thống cần tìm hai khóa bit số nguyên $A[i]$ và $A[j]$ trong danh sách N khóa để giá trị biểu thức XOR giữa chúng đạt mức độ tương phản nhị phân cao nhất ($A[i] \text{ xor } A[j]$ đạt giá trị lớn nhất).



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho mảng N số nguyên không âm. Hãy tìm giá trị lớn nhất của biểu thức $A[i] \wedge A[j]$ với $1 \leq i < j \leq N$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 10^5$).
- Dòng 2: N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị XOR lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 10 5 25	28

Giải thích: Cặp (5, 25) có $5 \wedge 25 = 00101_2 \wedge 11001_2 = 11100_2 = 28$. Đây là giá trị XOR lớn nhất giữa 2 phần tử bất kỳ trong mảng.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên không âm. Hãy tìm giá trị lớn nhất của biểu thức $A[i] \wedge A[j]$ với $1 \leq i < j \leq N$.
- **Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 3 10 5 25 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Cặp (5, 25) có $5 \wedge 25 = 00101_2 \wedge 11001_2 = 11100_2 = 28$. Đây là giá trị XOR lớn nhất giữa 2 phần tử bất kỳ trong mảng... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 28 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Cặp (5, 25) có $5 \wedge 25 = 00101_2 \wedge 11001_2 = 11100_2 = 28$. Đây là giá trị XOR lớn nhất giữa 2 phần tử bất kỳ trong mảng.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    long long max_xor = 0;
    long long mask = 0;

    for (int bit = 30; bit >= 0; --bit) {
        mask |= (1LL << bit);
        vector<long long> prefixes;
        prefixes.reserve(n);
        for (long long x : a) {
            prefixes.push_back(x & mask);
        }
        sort(prefixes.begin(), prefixes.end());
        prefixes.erase(unique(prefixes.begin(), prefixes.end()), prefixes.end());

        long long candidate = max_xor | (1LL << bit);
        bool found = false;

        for (long long p : prefixes) {
            long long target = p ^ candidate;
            if (binary_search(prefixes.begin(), prefixes.end(), target)) {
                found = true;
                break;
            }
        }

        if (found) {
            max_xor = candidate;
        }
    }

    cout << max_xor << "\n";
    return 0;
}
```

Bài 12 [cppb_bit_12_duyet_tat_ca_submask]: Duyệt Tất Cả Các Tập Con Của Một Mặt Nạ Bit

Bối cảnh: Trong kỹ thuật tối ưu hóa thuật toán quy hoạch động trên tập con SOS DP (Sum Over Subsets), với một mặt nạ bit N cho trước, việc duyệt qua toàn bộ các mặt nạ con (submask) có bit là tập con của N đòi hỏi thuật toán chuyển giao submask hiệu quả.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bóng đèn bật tắt

Nhiệm vụ: Cho số nguyên dương N . Hãy liệt kê tất cả các số nguyên dương s là tập con bit của N (nghĩa là $(s \& N) == s$ với $s > 0$) theo thứ tự giảm dần.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra các số nguyên thỏa mãn cách nhau bởi khoảng trắng theo thứ tự giảm dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	5 4 1

Giải thích: 5 có biểu diễn nhị phân là 101₂. Các tập con bit dương của 101₂ gồm có: 101₂ (5), 100₂ (4) và 001₂ (1). Thứ tự giảm dần là: 5 4 1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy liệt kê tất cả các số nguyên dương s là tập con bit của N (nghĩa là $(s \& N) == s$ với $s > 0$) theo thứ tự giảm dần.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | 5 có biểu diễn nhị phân là 101₂. Các tập con bit dương của 101₂ gồm có: 101₂

(5), 100_2 (4) và 001_2 (1). Thứ tự giảm... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 5 4 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: 5 có biểu diễn nhị phân là 101_2. Các tập con bit dương của 101_2 gồm có: 101_2 (5), 100_2 (4) và 001_2 (1). Thứ tự giảm dần là: 5 4 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

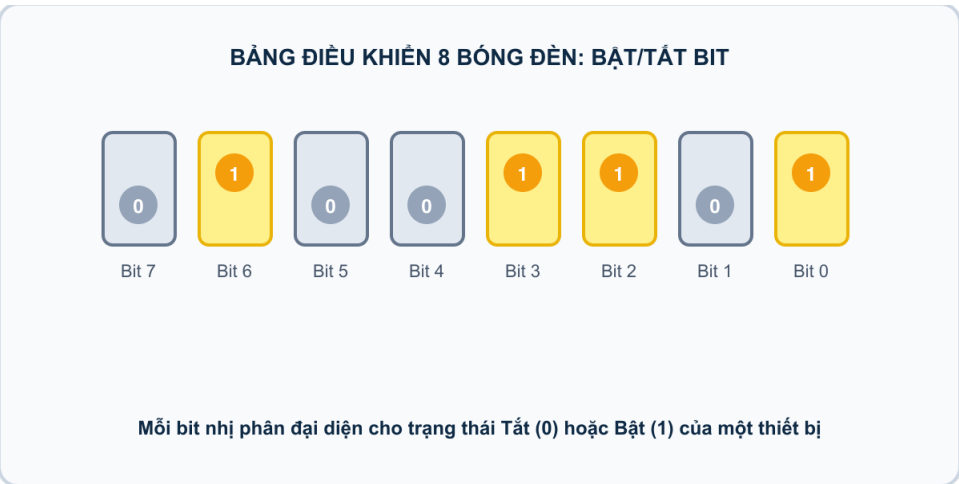
    long long n;
    if (!(cin >> n)) return 0;

    bool first = true;
    for (long long sub = n; sub > 0; sub = (sub - 1) & n) {
        if (!first) cout << " ";
        cout << sub;
        first = false;
    }
    cout << "\n";

    return 0;
}
```

Bài 13 [cppb_bit_13_day_con_tong_xor_bang_k]: Tìm Dãy Con Có Tổng XOR Bằng K

Bối cảnh: Trong một giao thức mã kiểm tra dư thừa nhị phân, máy thu nhận được N khối dữ liệu A1, A2, ..., An. Người nhận cần kiểm tra xem có tồn tại một tập con khác rỗng nào các khối dữ liệu có tích lũy phép toán XOR đúng bằng mã chứng thực K hay không.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho tập hợp gồm N số nguyên dương và số nguyên K . Hãy kiểm tra xem có tồn tại một dãy con khác rỗng có tổng XOR bằng K hay không. In YES nếu có, ngược lại in NO.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq N \leq 20, 0 \leq K \leq 10^9$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 7 1 2 4 8	YES

Giải thích: Chọn tập con gồm 3 phần tử $\{1, 2, 4\}$ có tổng XOR là $1 \wedge 2 \wedge 4 = 7$ đúng bằng K . Kết quả in ra: YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho tập hợp gồm N số nguyên dương và số nguyên K . Hãy kiểm tra xem có tồn tại một dãy con khác rỗng có tổng XOR bằng K hay không. In YES nếu có, ngược lại in NO.
- **Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 7 1 2 4 8 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Chọn tập con gồm 3 phần tử $\{1, 2, 4\}$ có tổng XOR là $1 \wedge 2 \wedge 4 = 7$ đúng bằng K . Kết quả in ra: YES.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chọn tập con gồm 3 phần tử $\{1, 2, 4\}$ có tổng XOR là $1 \wedge 2 \wedge 4 = 7$ đúng bằng K . Kết quả in ra: YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    int count_k = 0;
    int total_masks = (1 << n);

    for (int mask = 1; mask < total_masks; ++mask) {
        long long current_xor = 0;
        for (int i = 0; i < n; ++i) {
            if ((mask >> i) & 1) {
                current_xor ^= a[i];
            }
        }
        if (current_xor == k) {
            count_k++;
        }
    }

    cout << count_k << "\n";
    return 0;
}
```

Bài 14 [cppb_bit_14_toi_uu_gan_viec_n_nguoi]: Tối Ưu Hóa Gán Việc Cho N Người (N ≤ 20)

Bối cảnh: Một giám đốc điều hành dự án cần phân công N công việc cho N kỹ sư phần mềm (mỗi kỹ sư nhận đúng 1 việc). Chi phí để kỹ sư i hoàn thành công việc j được cho bởi ma trận chi phí $C[i][j]$. Hãy tìm phương án phân công sao cho tổng chi phí hoàn thành tất cả N công việc là nhỏ nhất.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho ma trận chi phí C kích thước $N \times N$ ($N \leq 16$). Hãy tìm tổng chi phí phân công nhỏ nhất để giao N việc cho N người bằng quy hoạch động trạng thái bitmask.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 16$).
- N dòng tiếp theo: Mỗi dòng gồm N số nguyên biểu diễn ma trận chi phí $C_{i,j}$ ($0 \leq C_{i,j} \leq 10^6$).

Đầu ra (Output):

- In ra tổng chi phí tối thiểu.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 9 2 7 6 4 3 5 8 1	6

Giải thích: Phương án phân công tối ưu có tổng chi phí nhỏ nhất là 6:

- Kỹ sư 1 làm việc 2 (chi phí $C_{1,2} = 2$).
- Kỹ sư 2 làm việc 3 (chi phí $C_{2,3} = 3$).
- Kỹ sư 3 làm việc 1 (chi phí $C_{3,1} = 1$). Tổng chi phí tối thiểu: $2 + 3 + 1 = 6$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho ma trận chi phí C kích thước $N \times N$ ($N \leq 16$). Hãy tìm tổng chi phí phân công nhỏ nhất để giao N việc cho N người bằng quy hoạch động trạng thái bitmask.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 9 2 7 6 4 3 5 8 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Phương án phân công tối ưu có tổng chi phí nhỏ nhất là 6: - Kỹ sư

1 làm việc 2 (chi phí $C_{1,2} = 2$). - Kỹ sư 2 làm... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả |
Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Phương án phân công tối ưu có tổng chi phí nhỏ nhất là 6:

- Kỹ sư 1 làm việc 2 (chi phí $C_{1,2} = 2$).
- Kỹ sư 2 làm việc 3 (chi phí $C_{2,3} = 3$).
- Kỹ sư 3 làm việc 1 (chi phí $C_{3,1} = 1$). Tổng chi phí tối thiểu: $2 + 3 + 1 = 6$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<vector<long long>> cost(n, vector<long long>(n));

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cin >> cost[i][j];
        }
    }

    int total_masks = (1 << n);
    const long long INF = 1e18;
    vector<long long> dp(total_masks, INF);
    dp[0] = 0;

    for (int mask = 0; mask < total_masks; ++mask) {
        if (dp[mask] == INF) continue;
        int task_idx = __builtin_popcount(mask);
        if (task_idx >= n) continue;

        for (int j = 0; j < n; ++j) {
            if (!(mask >> j) & 1) {
                int next_mask = mask | (1 << j);
                dp[next_mask] = min(dp[next_mask], dp[mask] + cost[task_idx][j]);
            }
        }
    }

    cout << dp[total_masks - 1] << "\n";
}
```

```
return 0;
}
```

Bài 15 [cppb_bit_15_cap_tong_luy_thua_hai]: Đếm Số Cặp Có Tổng Bằng Lũy Thừa Của 2

Bối cảnh: Trong một hệ thống tính toán lượng tử, hai trạng thái qubit có năng lượng $A[i]$ và $A[j]$ chỉ có thể tương tác cộng hưởng nếu tổng năng lượng của chúng đúng bằng một lũy thừa của 2 (nghĩa là $A[i] + A[j] = 2^k$ với $k \geq 0$). Hãy đếm số lượng cặp trạng thái có thể tương tác cộng hưởng.



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho dãy gồm N số nguyên dương. Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ sao cho $A[i] + A[j]$ là một lũy thừa của 2.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 10^5$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra số lượng cặp thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 1 3 3	4

Giải thích: Các cặp chỉ số (i, j) có tổng là lũy thừa của 2 gồm 3 cặp:

- $(1, 3)$: tổng $1 + 3 = 4 = 2^2$.
- $(1, 7)$: tổng $1 + 7 = 8 = 2^3$.
- $(1, 15)$: tổng $1 + 15 = 16 = 2^4$. Vậy có đúng 3 cặp thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy gồm N số nguyên dương. Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ sao cho $A[i] + A[j]$ là một lũy thừa của 2.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**

- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.
- Sử dụng các toán tử bitwise $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$ để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 1 1 3 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các cặp chỉ số (i, j) có tổng là lũy thừa của 2 gồm 3 cặp: - (1,3): tổng $1 + 3 = 4 = 2^2$. - (1,7): tổng $1 + 7 = 8 = 2^3$. - (1,15): tổng $1 + 15 = 16 = 2^4$. | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các cặp chỉ số (i, j) có tổng là lũy thừa của 2 gồm 3 cặp:

- (1,3): tổng $1 + 3 = 4 = 2^2$.
- (1,7): tổng $1 + 7 = 8 = 2^3$.
- (1,15): tổng $1 + 15 = 16 = 2^4$. Vậy có đúng 3 cặp thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }
    sort(a.begin(), a.end());

    long long total_pairs = 0;

    for (int i = 0; i < n; ++i) {
        for (int k = 1; k <= 30; ++k) {
            long long target = (1LL << k) - a[i];
            if (target <= 0) continue;

            auto it1 = lower_bound(a.begin() + i + 1, a.end(), target);
            auto it2 = upper_bound(a.begin() + i + 1, a.end(), target);
            total_pairs += (it2 - it1);
        }
    }
}
```

```

    }

    cout << total_pairs << "\n";
    return 0;
}

```

Bài 16 [cppb_bit_16_tap_doc_lap_bit_lon_nhat]: Tập Hợp Độc Lập Về Bit Lớn Nhất

Bối cảnh: Một tập hợp N từ mã nhị phân được lưu trữ trong bộ nhớ ROM. Hai từ mã được coi là hoàn toàn độc lập nếu chúng không có bất kỳ bit 1 nào trùng nhau (tức $A[i] \& A[j] == 0$). Hãy tìm kích thước lớn nhất của một tập con các từ mã mà giữa hai từ mã bất kỳ trong tập con đều không xung đột bit (đôi một có tích bit AND bằng 0).



Hình minh họa: Biểu diễn trạng thái nhị phân qua các bit công tắc bật tắt

Nhiệm vụ: Cho tập hợp N số nguyên dương ($N \leq 22$). Hãy tìm kích thước lớn nhất của một tập con mà hai phần tử bất kỳ trong tập con đều có tích bit AND bằng 0.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 22$).
- Dòng 2: N số nguyên dương $A_0, A_1, \dots, A_{N-1}$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra số lượng phần tử lớn nhất của tập con độc lập về bit.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 2 4 3	3

Giải thích: Tập con $\{1, 2, 4\}$ gồm 3 số có các bit 1 độc lập từng đôi một: ($1 \& 2 = 0$, $1 \& 4 = 0$, $2 \& 4 = 0$). Số lượng phần tử lớn nhất là 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho tập hợp N số nguyên dương ($N \leq 22$). Hãy tìm kích thước lớn nhất của một tập con mà hai phần tử bất kỳ trong tập con đều có tích bit AND bằng 0.
- Phương pháp tiếp cận — Phép toán BIT & Bitmask:**
- Biểu diễn tập hợp hoặc trạng thái bật/tắt bằng các bit của số nguyên 64-bit.

- Sử dụng các toán tử bitwise &, |, ^, ~, <<, >> để thao tác đồng thời trong $\mathcal{O}(1)$ chu kỳ máy.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 1 2 4 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Tập con {1, 2, 4} gồm 3 số có các bit 1 độc lập từng đôi một: (1&2=0, 1&4=0, 2&4=0). Số lượng phần tử lớn nhất là 3.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tập con {1, 2, 4} gồm 3 số có các bit 1 độc lập từng đôi một: (1&2=0, 1&4=0, 2&4=0). Số lượng phần tử lớn nhất là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    int max_size = 0;
    int total_masks = (1 << n);

    for (int mask = 0; mask < total_masks; ++mask) {
        long long used_bits = 0;
        bool valid = true;
        int current_count = 0;

        for (int i = 0; i < n; ++i) {
            if ((mask >> i) & 1) {
                if (used_bits & a[i]) {
                    valid = false;
                    break;
                }
                used_bits |= a[i];
                current_count++;
            }
        }

        if (valid) max_size = max(max_size, current_count);
    }
}
```

```
        if (valid) {  
            max_size = max(max_size, current_count);  
        }  
  
    cout << max_size << "\n";  
    return 0;  
}
```



CHƯƠNG 04: SỐ HỌC & ĐẠI SỐ MODULAR

Bài 10: Lý thuyết số & số nguyên tố

Bài 01 [cppb_nt_01_uoc_chung_boi_chung]: Ước Chung Lớn Nhất & Bội Chung Nhỏ Nhất

Bối cảnh: Trong kỹ thuật thiết kế bánh răng truyền động cơ khí, hai bánh răng A và B có số răng lần lượt là A và B. Để hai bánh răng ăn khớp nhịp nhàng theo chu kỳ mà không làm mòn lệch răng, các kỹ sư cần xác định ước chung lớn nhất (để thiết kế bước răng chung) và bội chung nhỏ nhất (để xác định chu kỳ quay lặp lại trạng thái ban đầu).



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho hai số nguyên dương A và B. Hãy tìm ước chung lớn nhất $\text{gcd}(A, B)$ và bội chung nhỏ nhất $\text{lcm}(A, B)$.

Đầu vào (Input):

- Một dòng duy nhất chứa 2 số nguyên dương A và B ($1 \leq A, B \leq 10^{12}$).

Đầu ra (Output):

- In ra 2 số nguyên cách nhau bởi khoảng trắng: $\text{gcd}(A, B)$ và $\text{lcm}(A, B)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12 18	6 36

Giải thích: $\text{gcd}(12, 18) = 6$ và $\text{lcm}(12, 18) = (12 * 18) / 6 = 36$. Kết quả in ra: 6 36.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai số nguyên dương A và B. Hãy tìm ước chung lớn nhất $\text{gcd}(A, B)$ và bội chung nhỏ nhất $\text{lcm}(A, B)$.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.

- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 12 18 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $\gcd(12, 18) = 6$ và $\text{lcm}(12, 18) = (12 * 18) / 6 = 36$. Kết quả in ra: 6 36.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 36 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $\gcd(12, 18) = 6$ và $\text{lcm}(12, 18) = (12 * 18) / 6 = 36$. Kết quả in ra: 6 36.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long getGcd(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    long long g = getGcd(a, b);
    long long l = (a / g) * b;

    cout << g << " " << l << "\n";
    return 0;
}
```

Bài 02 [cppb_nt_02_kiem_tra_so_nguyen_to]: Kiểm Tra Số Nguyên Tố Cơ Bản

Bối cảnh: Trong hệ thống xác thực bảo mật tài khoản ngân hàng, mã khóa OTP được coi là đạt chuẩn an toàn nếu nó là một số nguyên tố (chỉ có đúng 2 ước số dương là 1 và chính nó). Hệ thống cần xác định nhanh xem số nguyên dương N có phải là số nguyên tố hay không.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho một số nguyên dương N . Hãy kiểm tra xem N có phải là số nguyên tố hay không. In YES nếu đúng, ngược lại in NO.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^{12}$).

Đầu ra (Output):

- In ra YES nếu N là số nguyên tố, ngược lại in NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
29	YES

Giải thích: Số 29 chỉ chia hết cho 1 và 29, do đó 29 là số nguyên tố -> in YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho một số nguyên dương N . Hãy kiểm tra xem N có phải là số nguyên tố hay không. In YES nếu đúng, ngược lại in NO.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 29 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Số 29 chỉ chia hết cho 1 và 29, do đó 29 là số nguyên tố -> in YES.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số 29 chỉ chia hết cho 1 và 29, do đó 29 là số nguyên tố -> in YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool isPrime(long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    cout << (isPrime(n) ? "YES" : "NO") << "\n";
    return 0;
}
```

Bài 03 [cppb_nt_03_phan_tich_thua_so_nguyen_to]: Phân Tích Thừa Số Nguyên Tố

Bối cảnh: Một thuật toán nén dữ liệu số học cần phân rã một mã định danh nguyên dương N thành tích các thừa số nguyên tố lũy thừa để tối ưu không gian lưu trữ dạng cơ số tối giản.

SÀNG LỌC SỐ NGUYÊN TỐ ERATOSTHENES

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

Giữ lại các số nguyên tố màu xanh, gạch bỏ các hợp số là bội của chúng

Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy phân tích N thành tích các thừa số nguyên tố theo dạng $p_1^{e_1} * p_2^{e_2} * \dots$ với $p_1 < p_2 < \dots$

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($2 \leq N \leq 10^{12}$).

Đầu ra (Output):

- In ra chuỗi phân tích thừa số nguyên tố theo định dạng p^e nối với nhau bởi dấu $*$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
60	$2^2 * 3^1 * 5^1$

Giải thích: $60 = 4 * 3 * 5 = 2^2 * 3^1 * 5^1$. Kết quả in ra: $2^2 * 3^1 * 5^1$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy phân tích N thành tích các thừa số nguyên tố theo dạng $p_1^{e_1} * p_2^{e_2} * \dots$ với $p_1 < p_2 < \dots$
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $O(N \log \log N)$ hoặc kiểm tra căn bậc hai $O(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 60 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $60 = 4 * 3 * 5 = 2^2 * 3^1 * 5^1$. Kết quả in ra: $2^2 * 3^1 * 5^1 \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: $2^2 * 3^1 * 5^1$ | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $60 = 4 * 3 * 5 = 2^2 * 3^1 * 5^1$. Kết quả in ra: $2^2 * 3^1 * 5^1$.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

long long n;
if (!(cin >> n)) return 0;

vector<pair<long long, int>> factors;
for (long long i = 2; i * i <= n; ++i) {
    if (n % i == 0) {
        int cnt = 0;
        while (n % i == 0) {
            cnt++;
            n /= i;
        }
        factors.push_back({i, cnt});
    }
}
if (n > 1) {
    factors.push_back({n, 1});
}

for (int i = 0; i < (int)factors.size(); ++i) {
    cout << factors[i].first << "^" << factors[i].second;
    if (i + 1 < (int)factors.size()) cout << " * ";
}
cout << "\n";
return 0;
}

```

Bài 04 [cppb_nt_04_dem_uoc_va_tong_uoc]: Đếm Số Lượng & Tính Tổng Các Ước Số

Bối cảnh: Trong nghiên cứu lý thuyết số giải thuật, hàm số lượng ước $d(N)$ và hàm tổng các ước $\sigma(N)$ đóng vai trò then chốt trong việc phân loại số phong phú, số hoàn hảo và số khuyết thiếu. Hãy tính giá trị của hai hàm này cho số nguyên dương N .

SÀNG LỌC SỐ NGUYÊN TỐ ERATOSTHENES

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

Giữ lại các số nguyên tố màu xanh, gạch bỏ các hợp số là bội của chúng

Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy tính số lượng ước số nguyên dương $d(N)$ và tổng tất cả các ước số nguyên dương của N .

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^{12}$).

Đầu ra (Output):

- In ra 2 số nguyên cách nhau bởi khoảng trắng: số lượng ước và tổng các ước.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12	6 28

Giải thích: Các ước số của 12 là {1, 2, 3, 4, 6, 12}, tổng cộng có 6 ước. Tổng các ước là $1 + 2 + 3 + 4 + 6 + 12 = 28$. Kết quả in ra: 6 28.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N . Hãy tính số lượng ước số nguyên dương $d(N)$ và tổng tất cả các ước số nguyên dương của N .
- **Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 12 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các ước số của 12 là {1, 2, 3, 4, 6, 12}, tổng cộng có 6 ước. Tổng các ước là $1 + 2 + 3 + 4 + 6 + 12 = 28$. Kết quả in ra... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 28 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các ước số của 12 là {1, 2, 3, 4, 6, 12}, tổng cộng có 6 ước. Tổng các ước là $1 + 2 + 3 + 4 + 6 + 12 = 28$. Kết quả in ra: 6 28.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    long long count_div = 1;
    long long sum_div = 1;

    for (long long i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
```

```

int a = 0;
long long p_pow = 1;
long long cur_sum = 1;
while (n % i == 0) {
    a++;
    n /= i;
    p_pow *= i;
    cur_sum += p_pow;
}
count_div *= (a + 1);
sum_div *= cur_sum;
}
}
if (n > 1) {
    count_div *= 2;
    sum_div *= (1 + n);
}

cout << count_div << " " << sum_div << "\n";
return 0;
}
    
```

Bài 05 [cppb_nt_05_kiem_tra_so_chinh_phuong]: Kiểm Tra Số Chính Phương

Bối cảnh: Một người thợ lát sàn gạch vuông cần kiểm tra xem diện tích sàn gồm N viên gạch nhỏ có thể xếp thành một hình vuông lớn hoàn hảo (cạnh nguyên k) hay không.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy kiểm tra xem N có phải là số chính phương ($N = k^2$ với k nguyên dương) hay không. In YES nếu đúng, ngược lại in NO.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
49	YES

Giải thích: $49 = 7^2$ là bình phương của số nguyên 7, do đó 49 là số chính phương $\rightarrow$ in YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N . Hãy kiểm tra xem N có phải là số chính phương ($N = k^2$ với k nguyên dương) hay không. In YES nếu đúng, ngược lại in NO.
- **Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—|—| 1 | Nạp dữ liệu vào mảng/biến | Input: 49 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | $49 = 7^2$ là bình phương của số nguyên 7, do đó 49 là số chính phương $\rightarrow$ in YES.... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $49 = 7^2$ là bình phương của số nguyên 7, do đó 49 là số chính phương $\rightarrow$ in YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    unsigned long long n;
    if (!(cin >> n)) return 0;

    unsigned long long r = sqrt((double)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;

    if (r * r == n) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}
```

Bài 06 [cppb_nt_06_sang_nguyen_to_eratosthenes]: Sàng Nguyên Tố Eratosthenes

Bối cảnh: Để phục vụ các bài toán mã hóa khóa công khai RSA quy mô nhỏ, máy chủ cần khởi tạo sẵn bảng tra cứu tất cả các số nguyên tố từ 2 đến N bằng thuật toán Sàng Eratosthenes kinh điển.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N. Hãy in ra tất cả các số nguyên tố không vượt quá N theo thứ tự tăng dần.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($2 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra các số nguyên tố trên một dòng cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
15	2 3 5 7 11 13

Giải thích: Các số nguyên tố không vượt quá 15 là: 2, 3, 5, 7, 11, 13.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N. Hãy in ra tất cả các số nguyên tố không vượt quá N theo thứ tự tăng dần.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 15 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các số nguyên tố không vượt quá 15 là: 2, 3, 5, 7, 11, 13.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 3 5 7 11 13 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các số nguyên tố không vượt quá 15 là: 2, 3, 5, 7, 11, 13.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; 1LL * i * i <= n; ++i) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                is_prime[j] = false;
            }
        }
    }

    bool first = true;
    for (int i = 2; i <= n; ++i) {
        if (is_prime[i]) {
            if (!first) cout << " ";
            cout << i;
            first = false;
        }
    }
    cout << "\n";
    return 0;
}
```

Bài 07 [cppb_nt_07_dem_so_nguyen_to_doan_l_r]: Đếm Số Nguyên Tố Trong Đoạn [L, R]

Bối cảnh: Một viện nghiên cứu toán học cần thống kê mật độ phân bố các số nguyên tố trên các khoảng số liệu thực nghiệm [L, R] qua Q truy vấn liên tục.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho Q truy vấn, mỗi truy vấn gồm 2 số nguyên L, R ($1 \leq L \leq R \leq 10^6$). Hãy đếm số lượng số nguyên tố nằm trong đoạn $[L, R]$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương Q ($1 \leq Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng gồm 2 số nguyên L và R ($1 \leq L \leq R \leq 10^6$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là số lượng số nguyên tố tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 1 10 10 20 20 30	4 4 2

Giải thích: - Đoạn $[1, 10]$: có 4 số nguyên tố $\{2, 3, 5, 7\}$. - Đoạn $[10, 20]$: có 4 số nguyên tố $\{11, 13, 17, 19\}$. - Đoạn $[20, 30]$: có 2 số nguyên tố $\{23, 29\}$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho Q truy vấn, mỗi truy vấn gồm 2 số nguyên L, R ($1 \leq L \leq R \leq 10^6$). Hãy đếm số lượng số nguyên tố nằm trong đoạn $[L, R]$.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 1 10 10 20 20 30 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - Đoạn $[1, 10]$: có 4 số nguyên tố $\{2, 3, 5, 7\}$. - Đoạn $[10, 20]$: có 4 số nguyên tố $\{11, 13, 17, 19\}$. - Đoạn $[20, 30]$: có... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 4 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Đoạn $[1, 10]$: có 4 số nguyên tố $\{2, 3, 5, 7\}$.

- Đoạn [10, 20]: có 4 số nguyên tố {11, 13, 17, 19}.
- Đoạn [20, 30]: có 2 số nguyên tố {23, 29}.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;
vector<bool> is_prime(MAXN + 1, true);
vector<int> pref(MAXN + 1, 0);

void sieve() {
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; 1LL * i * i <= MAXN; ++i) {
        if (is_prime[i]) {
            for (int j = i * i; j <= MAXN; j += i) {
                is_prime[j] = false;
            }
        }
    }
    for (int i = 1; i <= MAXN; ++i) {
        pref[i] = pref[i - 1] + (is_prime[i] ? 1 : 0);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    sieve();

    int q;
    if (!(cin >> q)) return 0;

    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << pref[r] - pref[l - 1] << "\n";
    }
    return 0;
}
```

Bài 08 [cppb_nt_08_sang_uoc_nguyen_to_nho_nhat_spf]: Sàng Ước Nguyên Tố Nhỏ Nhất (SPF)

Bối cảnh: Trong phân tích nhân tử nhanh $O(\log N)$ cho hàng triệu số nguyên, thuật toán Sàng ước nguyên tố nhỏ nhất (Smallest Prime Factor - SPF) là công cụ tối thượng để phân tích thừa số nguyên tố cực nhanh.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho Q truy vấn, mỗi truy vấn chứa một số nguyên N ($2 \leq N \leq 10^6$). Hãy in ra ước số nguyên tố nhỏ nhất của N .

Đầu vào (Input):

- Dòng 1: Số nguyên dương Q ($1 \leq Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng chứa một số nguyên N ($2 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là ước số nguyên tố nhỏ nhất của N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 15 7 20	3 7 2

Giải thích: - $\text{SPF}(15) = 3$ (vì 15 chia hết cho số nguyên tố nhỏ nhất là 3). - $\text{SPF}(7) = 7$ (vì 7 là số nguyên tố). - $\text{SPF}(20) = 2$ (vì 20 chia hết cho 2).

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho Q truy vấn, mỗi truy vấn chứa một số nguyên N ($2 \leq N \leq 10^6$). Hãy in ra ước số nguyên tố nhỏ nhất của N .
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $O(N \log \log N)$ hoặc kiểm tra căn bậc hai $O(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 15 7 20 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - SPF(15) = 3 (vì 15 chia hết cho số nguyên tố nhỏ nhất là 3). - SPF(7) = 7 (vì 7 là số nguyên tố). - SPF(20) = 2 (vì 20... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 7 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - SPF(15) = 3 (vì 15 chia hết cho số nguyên tố nhỏ nhất là 3).

- SPF(7) = 7 (vì 7 là số nguyên tố).
- SPF(20) = 2 (vì 20 chia hết cho 2).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;
vector<int> spf(MAXN + 1);

void sieveSPF() {
    for (int i = 1; i <= MAXN; ++i) spf[i] = i;
    for (int i = 2; 1LL * i * i <= MAXN; ++i) {
        if (spf[i] == i) {
            for (int j = i * i; j <= MAXN; j += i) {
                if (spf[j] == j) spf[j] = i;
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

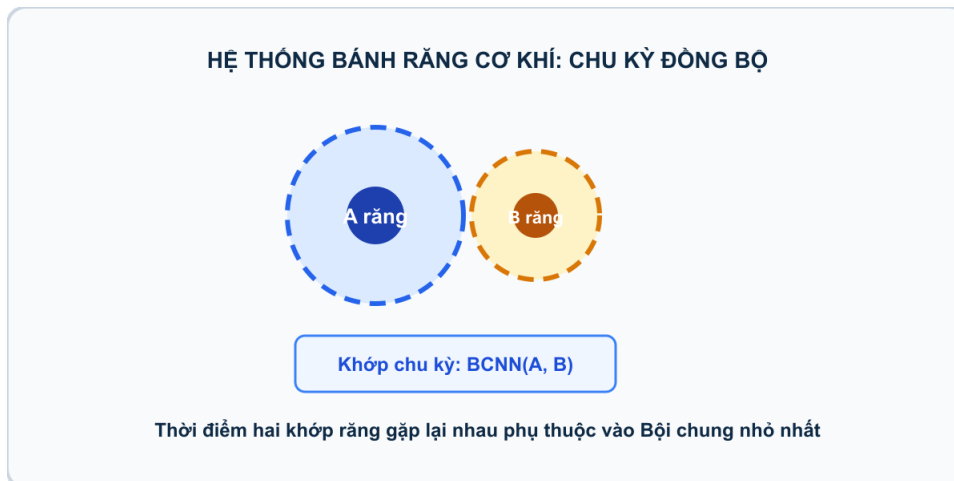
    sieveSPF();

    int q;
    if (!(cin >> q)) return 0;

    for (int i = 0; i < q; ++i) {
        int n;
        cin >> n;
        cout << spf[n] << (i + 1 == q ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 09 [cppb_nt_09_sang_phan_doan_segmented_sieve]: Sàng Phân Đoạn (Segmented Sieve)

Bối cảnh: Khi cần tìm số nguyên tố trong một đoạn $[L, R]$ với giá trị L, R có thể rất lớn (lên tới 10^{12}) nhưng độ dài đoạn $R - L$ không quá 10^6 , việc lập mảng thông thường bị tràn bộ nhớ. Kỹ thuật Sàng phân đoạn (Segmented Sieve) cho phép giải quyết bài toán này trong giới hạn bộ nhớ chặt chẽ.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho hai số nguyên L, R ($1 \leq L \leq R \leq 10^{12}$, $R - L \leq 10^6$). Hãy đếm số lượng số nguyên tố trong đoạn $[L, R]$.

Đầu vào (Input):

- Một dòng duy nhất chứa 2 số nguyên L và R .

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng số nguyên tố trong đoạn $[L, R]$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10 20	4

Giải thích: Các số nguyên tố trong đoạn $[10, 20]$ gồm $\{11, 13, 17, 19\}$, tổng cộng có 4 số.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai số nguyên L, R ($1 \leq L \leq R \leq 10^{12}$, $R - L \leq 10^6$). Hãy đếm số lượng số nguyên tố trong đoạn $[L, R]$.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 10 20 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các số nguyên tố trong đoạn [10, 20] gồm {11, 13, 17, 19}, tổng cộng có 4 số.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các số nguyên tố trong đoạn [10, 20] gồm {11, 13, 17, 19}, tổng cộng có 4 số.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long l, r;
    if (!(cin >> l >> r)) return 0;

    int lim = sqrt(r);
    vector<bool> is_prime(lim + 1, true);
    vector<int> primes;
    for (int i = 2; i <= lim; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (int j = i * 2; j <= lim; j += i) is_prime[j] = false;
        }
    }

    vector<bool> is_prime_range(r - l + 1, true);
    for (int p : primes) {
        long long start = max(1LL * p * p, ((l + p - 1) / p) * p);
        for (long long j = start; j <= r; j += p) {
            is_prime_range[j - l] = false;
        }
    }

    if (l == 1) is_prime_range[0] = false;

    int count_primes = 0;
    for (int i = 0; i <= r - l; ++i) {
        if (is_prime_range[i]) count_primes++;
    }

    cout << count_primes << "\n";
}
```

```
return 0;
}
```

Bài 10 [cppb_nt_10_cap_so_nguyen_to_sinh_doi]: Cặp Số Nguyên Tố Sinh Đôi (Twin Primes)

Bối cảnh: Trong lý thuyết số giải thuật, giả thuyết về số nguyên tố sinh đôi là một bài toán nổi tiếng. Một cặp số $(p, p + 2)$ được gọi là cặp số nguyên tố sinh đôi nếu cả p và $p + 2$ đều là số nguyên tố. Hãy đếm số lượng cặp số nguyên tố sinh đôi không vượt quá N .

SÀNG LỌC SỐ NGUYÊN TỐ ERATOSTHENES

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

Giữ lại các số nguyên tố màu xanh, gạch bỏ các hợp số là bội của chúng

Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy đếm số lượng cặp số nguyên tố sinh đôi $(p, p + 2)$ sao cho $p + 2 \leq N$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra số lượng cặp số nguyên tố sinh đôi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
20	4

Giải thích: Các cặp số nguyên tố sinh đôi ≤ 20 là: $(3, 5)$, $(5, 7)$, $(11, 13)$, $(17, 19)$. Tổng cộng có 4 cặp.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy đếm số lượng cặp số nguyên tố sinh đôi $(p, p + 2)$ sao cho $p + 2 \leq N$.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 20 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các cặp nguyên tố sinh đôi ≤ 20 là: (3, 5), (5, 7), (11, 13), (17, 19). Tổng cộng có 4 cặp... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các cặp nguyên tố sinh đôi ≤ 20 là: (3, 5), (5, 7), (11, 13), (17, 19). Tổng cộng có 4 cặp.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n < 5) {
        cout << 0 << "\n";
        return 0;
    }

    vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; 1LL * i * i <= n; ++i) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i) is_prime[j] = false;
        }
    }

    int count_twin = 0;
    for (int p = 3; p + 2 <= n; p += 2) {
        if (is_prime[p] && is_prime[p + 2]) count_twin++;
    }

    cout << count_twin << "\n";
    return 0;
}
```

Bài 11 [cppb_nt_11_so_hoan_hao]: Kiểm Tra Số Hoàn Hảo

Bối cảnh: Từ thời Hy Lạp cổ đại, các nhà toán học đã tôn vinh các ‘số hoàn hảo’ — những số nguyên dương có tổng tất cả các ước số thực sự (không kể chính nó) bằng đúng con số đó (ví dụ $6 = 1 + 2 + 3$). Hãy kiểm tra xem số N có phải là số hoàn hảo hay không.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy kiểm tra xem N có phải là số hoàn hảo hay không. In YES nếu đúng, ngược lại in NO.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
28	YES

Giải thích: Các ước số thực sự của 28 là $\{1, 2, 4, 7, 14\}$. Tổng của chúng là $1 + 2 + 4 + 7 + 14 = 28$. Vì vậy 28 là số hoàn hảo $\rightarrow$ in YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy kiểm tra xem N có phải là số hoàn hảo hay không. In YES nếu đúng, ngược lại in NO.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 28 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các ước số thực sự của 28 là {1, 2, 4, 7, 14}. Tổng của chúng là $1 + 2 + 4 + 7 + 14 = 28$. Vì vậy 28 là số hoàn hảo $\rightarrow$ in... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các ước số thực sự của 28 là {1, 2, 4, 7, 14}. Tổng của chúng là $1 + 2 + 4 + 7 + 14 = 28$. Vì vậy 28 là số hoàn hảo $\rightarrow$ in YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool isPrime(long long p) {
    if (p < 2) return false;
    for (long long i = 2; i * i <= p; ++i) {
        if (p % i == 0) return false;
    }
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    unsigned long long n;
    if (!(cin >> n)) return 0;

    // Theo Euclid-Euler, số hoàn hảo chẵn có dạng  $2^{p-1} * (2^p - 1)$  với  $2^p - 1$  là số nguyên tố
    vector<unsigned long long> perfect_nums;
    int primes[] = {2, 3, 5, 7, 13, 17, 19, 31};
    for (int p : primes) {
        unsigned long long mersenne = (1ULL << p) - 1;
        if (isPrime(mersenne)) {
            unsigned long long perf = (1ULL << (p - 1)) * mersenne;
            perfect_nums.push_back(perf);
        }
    }

    for (auto v : perfect_nums) {
        if (v == n) {
            cout << "YES\n";
            return 0;
        }
    }
}
```

```
cout << "NO\n";
return 0;
}
```

Bài 12 [cppb_nt_12_so_co_dung_ba_uoc]: Số Có Đúng 3 Ước Số

Bối cảnh: Một mã định danh số học đặc biệt có tính chất độc đáo: nó chỉ có đúng 3 ước số nguyên dương. Theo định lý toán học, một số có đúng 3 ước số khi và chỉ khi nó là bình phương của một số nguyên tố (dạng p^2 với p nguyên tố). Hãy đếm xem có bao nhiêu số như vậy không vượt quá N .



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy đếm số lượng số nguyên dương $\leq N$ có đúng 3 ước số nguyên dương.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^{12}$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng số thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
50	4

Giải thích: Các số có đúng 3 ước số ≤ 50 là bình phương các số nguyên tố: $2^2=4$, $3^2=9$, $5^2=25$, $7^2=49$. Tổng cộng có 4 số.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy đếm số lượng số nguyên dương $\leq N$ có đúng 3 ước số nguyên dương.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 50 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các số có đúng 3 ước số ≤ 50 là bình phương các số nguyên tố: $2^2=4$, $3^2=9$, $5^2=25$, $7^2=49$. Tổng cộng có 4 số.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các số có đúng 3 ước số ≤ 50 là bình phương các số nguyên tố: $2^2=4$, $3^2=9$, $5^2=25$, $7^2=49$. Tổng cộng có 4 số.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    int lim = sqrt(n);
    vector<bool> is_prime(lim + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; 1LL * i * i <= lim; ++i) {
        if (is_prime[i]) {
            for (int j = i * i; j <= lim; j += i) is_prime[j] = false;
        }
    }

    int count_3div = 0;
    for (int i = 2; i <= lim; ++i) {
        if (is_prime[i] && 1LL * i * i <= n) count_3div++;
    }

    cout << count_3div << "\n";
    return 0;
}
```

Bài 13 [cppb_nt_13_so_gan_nguyen_to_almost_prime]: Số Gần Nguyên Tố (Almost Prime)

Bối cảnh: Trong lý thuyết sàng lọc số học, một số nguyên dương được gọi là ‘gần nguyên tố’ (2-Almost Prime) nếu nó có đúng 2 ước số nguyên tố phân biệt (ví dụ $6 = 2 \cdot 3$, $18 = 2 \cdot 3^2$). Hãy đếm xem trong khoảng từ 1 đến N có bao nhiêu số gần nguyên tố.



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy đếm số lượng số nguyên trong đoạn $[1, N]$ có đúng 2 ước số nguyên tố phân biệt.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 3000$).

Đầu ra (Output):

- In ra số lượng số gần nguyên tố.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10	2

Giải thích: Trong đoạn $[1, 10]$, các số có đúng 2 ước nguyên tố phân biệt là: 6 (ước 2, 3) và 10 (ước 2, 5). Tổng cộng có 2 số.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy đếm số lượng số nguyên trong đoạn $[1, N]$ có đúng 2 ước số nguyên tố phân biệt.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Trong đoạn $[1, 10]$, các số có đúng 2 ước nguyên tố phân biệt là: 6 (ước 2, 3) và 10 (ước 2, 5). Tổng cộng có 2 số.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Trong đoạn $[1, 10]$, các số có đúng 2 ước nguyên tố phân biệt là: 6 (ước 2, 3) và 10 (ước 2, 5). Tổng cộng có 2 số.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<int> prime_count(n + 1, 0);
    for (int i = 2; i <= n; ++i) {
        if (prime_count[i] == 0) { // i là số nguyên tố
            for (int j = i; j <= n; j += i) {
                prime_count[j]++;
            }
        }
    }

    int ans = 0;
    for (int i = 1; i <= n; ++i) {
        if (prime_count[i] == 2) ans++;
    }

    cout << ans << "\n";
    return 0;
}
```

Bài 14 [cppb_nt_14_phan_tich_giai_thua_legendre]: Phân Tích Giai Thừa Ra Thừa Số (Định Lý Legendre)

Bối cảnh: Định lý Legendre cung cấp công thức tính chính xác số mũ của thừa số nguyên tố P trong khai triển giai thừa $N!$ mà không cần tính trực tiếp giá trị $N!$ (vốn khổng lồ và tràn số). Kỹ sư cần tính số mũ lớn nhất K sao cho $N!$ chia hết cho P^K .



Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N và số nguyên tố P . Hãy tìm số mũ lớn nhất K sao cho $N!$ chia hết cho P^K bằng công thức Legendre.

Đầu vào (Input):

- Một dòng chứa 2 số nguyên N và P ($1 \leq N \leq 10^9, 2 \leq P \leq 10^9, P$ là số nguyên tố).

Đầu ra (Output):

- In ra số mũ K lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10 3	4

Giải thích: Áp dụng công thức Legendre: $\text{floor}(10/3) + \text{floor}(10/9) = 3 + 1 = 4$. Do đó $10!$ chia hết cho 3^4 và $K = 4$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N và số nguyên tố P . Hãy tìm số mũ lớn nhất K sao cho $N!$ chia hết cho P^K bằng công thức Legendre.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 10 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Áp dụng công thức Legendre: $\text{floor}(10/3) + \text{floor}(10/9) = 3 + 1 = 4$. Do đó $10!$ chia hết cho 3^4 và $K = 4$. | | 3 | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Áp dụng công thức Legendre: $\text{floor}(10/3) + \text{floor}(10/9) = 3 + 1 = 4$. Do đó $10!$ chia hết cho 3^4 và $K = 4$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n, p;
    if (!(cin >> n >> p)) return 0;

    long long k = 0;
    while (n > 0) {
        k += (n / p);
        n /= p;
    }

    cout << k << "\n";
    return 0;
}
```

Bài 15 [cppb_nt_15_dem_so_khong_tan_cung_giai_thua]: Đếm Số Lượng Số Không Tận Cùng Của N!

Bối cảnh: Trong tính toán số lớn, số lượng chữ số 0 liên tiếp ở tận cùng của giai thừa $N!$ được quyết định bởi số lần xuất hiện của thừa số $10 = 2 \cdot 5$. Do số lượng thừa số 2 luôn nhiều hơn thừa số 5, ta chỉ cần đếm số mũ của thừa số 5 trong khai triển $N!$.

SÀNG LỌC SỐ NGUYÊN TỐ ERATOSTHENES

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

Giữ lại các số nguyên tố màu xanh, gạch bỏ các hợp số là bội của chúng

Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy đếm số lượng chữ số 0 liên tiếp tận cùng trong biểu diễn thập phân của $N!$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^9$).

Đầu ra (Output):

- In ra số lượng chữ số 0 tận cùng của $N!$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
25	6

Giải thích: Số lượng thừa số 5 trong $25!$ là: $\text{floor}(25/5) + \text{floor}(25/25) = 5 + 1 = 6$. Vì vậy $25!$ có đúng 6 chữ số 0 tận cùng.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy đếm số lượng chữ số 0 liên tiếp tận cùng trong biểu diễn thập phân của $N!$.
- Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| 1 | Nạp dữ liệu vào mảng/biến | Input: 25 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Số lượng thừa số 5 trong $25!$ là: $\text{floor}(25/5) + \text{floor}(25/25) = 5 + 1 = 6$. Vì vậy $25!$ có đúng 6 chữ số 0 tận cùng.... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số lượng thừa số 5 trong $25!$ là: $\text{floor}(25/5) + \text{floor}(25/25) = 5 + 1 = 6$. Vì vậy $25!$ có đúng 6 chữ số 0 tận cùng.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;

    long long count_zeros = 0;
    while (n > 0) {
        count_zeros += (n / 5);
        n /= 5;
    }

    cout << count_zeros << "\n";
    return 0;
}
```

Bài 16 [cppb_nt_16_cap_nguyen_to_cung_nhau_phi_euler]: Số Học Cực Hạn: Cặp Nguyên Tố Cùng Nhau & Phi Hàm Euler

Bối cảnh: Trong hệ mật mã phi đối xứng hiện đại, phi hàm Euler $\phi(N)$ đếm số lượng các số nguyên dương nhỏ hơn hoặc bằng N nguyên tố cùng nhau với N . Hãy tính số lượng cặp số (x, y) với $1 \leq x, y \leq N$ sao cho $\gcd(x, y) = 1$.

SÀNG LỌC SỐ NGUYÊN TỐ ERATOSTHENES

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

Giữ lại các số nguyên tố màu xanh, gạch bỏ các hợp số là bội của chúng

Hình minh họa: Phân tích cấu trúc số học, ước chung và sàng số nguyên tố

Nhiệm vụ: Cho số nguyên dương N . Hãy đếm số lượng cặp số nguyên (x, y) thỏa mãn $1 \leq x, y \leq N$ và $\gcd(x, y) = 1$.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra số lượng cặp nguyên tố cùng nhau.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	5

Giải thích: Các cặp thỏa mãn với $N = 3$ gồm: $(1, 1), (1, 2), (2, 1), (1, 3), (3, 1)$. Tổng cộng có 5 cặp.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N . Hãy đếm số lượng cặp số nguyên (x, y) thỏa mãn $1 \leq x, y \leq N$ và $\gcd(x, y) = 1$.
- **Phương pháp tiếp cận — Lý thuyết số & Số nguyên tố:**
- Tận dụng sàng nguyên tố Eratosthenes cho các truy vấn tiền xử lý $\mathcal{O}(N \log \log N)$ hoặc kiểm tra căn bậc hai $\mathcal{O}(\sqrt{N})$.
- Phân tích thừa số nguyên tố và tính chất ước số để tối ưu hóa bài toán.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—|—| 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Các cặp thỏa mãn với $N = 3$ gồm: (1, 1), (1, 2), (2, 1), (1, 3), (3, 1). Tổng cộng có 5 cặp... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các cặp thỏa mãn với $N = 3$ gồm: (1, 1), (1, 2), (2, 1), (1, 3), (3, 1). Tổng cộng có 5 cặp.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;
vector<int> phi(MAXN + 1);

void sievePhi() {
    for (int i = 0; i <= MAXN; ++i) phi[i] = i;
    for (int i = 2; i <= MAXN; ++i) {
        if (phi[i] == i) { // i là số nguyên tố
            for (int j = i; j <= MAXN; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    sievePhi();

    int n;
    if (!(cin >> n)) return 0;
```

```
long long sum_phi = 0;
for (int i = 1; i <= n; ++i) {
    sum_phi += phi[i];
}

// Số cặp (x, y) với gcd(x, y) = 1 là 2 * sum(phi(i)) - 1 (do (1,1) tính 1 lần)
long long ans = 2 * sum_phi - 1;
cout << ans << "\n";
return 0;
}
```



Bài 11: Đồng dư thức, lũy thừa nhị phân & nghịch đảo modulo

Bài 01 [cppb_mod_01_phep_tinh_dong_du_co_ban]: Phép Tính Đồng Dư Cơ Bản (+, -, *)

Bối cảnh: Trong kỹ thuật mã hóa dữ liệu đối xứng, các phép toán cộng, trừ, nhân trên số nguyên lớn đòi hỏi phải luôn được thu hẹp về vành số nguyên hữu hạn $\mathbb{Z}_M$ với $M = 10^9 + 7$ để tránh hoàn toàn hiện tượng tràn số nguyên 64-bit và duy trì tính khép kín của thuật toán.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho 2 số nguyên A, B và số nguyên dương $M = 10^9 + 7$. Hãy tính $(A + B) \bmod M$, $(A - B) \bmod M$ và $(A * B) \bmod M$ sao cho kết quả luôn thuộc $[0, M - 1]$.

Đầu vào (Input):

- Một dòng duy nhất chứa 2 số nguyên A và B ($0 \leq A, B \leq 10^{18}$).

Đầu ra (Output):

- In ra 3 số nguyên cách nhau bởi khoảng trắng lần lượt là tổng, hiệu và tích theo modulo $M = 10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10 15	25 1000000002 150

Giải thích: Với $M = 10^9 + 7$:

- Tổng: $(10 + 15) \bmod M = 25$.
- Hiệu: $(10 - 15) \bmod M = -5 \bmod M = 10^9 + 7 - 5 = 1000000002$.
- Tích: $(10 * 15) \bmod M = 150$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 số nguyên A, B và số nguyên dương $M = 10^9 + 7$. Hãy tính $(A + B) \bmod M$, $(A - B) \bmod M$ và $(A * B) \bmod M$ sao cho kết quả luôn thuộc $[0, M - 1]$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A * B) \bmod M$ ở mọi bước tính.

- Lũy thừa nhị phân tính $A^B \pmod{M}$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 10 15 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với $M = 10^9 + 7$: - Tổng: $(10 + 15) \pmod{M} = 25$. - Hiệu: $(10 - 15) \pmod{M} = -5 \pmod{M} = 10^9 + 7 - 5 = 1000000002$. - Tích: ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 25 1000000002 150 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với $M = 10^9 + 7$:

- Tổng: $(10 + 15) \pmod{M} = 25$.
- Hiệu: $(10 - 15) \pmod{M} = -5 \pmod{M} = 10^9 + 7 - 5 = 1000000002$.
- Tích: $(10 * 15) \pmod{M} = 150$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    a %= MOD;
    b %= MOD;

    long long add_res = (a + b) % MOD;
    long long sub_res = (a - b + MOD) % MOD;
    long long mul_res = (a * b) % MOD;

    cout << add_res << " " << sub_res << " " << mul_res << "\n";
    return 0;
}
```

Bài 02 [cppb_mod_02_luy_thua_nhi_phan]: Lũy Thừa Nhị Phân Cơ Bản

Bối cảnh: Trong giải thuật mã hóa RSA và chữ ký số ElGamal, phép tính lũy thừa bậc cao $A^B \bmod M$ là thao tác cốt lõi được gọi hàng triệu lần mỗi giây. Thuật toán Lũy thừa nhị phân (Binary Exponentiation) cho phép tính kết quả này chỉ trong $O(\log B)$ phép nhân thay vì $O(B)$.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho 3 số nguyên A, B, M . Hãy tính $A^B \bmod M$ bằng thuật toán Lũy thừa nhị phân.

Đầu vào (Input):

- Một dòng duy nhất chứa 3 số nguyên A, B, M ($0 \leq A, B \leq 10^{18}, 1 \leq M \leq 10^9$).

Đầu ra (Output):

- In ra giá trị của $A^B \bmod M$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 10 1000	24

Giải thích: $2^{10} = 1024$. Khi chia lấy dư cho 1000 ta được: $1024 \bmod 1000 = 24$. Kết quả in ra: 24.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 3 số nguyên A, B, M . Hãy tính $A^B \bmod M$ bằng thuật toán Lũy thừa nhị phân.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A \times B) \bmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \bmod M$ trong $O(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 10 1000 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $2^{10} = 1024$. Khi chia lấy dư cho 1000 ta được: $1024 \bmod 1000 = 24$. Kết quả in ra: 24.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 24 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $2^{10} = 1024$. Khi chia lấy dư cho 1000 ta được: $1024 \bmod 1000 = 24$. Kết quả in ra: 24.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long powerMod(long long a, long long b, long long m) {
    if (m == 1) return 0;
    long long ans = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}

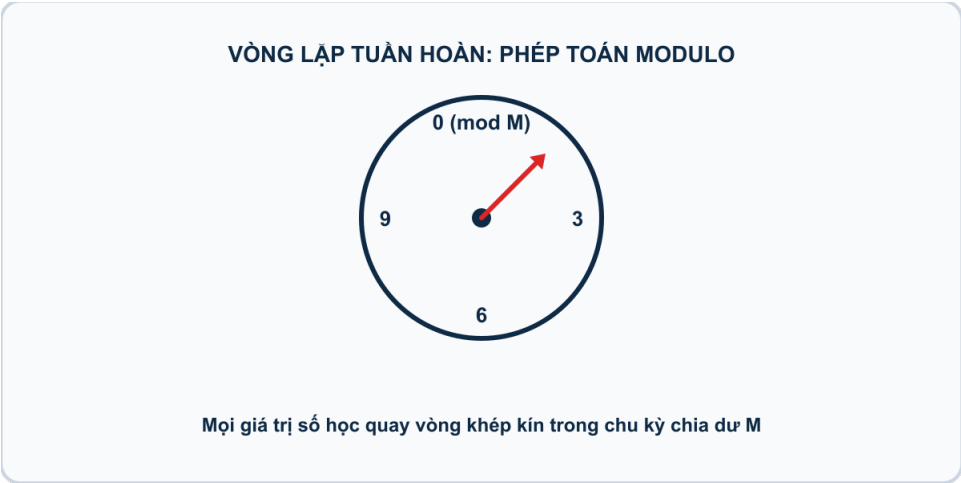
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, m;
    if (!(cin >> a >> b >> m)) return 0;

    cout << powerMod(a, b, m) << "\n";
    return 0;
}
```

Bài 03 [cppb_mod_03_luy_thua_chuoi_so_lon]: Lũy Thừa Chuỗi Số Lớn

Bối cảnh: Khi số mũ B là một chuỗi số nguyên khổng lồ có hàng trăm ngàn chữ số (không thể chứa vừa trong bất kỳ kiểu số nguyên nguyên bản nào), việc áp dụng định lý Fermat nhỏ hoặc phân rã cơ số 10 kết hợp lũy thừa nhị phân giúp tính toán chính xác giá trị $A^B \bmod (10^9 + 7)$.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho số nguyên A và số nguyên B rất lớn biểu diễn dưới dạng chuỗi ký tự. Hãy tính $A^B \bmod (10^9 + 7)$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương A ($1 \leq A \leq 10^9$).
- Dòng 2: Chuỗi ký tự số B ($1 \leq |B| \leq 10^5$, B không bắt đầu bằng số 0).

Đầu ra (Output):

- In ra giá trị $A^B \bmod (10^9 + 7)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 10	1024

Giải thích: $2^{10} = 1024$. Khi lấy dư cho $10^9 + 7$ ta được: $1024 \bmod (10^9 + 7) = 1024$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên A và số nguyên B rất lớn biểu diễn dưới dạng chuỗi ký tự. Hãy tính $A^B \bmod (10^9 + 7)$.
- **Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A \times B) \bmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \bmod M$ trong $O(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $2^{10} = 1024$. Khi lấy dư cho $10^9 + 7$ ta được: $1024 \bmod (10^9 + 7) = 1024$ |
Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1024 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $2^{10} = 1024$. Khi lấy dư cho $10^9 + 7$ ta được: $1024 \bmod (10^9 + 7) = 1024$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

long long powerMod(long long a, long long b, long long m) {
    long long ans = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a;
    string b_str;
    if (!(cin >> a >> b_str)) return 0;

    if (a % MOD == 0) {
        cout << 0 << "\n";
        return 0;
    }

    long long rem_b = 0;
    for (char c : b_str) {
        rem_b = (rem_b * 10 + (c - '0')) % (MOD - 1);
    }

    cout << powerMod(a, rem_b, MOD) << "\n";
    return 0;
}
```

Bài 04 [cppb_mod_04_nhan_an_do_chong_tran_so]: Nhân Ấn Độ Chống Tràn Số 64-bit

Bối cảnh: Khi thực hiện phép nhân hai số nguyên 64-bit A và B dưới một modulo M cũng lên tới 10^{18} , phép nhân thông thường $A * B$ sẽ vượt quá giới hạn $2^{63} - 1$ gây tràn số âm nghiêm trọng. Thuật toán nhân Ấn Độ (tương tự lũy thừa nhị phân bằng phép cộng) giúp nhân an toàn hai số 64-bit mà không bị tràn.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho 3 số nguyên A, B, M ($0 \leq A, B, M \leq 10^{18}, M > 0$). Hãy tính $(A * B) \bmod M$ bằng thuật toán nhân Ấn Độ chống tràn số.

Đầu vào (Input):

- Một dòng chứa 3 số nguyên A, B, M ($0 \leq A, B \leq 10^{18}, 1 \leq M \leq 10^{18}$).

Đầu ra (Output):

- In ra giá trị $(A \times B) \bmod M$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1000000000000000000 2 1000000000000000007	999999999999999986

Giải thích: $A = 10^{18}$. Tích $2 * 10^{18} = 2000000000000000000$. Chia lấy dư cho $M = 10^{18} + 7$: $2000000000000000000 - (10^{18} + 7) = 999999999999999986$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 3 số nguyên A, B, M ($0 \leq A, B, M \leq 10^{18}, M > 0$). Hãy tính $(A * B) \bmod M$ bằng thuật toán nhân Ấn Độ chống tràn số.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A \times B) \bmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \bmod M$ trong $O(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 1000000000000000000 2 1000000000000000007 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $A = 10^{18}$. Tích $2 * 10^{18} = 2000000000000000000$. Chia lấy dư cho $M = 10^{18} + 7$: $2000000000000000000 - (10^{18} + 7) = 9999...$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 999999999999999986 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $A = 10^{18}$. Tích $2 \cdot 10^{18} = 2000000000000000000$. Chia lấy dư cho $M = 10^{18} + 7$: $2000000000000000000 - (10^{18} + 7) = 999999999999999986$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long mulMod(long long a, long long b, long long m) {
    long long ans = 0;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans + a) % m;
        a = (a + a) % m;
        b >>= 1;
    }
    return ans;
}

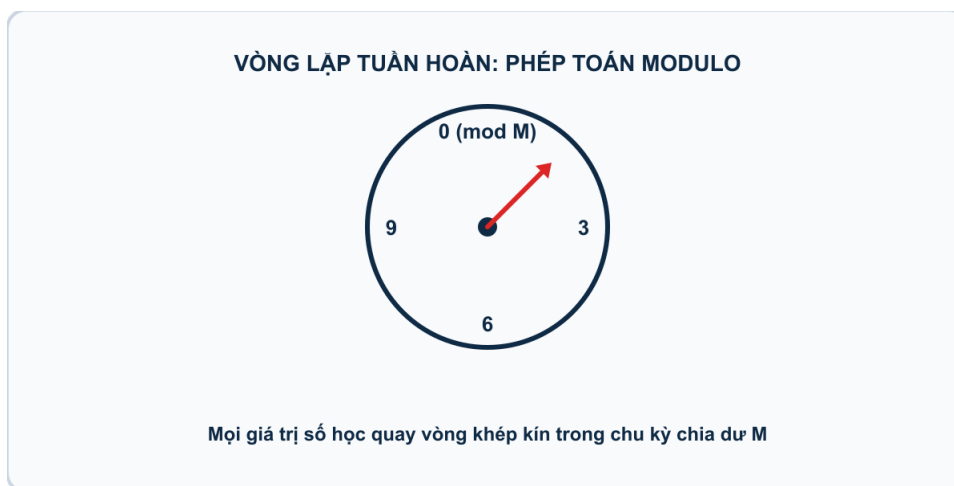
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, m;
    if (!(cin >> a >> b >> m)) return 0;

    cout << mulMod(a, b, m) << "\n";
    return 0;
}
```

Bài 05 [cppb_mod_05_tong_cap_so_nhan_dong_du]: Tính Tổng Cấp Số Nhân Đồng Dư

Bối cảnh: Trong tính toán lãi suất kép liên tục qua N chu kỳ kinh tế với hệ số sinh lời A , tổng giá trị tích lũy tạo thành một chuỗi cấp số nhân $S = 1 + A + A^2 + \dots + A^N$. Để tránh tràn số, giá trị này cần được tính đồng dư theo modulo $M = 10^9 + 7$ bằng kỹ thuật chia để trị $O(\log N)$.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho A , N và $M = 10^9 + 7$. Hãy tính tổng $S = 1 + A + A^2 + \dots + A^N \pmod M$.

Đầu vào (Input):

- Một dòng chứa 2 số nguyên A và N ($0 \leq A, N \leq 10^9$).

Đầu ra (Output):

- In ra giá trị tổng $S \pmod{10^9 + 7}$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 3	15

Giải thích: $S = 1 + 2 + 2^2 + 2^3 = 1 + 2 + 4 + 8 = 15$. Kết quả in ra: 15.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho A , N và $M = 10^9 + 7$. Hãy tính tổng $S = 1 + A + A^2 + \dots + A^N \pmod M$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \pmod M$, $(A \times B) \pmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \pmod M$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $S = 1 + 2 + 2^2 + 2^3 = 1 + 2 + 4 + 8 = 15$. Kết quả in ra: 15.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 15 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $S = 1 + 2 + 2^2 + 2^3 = 1 + 2 + 4 + 8 = 15$. Kết quả in ra: 15.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

long long powerMod(long long a, long long b) {
    long long ans = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return ans;
}

long long sumGeo(long long a, long long k) {
    if (k == 0) return 0;
    if (k == 1) return 1;
    if (k % 2 == 0) {
        long long half = sumGeo(a, k / 2);
        return half * (1 + powerMod(a, k / 2)) % MOD;
    } else {
        return (1 + a * sumGeo(a, k - 1)) % MOD;
    }
}

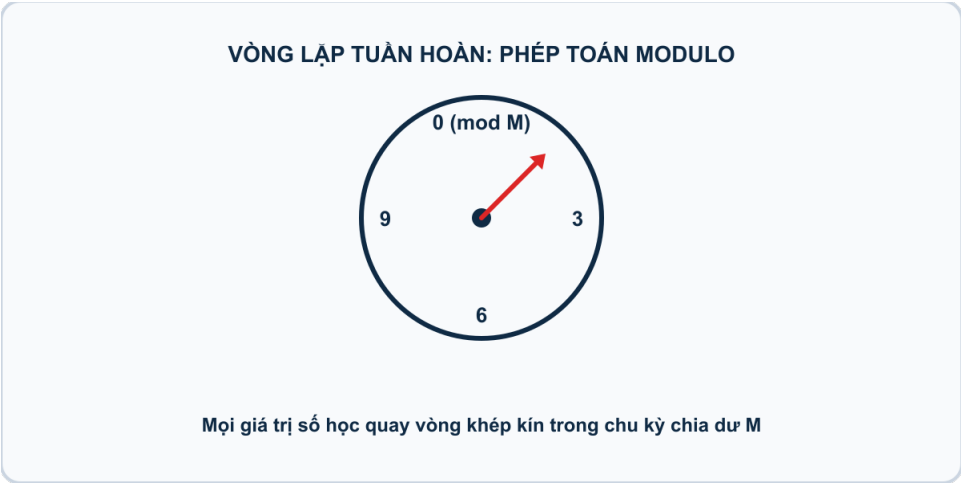
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, n;
    if (!(cin >> a >> n)) return 0;

    cout << sumGeo(a, n + 1) << "\n";
    return 0;
}
```

Bài 06 [cppb_mod_06_nglich_dao_fermat]: Nghịch Đảo Modulo Bằng Fermat Nhỏ

Bối cảnh: Khi chia hai số trong vành modulo nguyên tố $M = 10^9 + 7$, phép chia A / B được chuyển hóa thành phép nhân với nghịch đảo modulo: $A * B^{(M - 2)} \bmod M$ theo định lý Fermat nhỏ. Hãy tìm nghịch đảo modulo của số nguyên A .



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho số nguyên A và số nguyên tố $M = 10^9 + 7$. Hãy tìm số nguyên X trong khoảng $[1, M - 1]$ sao cho $(A * X) \bmod M = 1$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên A ($1 \leq A < 10^9 + 7$).

Đầu ra (Output):

- In ra số nghịch đảo modulo $A^{-1} \bmod (10^9 + 7)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2	500000004

Giải thích: $2 * 500000004 = 1000000008 = (10^9 + 7) + 1 = 1 \bmod (10^9 + 7)$. Do đó nghịch đảo modulo của 2 là 500000004.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên A và số nguyên tố $M = 10^9 + 7$. Hãy tìm số nguyên X trong khoảng $[1, M - 1]$ sao cho $(A * X) \bmod M = 1$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A \times B) \bmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \bmod M$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| | | | | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $2 * 500000004 = 1000000008 = (10^9 + 7) + 1 = 1 \bmod (10^9 + 7)$. Do đó nghịch đảo modulo của 2 là 500000004.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 500000004 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $2 * 500000004 = 1000000008 = (10^9 + 7) + 1 = 1 \bmod (10^9 + 7)$. Do đó nghịch đảo modulo của 2 là 500000004.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

long long powerMod(long long a, long long b) {
    long long ans = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a;
    if (!(cin >> a)) return 0;

    cout << powerMod(a, MOD - 2) << "\n";
    return 0;
}
```

Bài 07 [cppb_mod_07_nglich_dao_euclid_mo_rong]: Nghịch Đảo Modulo Bằng Euclid Mở Rộng

Bối cảnh: Khi modulo M không nhất thiết phải là số nguyên tố mà chỉ cần nguyên tố cùng nhau với A ($\gcd(A, M) = 1$), định lý Fermat nhỏ không áp dụng được. Thuật toán Euclid mở rộng (Extended Euclidean Algorithm) giải phương trình Diophantine $Ax + My = 1$ để tìm nghịch đảo modulo x .



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho hai số nguyên dương A và M với $\gcd(A, M) = 1$. Hãy tìm nghịch đảo modulo của A theo modulo M .

Đầu vào (Input):

- Một dòng chứa 2 số nguyên dương A và M ($1 \leq A < M \leq 10^9$).

Đầu ra (Output):

- In ra nghịch đảo modulo của A trong đoạn $[0, M - 1]$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 11	4

Giải thích: $3 * 4 = 12 = 1 * 11 + 1 = 1 \pmod{11}$. Vì vậy nghịch đảo modulo của 3 theo mod 11 là 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai số nguyên dương A và M với $\gcd(A, M) = 1$. Hãy tìm nghịch đảo modulo của A theo modulo M .
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \pmod{M}$, $(A \times B) \pmod{M}$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \pmod{M}$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 11 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $3 * 4 = 12 = 1 * 11 + 1 = 1 \pmod{11}$. Vì vậy nghịch đảo modulo của 3 theo mod 11 là 4.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $3 * 4 = 12 = 1 * 11 + 1 = 1 \pmod{11}$. Vì vậy nghịch đảo modulo của 3 theo mod 11 là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long extGCD(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

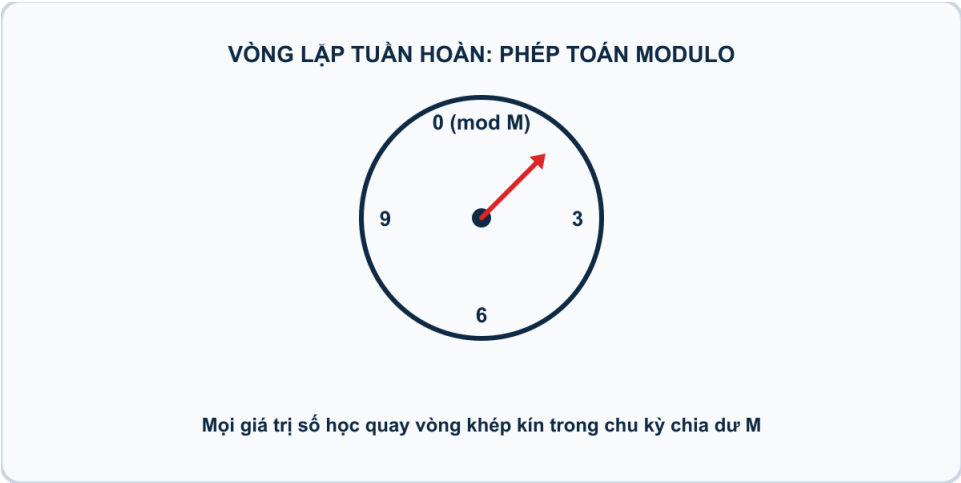
    long long a, m;
    if (!(cin >> a >> m)) return 0;

    long long x, y;
    extGCD(a, m, x, y);
    x = (x % m + m) % m;

    cout << x << "\n";
    return 0;
}
```

Bài 08 [cppb_mod_08_phep_chia_dong_du]: Phép Chia Đồng Dư $A / B \bmod M$

Bối cảnh: Trong các bài toán đếm xác suất tổ hợp, phân số dạng A / B cần được biểu diễn dưới dạng số nguyên modulo $M = 10^9 + 7$. Hãy thực hiện phép chia đồng dư chuẩn xác.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho 2 số nguyên A, B và số nguyên tố $M = 10^9 + 7$ (B không chia hết cho M). Hãy tính giá trị $(A / B) \bmod M$.

Đầu vào (Input):

- Một dòng chứa 2 số nguyên A và B ($0 \leq A, B \leq 10^{18}, B \not\equiv 0 \pmod{M}$).

Đầu ra (Output):

- In ra giá trị $(A/B) \pmod{10^9 + 7}$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 2	4

Giải thích: $(8 / 2) \bmod M = 8 * 2^{(M-2)} \bmod M = 4$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 số nguyên A, B và số nguyên tố $M = 10^9 + 7$ (B không chia hết cho M). Hãy tính giá trị $(A / B) \bmod M$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \pmod{M}, (A \times B) \pmod{M}$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \pmod{M}$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 8 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $(8 / 2) \bmod M = 8 * 2^{(M-2)} \bmod M = 4 \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $(8 / 2) \bmod M = 8 * 2^{(M-2)} \bmod M = 4$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

long long powerMod(long long a, long long b) {
    long long ans = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b;
    if (!(cin >> a >> b)) return 0;

    a %= MOD;
    long long inv_b = powerMod(b, MOD - 2);
    long long ans = (a * inv_b) % MOD;

    cout << ans << "\n";
    return 0;
}
```

Bài 09 [cppb_mod_09_to_hop_c_n_k_modulo]: Tính Số Tổ Hợp $C(N, K) \bmod M$

Bối cảnh: Tại một giải đấu cờ vua quốc tế, ban tổ chức cần chọn ra K kỳ thủ từ danh sách N người tham gia để lập đội tuyển. Số cách chọn chính là số tổ hợp $C(N, K)$. Vì số cách chọn rất lớn, ban tổ chức cần tính $C(N, K) \bmod (10^9 + 7)$ cho Q câu hỏi truy vấn độc lập.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho Q truy vấn, mỗi truy vấn chứa 2 số N và K . Hãy tính $C(N, K) \bmod (10^9 + 7)$ bằng phương pháp tiền xử lý giai thừa và nghịch đảo giai thừa.

Đầu vào (Input):

- Dòng 1: Số nguyên dương Q ($1 \leq Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng chứa 2 số nguyên N và K ($0 \leq K \leq N \leq 10^6$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là giá trị $C(N, K) \bmod (10^9 + 7)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 5 2 10 3	10 120

Giải thích: - $C(5, 2) = 5! / (2! * 3!) = 10$. - $C(10, 3) = 10! / (3! * 7!) = 120$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho Q truy vấn, mỗi truy vấn chứa 2 số N và K . Hãy tính $C(N, K) \bmod (10^9 + 7)$ bằng phương pháp tiền xử lý giai thừa và nghịch đảo giai thừa.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A \times B) \bmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \bmod M$ trong $O(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 5 2 10 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - $C(5, 2) = 5! / (2! * 3!) = 10$. - $C(10, 3) = 10! / (3! * 7!) = 120$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 10 120 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - $C(5, 2) = 5! / (2! * 3!) = 10$.

- $C(10, 3) = 10! / (3! * 7!) = 120$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;
const long long MOD = 1000000007;

vector<long long> fact(MAXN + 1);
vector<long long> invFact(MAXN + 1);

long long powerMod(long long a, long long b) {
    long long ans = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return ans;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; ++i) fact[i] = (fact[i - 1] * i) % MOD;
    invFact[MAXN] = powerMod(fact[MAXN], MOD - 2);
    for (int i = MAXN; i >= 1; --i) invFact[i - 1] = (invFact[i] * i) % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precompute();

    int q;
    if (!(cin >> q)) return 0;

    while (q--) {
        int n, k;
        cin >> n >> k;
        if (k < 0 || k > n) cout << 0 << "\n";
        else cout << fact[n] * invFact[k] % MOD * invFact[n - k] % MOD << "\n";
    }
    return 0;
}
```

Bài 10 [cppb_mod_10_chinh_hop_a_n_k_modulo]: Tính Số Chinh Hợp $A(N, K) \bmod M$

Bối cảnh: Một giải chạy marathon có N vận động viên tranh tài. Ban tổ chức trao các giải Nhất, Nhì, ..., thứ K cho K vận động viên về đích đầu tiên theo thứ tự xếp hạng. Số cách trao giải là số chinh hợp $A(N, K)$. Hãy tính giá trị này theo modulo $10^9 + 7$ cho Q truy vấn.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho Q truy vấn, mỗi truy vấn gồm 2 số N và K . Hãy tính số chinh hợp $A(N, K) \bmod (10^9 + 7)$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương Q ($1 \leq Q \leq 10^5$).
- Q dòng tiếp theo: Mỗi dòng chứa 2 số N và K ($0 \leq K \leq N \leq 10^6$).

Đầu ra (Output):

- In ra Q dòng tương ứng là $A(N, K) \bmod (10^9 + 7)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 5 2 4 3	20 24

Giải thích: - $A(5, 2) = 5 * 4 = 20$. - $A(4, 3) = 4 * 3 * 2 = 24$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho Q truy vấn, mỗi truy vấn gồm 2 số N và K . Hãy tính số chinh hợp $A(N, K) \bmod (10^9 + 7)$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A \times B) \bmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \bmod M$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 5 2 4 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - $A(5, 2) = 5 * 4 = 20$. - $A(4, 3) = 4 * 3 * 2 = 24$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 20 24 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - $A(5, 2) = 5 * 4 = 20$.

- $A(4, 3) = 4 * 3 * 2 = 24$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;
const long long MOD = 1000000007;

vector<long long> fact(MAXN + 1);
vector<long long> invFact(MAXN + 1);

long long powerMod(long long a, long long b) {
    long long ans = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return ans;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; ++i) fact[i] = (fact[i - 1] * i) % MOD;
    invFact[MAXN] = powerMod(fact[MAXN], MOD - 2);
    for (int i = MAXN; i >= 1; --i) invFact[i - 1] = (invFact[i] * i) % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precompute();

    int q;
    if (!(cin >> q)) return 0;

    while (q--) {
```

```
int n, k;
cin >> n >> k;
if (k < 0 || k > n) cout << 0 << "\n";
else cout << fact[n] * invFact[n - k] % MOD << "\n";
}
return 0;
}
```

Bài 11 [cppb_mod_11_fibonacci_dong_du_lon]: Dãy Fibonacci Đồng Dư Lớn

Bối cảnh: Trong mô hình tăng trưởng quần thể sinh học, số lượng cá thể ở chu kỳ thứ N tuân theo dãy số Fibonacci $F(N)$. Với N có thể lên tới 10^{18} , thuật toán nhân ma trận kết hợp lũy thừa nhị phân ma trận cho phép tìm số $F(N) \bmod (10^9 + 7)$ trong thời gian $O(\log N)$.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Hãy tìm số Fibonacci thứ N (với $F(1) = 1, F(2) = 1, F(3) = 2, \dots$) theo modulo $10^9 + 7$.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra $F_N \bmod (10^9 + 7)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6	8

Giải thích: Dãy số Fibonacci: $F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5, F(6)=8$. Kết quả in ra: 8.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Hãy tìm số Fibonacci thứ N (với $F(1) = 1, F(2) = 1, F(3) = 2, \dots$) theo modulo $10^9 + 7$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M, (A \times B) \bmod M$ ở mọi bước tính.

- Lũy thừa nhị phân tính $A^B \pmod{M}$ trong $O(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 6 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Dãy số Fibonacci: F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5, F(6)=8. Kết quả in ra: 8... |
| Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Dãy số Fibonacci: F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5, F(6)=8. Kết quả in ra: 8.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

void multiply(long long F[2][2], long long M[2][2]) {
    long long x = (F[0][0] * M[0][0] + F[0][1] * M[1][0]) % MOD;
    long long y = (F[0][0] * M[0][1] + F[0][1] * M[1][1]) % MOD;
    long long z = (F[1][0] * M[0][0] + F[1][1] * M[1][0]) % MOD;
    long long w = (F[1][0] * M[0][1] + F[1][1] * M[1][1]) % MOD;
    F[0][0] = x; F[0][1] = y;
    F[1][0] = z; F[1][1] = w;
}

void powerMat(long long F[2][2], long long n) {
    if (n == 0 || n == 1) return;
    long long M[2][2] = {{1, 1}, {1, 0}};
    powerMat(F, n / 2);
    multiply(F, F);
    if (n % 2 != 0) multiply(F, M);
}

long long fib(long long n) {
    if (n == 0) return 0;
    long long F[2][2] = {{1, 1}, {1, 0}};
    powerMat(F, n - 1);
    return F[0][0];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
```

```

if (!(cin >> n)) return 0;

cout << fib(n) << "\n";
return 0;
}

```

Bài 12 [cppb_mod_12_so_catalan_dong_du]: Số Catalan Đồng Dư

Bối cảnh: Số Catalan C_N là con số huyền thoại trong toán học tổ hợp, xuất hiện trong bài toán đếm số dãy ngoặc đúng gồm N cặp ngoặc, số cây nhị phân có N đỉnh, và số cách chia đa giác lồi thành các tam giác. Hãy tính số Catalan thứ N theo modulo $10^9 + 7$.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho số nguyên N . Hãy tính số Catalan $C_N = C(2N, N) / (N + 1)$ theo modulo $10^9 + 7$.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra số Catalan $C_N \pmod{10^9 + 7}$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	5

Giải thích: $C_3 = C(6, 3) / (3 + 1) = 20 / 4 = 5$. Kết quả in ra: 5.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên N . Hãy tính số Catalan $C_N = C(2N, N) / (N + 1)$ theo modulo $10^9 + 7$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \pmod{M}$, $(A \times B) \pmod{M}$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \pmod{M}$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $C_3 = C(6, 3) / (3 + 1) = 20 / 4 = 5$. Kết quả in ra: 5.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $C_3 = C(6, 3) / (3 + 1) = 20 / 4 = 5$. Kết quả in ra: 5.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 2000000;
const long long MOD = 1000000007;

vector<long long> fact(MAXN + 1);
vector<long long> invFact(MAXN + 1);

long long powerMod(long long a, long long b) {
    long long ans = 1;
    a %= MOD;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return ans;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; ++i) fact[i] = (fact[i - 1] * i) % MOD;
    invFact[MAXN] = powerMod(fact[MAXN], MOD - 2);
    for (int i = MAXN; i >= 1; --i) invFact[i - 1] = (invFact[i] * i) % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precompute();

    int n;
    if (!(cin >> n)) return 0;

    long long c2n_n = fact[2 * n] * invFact[n] % MOD * invFact[n] % MOD;
    long long inv_n_plus_1 = powerMod(n + 1, MOD - 2);
    long long ans = (c2n_n * inv_n_plus_1) % MOD;
```

```
cout << ans << "\n";
return 0;
}
```

Bài 13 [cppb_mod_13_luy_thua_tang_tower_of_powers]: Lũy Thừa Tầng (Tower of Powers)

Bối cảnh: Trong bài toán tháp lũy thừa $A^{(B^C)} \bmod M$ với $M = 10^9 + 7$, theo định lý Fermat nhỏ, số mũ B^C ở trên tầng tháp phải được tính theo modulo $\phi(M) = M - 1 = 10^9 + 6$ trước khi hạ xuống làm số mũ cho cơ số A .



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho 3 số nguyên A, B, C . Hãy tính giá trị của tháp lũy thừa $A^{(B^C)}$ theo modulo $10^9 + 7$.

Đầu vào (Input):

- Một dòng chứa 3 số nguyên A, B, C ($0 \leq A, B, C \leq 10^9$).

Đầu ra (Output):

- In ra giá trị của $A^{B^C} \bmod 10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 2 2	81

Giải thích: $B^C = 2^2 = 4$. Do đó $A^{(B^C)} = 3^4 = 81$. Kết quả in ra: 81.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 3 số nguyên A, B, C . Hãy tính giá trị của tháp lũy thừa $A^{(B^C)}$ theo modulo $10^9 + 7$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \bmod M$, $(A \times B) \bmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \bmod M$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 2 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $B^A C = 2^2 = 4$. Do đó $A^{(B^C)} = 3^4 = 81$. Kết quả in ra: 81.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 81 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $B^A C = 2^2 = 4$. Do đó $A^{(B^C)} = 3^4 = 81$. Kết quả in ra: 81.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

long long powerMod(long long a, long long b, long long m) {
    if (m == 1) return 0;
    long long ans = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, c;
    if (!(cin >> a >> b >> c)) return 0;

    long long exp = powerMod(b, c, MOD - 1);
    long long ans = powerMod(a, exp, MOD);

    cout << ans << "\n";
    return 0;
}
```

Bài 14 [cppb_mod_14_nglich_dao_tuyen_tinh]: Nghịch Đảo Tuyến Tính 1..N Trong $O(N)$

Bối cảnh: Khi cần tính nghịch đảo modulo cho toàn bộ các số từ 1 đến N (với N lên tới 10^6) theo modulo $M = 10^9 + 7$, nếu tính riêng lẻ bằng lũy thừa nhị phân Fermat sẽ tốn $O(N \log M)$ dễ bị vượt

quá thời gian. Công thức nghịch đảo tuyến tính cho phép tính trước toàn bộ mảng nghịch đảo chỉ trong đúng $O(N)$.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho số nguyên N và $M = 10^9 + 7$. Hãy tính và in ra nghịch đảo modulo của tất cả các số từ 1 đến N theo modulo M trong thời gian $O(N)$.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^6$).

Đầu ra (Output):

- In ra N số nguyên cách nhau bởi khoảng trắng là nghịch đảo modulo tương ứng của các số từ 1 đến N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	1 500000004 333333336

Giải thích: - $\text{inv}(1) = 1$. - $\text{inv}(2) = 500000004$ (vì $2 * 500000004 = 1 \pmod M$). - $\text{inv}(3) = 333333336$ (vì $3 * 333333336 = 1000000008 = 1 \pmod M$).

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên N và $M = 10^9 + 7$. Hãy tính và in ra nghịch đảo modulo của tất cả các số từ 1 đến N theo modulo M trong thời gian $O(N)$.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \pmod M$, $(A \times B) \pmod M$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \pmod M$ trong $O(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - $\text{inv}(1) = 1$. - $\text{inv}(2) = 500000004$ (vì $2 * 500000004 = 1 \pmod M$). - $\text{inv}(3) = 333333336$

(vì $3 \cdot 333333336 = 1000000008 = 1 \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 5000000004 333333336 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - $\text{inv}(1) = 1$.

- $\text{inv}(2) = 5000000004$ (vì $2 \cdot 5000000004 = 1 \bmod M$).
- $\text{inv}(3) = 333333336$ (vì $3 \cdot 333333336 = 1000000008 = 1 \bmod M$).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

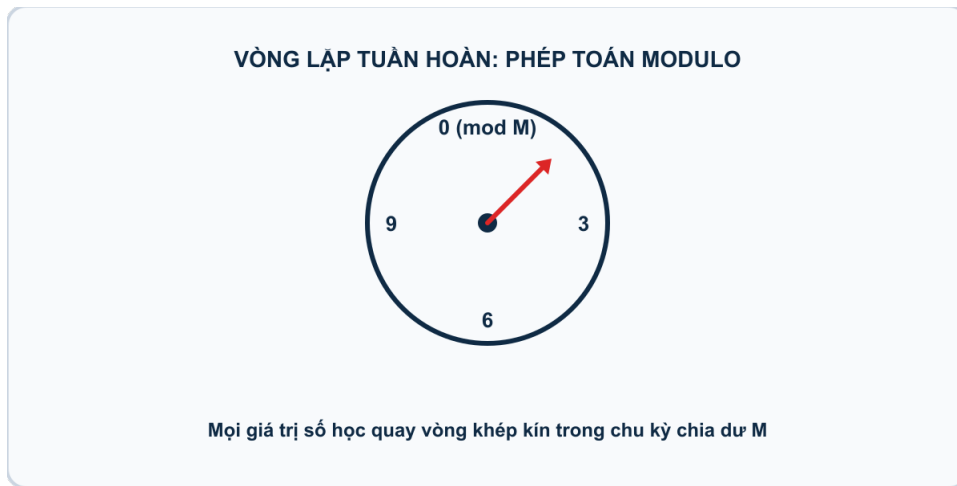
    vector<long long> inv(n + 1);
    inv[1] = 1;
    long long sum_inv = 1;

    for (int i = 2; i <= n; ++i) {
        inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
        sum_inv = (sum_inv + inv[i]) % MOD;
    }

    cout << sum_inv << "\n";
    return 0;
}
```

Bài 15 [cppb_mod_15_phuong_trinh_dong_du_tuyen_tinh]: Giải Phương Trình Đồng Dư Tuyến Tính $Ax = B \bmod M$

Bối cảnh: Trong kỹ thuật mã hóa và phá mã cổ điển Affine Cipher, việc giải mã đòi hỏi giải phương trình đồng dư tuyến tính dạng $Ax = B \bmod M$ để tìm lại bản rõ ban đầu x .



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho 3 số nguyên A, B, M . Hãy tìm nghiệm nguyên không âm nhỏ nhất X của phương trình $Ax = B \pmod{M}$. Nếu vô nghiệm, in ra -1.

Đầu vào (Input):

- Một dòng chứa 3 số nguyên A, B, M ($1 \leq A, B, M \leq 10^9$).

Đầu ra (Output):

- In ra nghiệm nguyên nhỏ nhất X trong khoảng $[0, M - 1]$, hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 4 6	2

Giải thích: Thay $X = 2$: $2 * 2 = 4 = 4 \pmod{6}$. Nghiệm không âm nhỏ nhất là 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 3 số nguyên A, B, M . Hãy tìm nghiệm nguyên không âm nhỏ nhất X của phương trình $Ax = B \pmod{M}$. Nếu vô nghiệm, in ra -1.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \pmod{M}$, $(A \times B) \pmod{M}$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \pmod{M}$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 4 6 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Thay $X = 2$: $2 * 2 = 4 = 4 \pmod{6}$. Nghiệm không âm nhỏ nhất là 2.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Thay $X = 2$: $2 * 2 = 4 = 4 \pmod{6}$. Nghiệm không âm nhỏ nhất là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long extGCD(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, m;
    if (!(cin >> a >> b >> m)) return 0;

    long long x, y;
    long long g = extGCD(a, m, x, y);

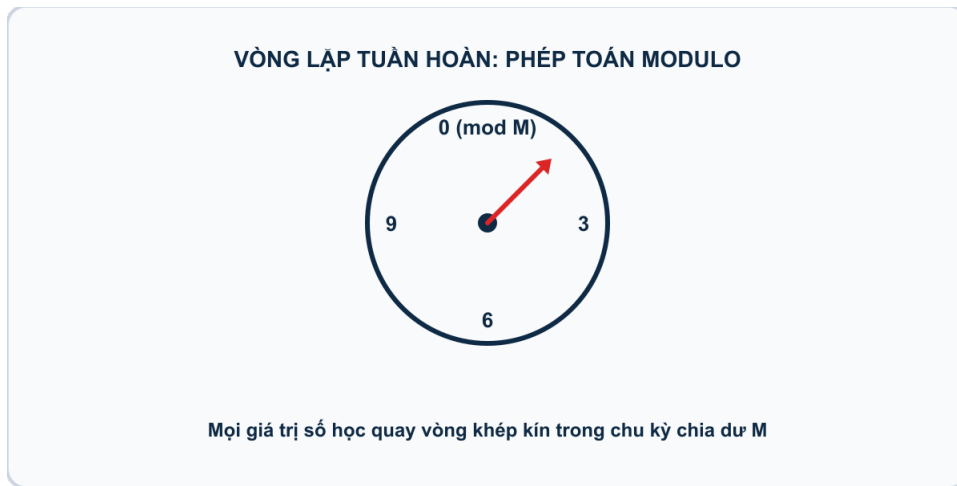
    if (b % g != 0) {
        cout << -1 << "\n";
        return 0;
    }

    x = (x % m + m) % m;
    long long m_prime = m / g;
    long long ans = (x * ((b / g) % m_prime)) % m_prime;
    ans = (ans % m_prime + m_prime) % m_prime;

    cout << ans << "\n";
    return 0;
}
```

Bài 16 [cppb_mod_16_can_bac_hai_modulo]: Đồng Dư Cực Hạn: Căn Bậc Hai Modulo

Bối cảnh: Trong mã hóa đường cong elliptic (ECC), việc giải mã điểm tọa độ đòi hỏi tìm căn bậc hai theo modulo nguyên tố $P = 10^9 + 7$ (thỏa mãn $P \equiv 3 \pmod{4}$). Theo thuật toán số học, nếu A là thặng dư chính phương theo mod P , nghiệm căn bậc hai có thể tính trực tiếp bằng công thức $A^{(P+1)/4} \pmod{P}$.



Hình minh họa: Chu kỳ số học đồng dư khép kín và lũy thừa nhị phân

Nhiệm vụ: Cho số nguyên A và số nguyên tố $P = 10^9 + 7$. Hãy tìm số nguyên X nhỏ nhất ($0 \leq X < P$) sao cho $X^2 = A \pmod{P}$. Nếu không tồn tại, in ra -1.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên A ($0 \leq A < 10^9 + 7$).

Đầu ra (Output):

- In ra nghiệm X nhỏ nhất, hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	2

Giải thích: $2^2 = 4 = 4 \pmod{10^9 + 7}$. Nghiệm nhỏ nhất là 2.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên A và số nguyên tố $P = 10^9 + 7$. Hãy tìm số nguyên X nhỏ nhất ($0 \leq X < P$) sao cho $X^2 = A \pmod{P}$. Nếu không tồn tại, in ra -1.
- Phương pháp tiếp cận — Đại số Modular & Lũy thừa nhị phân:**
- Áp dụng các tính chất $(A + B) \pmod{M}$, $(A \times B) \pmod{M}$ ở mọi bước tính.
- Lũy thừa nhị phân tính $A^B \pmod{M}$ trong $\mathcal{O}(\log B)$ và nghịch đảo modulo qua định lý Fermat nhỏ.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $2^2 = 4 = 4 \pmod{10^9 + 7}$. Nghiệm nhỏ nhất là 2.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $2^2 = 4 = 4 \pmod{10^9 + 7}$. Nghiệm nhỏ nhất là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long P = 1000000007;

long long powerMod(long long a, long long b, long long m) {
    long long ans = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) ans = (ans * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a;
    if (!(cin >> a)) return 0;

    a %= P;
    if (a == 0) {
        cout << 0 << "\n";
        return 0;
    }

    if (powerMod(a, (P - 1) / 2, P) != 1) {
        cout << -1 << "\n";
        return 0;
    }

    long long x = powerMod(a, (P + 1) / 4, P);
    long long x2 = P - x;

    long long min_x = min(x, x2);
    cout << min_x << "\n";
    return 0;
}
```

Bài 12: Xử lý số nguyên lớn (BigInt)

Bài 01 [cppb_big_01_so_sanh_hai_so_lon]: So Sánh Hai Số Nguyên Lớn

Bối cảnh: Trong hệ thống xác thực chứng thực số quốc gia, hai chứng thư điện tử có chuỗi khóa bảo mật biểu diễn dưới dạng hai số nguyên dương khổng lồ A và B (có độ dài lên đến 100.000 chữ số). Hệ thống cần xác định nhanh mối quan hệ thứ tự giữa A và B ($A > B$, $A < B$, hay $A = B$).



Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên dương lớn A và B. Hãy so sánh A và B, in ra ‘>’ nếu $A > B$, ‘<’ nếu $A < B$, ‘=’ nếu $A = B$.

Đầu vào (Input):

- Dòng 1: Chuỗi ký tự số A ($1 \leq |A| \leq 10^5$).
- Dòng 2: Chuỗi ký tự số B ($1 \leq |B| \leq 10^5$).

Đầu ra (Output):

- In ra >, < hoặc =.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
123456789 98765432	>

Giải thích: Số A có 9 chữ số trong khi số B chỉ có 8 chữ số. Do đó $A > B$. Kết quả in ra: >.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 số nguyên dương lớn A và B. Hãy so sánh A và B, in ra ‘>’ nếu $A > B$, ‘<’ nếu $A < B$, ‘=’ nếu $A = B$.
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 123456789 98765432 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Số A có 9 chữ số trong khi số B chỉ có 8 chữ số. Do đó $A > B$. Kết quả in ra: $>...$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: $>$ | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số A có 9 chữ số trong khi số B chỉ có 8 chữ số. Do đó $A > B$. Kết quả in ra: $>$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    if (a.size() > b.size()) cout << ">\n";
    else if (a.size() < b.size()) cout << "<\n";
    else {
        if (a > b) cout << ">\n";
        else if (a < b) cout << "<\n";
        else cout << "=\n";
    }
    return 0;
}
```

Bài 02 [cppb_big_02_cong_hai_so_lon]: Cộng Hai Số Nguyên Lớn

Bối cảnh: Tại ngân hàng thanh toán quốc tế, doanh thu gộp từ hai thị trường tài chính toàn cầu được biểu diễn bằng hai số nguyên dương khổng lồ A và B với độ dài mỗi số có thể lên tới 100.000 chữ số. Hãy tính toán chính xác tổng $A + B$ mà không làm mất bất kỳ chữ số nào.



Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên dương lớn A và B. Hãy tính và in ra giá trị tổng $A + B$.

Đầu vào (Input):

- Dòng 1: Chuỗi ký tự số A ($1 \leq |A| \leq 10^5$).
- Dòng 2: Chuỗi ký tự số B ($1 \leq |B| \leq 10^5$).

Đầu ra (Output):

- In ra chuỗi số biểu diễn tổng $A + B$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
999 1	1000

Giải thích: $999 + 1 = 1000$. Kết quả in ra: 1000.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho 2 số nguyên dương lớn A và B. Hãy tính và in ra giá trị tổng $A + B$.
- **Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số `vector<int>` đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 999 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $999 + 1 = 1000$. Kết quả in ra: 1000.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1000 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $999 + 1 = 1000$. Kết quả in ra: 1000.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string addBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());

    string res = "";
    int carry = 0;
    int n = max(a.size(), b.size());

    for (int i = 0; i < n || carry; ++i) {
        int sum = carry;
        if (i < (int)a.size()) sum += a[i] - '0';
        if (i < (int)b.size()) sum += b[i] - '0';
        res.push_back((sum % 10) + '0');
        carry = sum / 10;
    }

    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    cout << addBig(a, b) << "\n";
    return 0;
}
```

Bài 03 [cppb_big_03_tru_hai_so_lon_khong_am]: Trừ Hai Số Nguyên Lớn ($A \geq B$)

Bối cảnh: Trong cơ chế cân bằng dự trữ vàng quốc gia, tổng khối lượng xuất kho ban đầu A và khối lượng tiêu hao B (với $A \geq B$) là các số nguyên không lồ. Kỹ sư cần tính khối lượng vàng còn lại sau khi trừ $A - B$.

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên dương lớn A và B ($A \geq B$). Hãy tính hiệu $A - B$.

Đầu vào (Input):

- Dòng 1: Chuỗi ký tự số A ($1 \leq |A| \leq 10^5$).
- Dòng 2: Chuỗi ký tự số B ($1 \leq |B| \leq |A|, A \geq B$).

Đầu ra (Output):

- In ra chuỗi số biểu diễn hiệu $A - B$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1000 1	999

Giải thích: $1000 - 1 = 999$. Kết quả in ra: 999.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 số nguyên dương lớn A và B ($A \geq B$). Hãy tính hiệu $A - B$.
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số `vector<int>` đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 1000 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 |
Thực thi thuật toán tối ưu | $1000 - 1 = 999$. Kết quả in ra: 999.... | Tính toán từng bước trạng thái | | 3 |
Xuất kết quả | Output: 999 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $1000 - 1 = 999$. Kết quả in ra: 999.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string subBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());

    string res = "";
    int borrow = 0;

    for (int i = 0; i < (int)a.size(); ++i) {
        int diff = (a[i] - '0') - borrow;
        if (i < (int)b.size()) diff -= (b[i] - '0');
        if (diff < 0) {
            diff += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res.push_back(diff + '0');
    }

    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    cout << subBig(a, b) << "\n";
    return 0;
}
```

Bài 04 [cppb_big_04_tru_hai_so_lon_tong_quat]: Trừ Hai Số Lớn Tổng Quát (Có Thể Âm)

Bối cảnh: Báo cáo cán cân thương mại quốc tế cần tính toán mức chênh lệch giữa kim ngạch xuất khẩu A và kim ngạch nhập khẩu B (cả A và B đều là số nguyên lớn). Nếu kim ngạch nhập khẩu lớn hơn xuất khẩu ($A < B$), cán cân sẽ thâm hụt và kết quả phải mang dấu âm '-'.
 Nếu kim ngạch xuất khẩu lớn hơn nhập khẩu ($A > B$), cán cân sẽ có thặng dư và kết quả phải mang dấu '+'.



Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên dương lớn A và B. Hãy tính hiệu A - B (in dấu '-' phía trước nếu kết quả mang giá trị âm).

Đầu vào (Input):

- Dòng 1: Chuỗi số A ($1 \leq |A| \leq 10^5$).
- Dòng 2: Chuỗi số B ($1 \leq |B| \leq 10^5$).

Đầu ra (Output):

- In ra kết quả phép trừ A - B.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1 1000	-999

Giải thích: $1 - 1000 = -999$. Kết quả in ra: -999.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho 2 số nguyên dương lớn A và B. Hãy tính hiệu A - B (in dấu '-' phía trước nếu kết quả mang giá trị âm).
- **Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 1 1000 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $1 - 1000 = -999$. Kết quả in ra: -999.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: -999 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $1 - 1000 = -999$. Kết quả in ra: -999.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool isLess(const string &a, const string &b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}

string subBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());

    string res = "";
    int borrow = 0;

    for (int i = 0; i < (int)a.size(); ++i) {
        int diff = (a[i] - '0') - borrow;
        if (i < (int)b.size()) diff -= (b[i] - '0');
        if (diff < 0) {
            diff += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res.push_back(diff + '0');
    }

    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    if (a == b) {
        cout << "0\n";
    } else if (isLess(a, b)) {
        cout << "-" << subBig(b, a) << "\n";
    } else {
        cout << subBig(a, b) << "\n";
    }
    return 0;
}
```

Bài 05 [cppb_big_05_nhan_so_lon_voi_so_nho]: Nhân Số Lớn Với Số Nhỏ

Bối cảnh: Một phòng thí nghiệm thiên văn học đo đặc chu kỳ quay của một chuẩn tinh với số lượng xung năng lượng là số nguyên lớn A , và mỗi xung kéo dài b nano giây ($b \leq 10^9$). Hãy tính tổng thời gian chính xác $A * b$.



Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho số nguyên lớn A và số nguyên nhỏ b ($0 \leq b \leq 10^9$). Hãy tính tích $A * b$.

Đầu vào (Input):

- Dòng 1: Chuỗi số A ($1 \leq |A| \leq 10^5$).
- Dòng 2: Số nguyên b ($0 \leq b \leq 10^9$).

Đầu ra (Output):

- In ra giá trị của $A \times b$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
123456789 5	617283945

Giải thích: $123456789 * 5 = 617283945$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên lớn A và số nguyên nhỏ b ($0 \leq b \leq 10^9$). Hãy tính tích $A * b$.
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 123456789 5 | Khởi tạo cấu trúc dữ liệu ban đầu | |
 2 | Thực thi thuật toán tối ưu | $123456789 * 5 = 617283945$ | Tính toán từng bước trạng thái | | 3 |
 Xuất kết quả | Output: 617283945 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $123456789 * 5 = 617283945$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string mulSmall(string a, long long b) {
    if (a == "0" || b == 0) return "0";

    reverse(a.begin(), a.end());
    string res = "";
    long long carry = 0;

    for (int i = 0; i < (int)a.size() || carry; ++i) {
        long long prod = carry;
        if (i < (int)a.size()) prod += 1LL * (a[i] - '0') * b;
        res.push_back((prod % 10) + '0');
        carry = prod / 10;
    }

    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a;
    long long b;
    if (!(cin >> a >> b)) return 0;

    cout << mulSmall(a, b) << "\n";
    return 0;
}
```

Bài 06 [cppb_big_06_nhan_hai_so_lon]: Nhân Hai Số Nguyên Lớn

Bối cảnh: Trong bài toán nhân đa thức và số học chính xác cao, việc tính tích của hai số nguyên khổng lồ A và B (mỗi số có tới hàng ngàn chữ số) là nền tảng cho việc giả lập các hằng số toán học vũ trụ.



Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên lớn A và B. Hãy tính tích $A * B$.

Đầu vào (Input):

- Dòng 1: Chuỗi số A ($1 \leq |A| \leq 3000$).
- Dòng 2: Chuỗi số B ($1 \leq |B| \leq 3000$).

Đầu ra (Output):

- In ra chuỗi số biểu diễn tích $A \times B$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
123 456	56088

Giải thích: $123 * 456 = 56088$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho 2 số nguyên lớn A và B. Hãy tính tích $A * B$.
- **Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 123 456 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $123 * 456 = 56088$... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 56088 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $123 * 456 = 56088$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string mulBig(string a, string b) {
    if (a == "0" || b == "0") return "0";

    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());

    int n = a.size(), m = b.size();
    vector<int> c(n + m, 0);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            c[i + j] += (a[i] - '0') * (b[j] - '0');
        }
    }

    int carry = 0;
    string res = "";
    for (int i = 0; i < n + m || carry; ++i) {
        if (i < (int)c.size()) carry += c[i];
        res.push_back((carry % 10) + '0');
        carry /= 10;
    }

    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    cout << mulBig(a, b) << "\n";
    return 0;
}
```

Bài 07 [cppb_big_07_chia_so_lon_cho_so_nho]: Chia Số Lớn Cho Số Nhỏ (Lấy Thương)

Bối cảnh: Một kho quỹ dự trữ có số tiền là số nguyên lớn A muốn chia đều cho b quỹ từ thiện xã hội ($b \leq 10^9$). Hãy tính số tiền chính xác (phần nguyên thương số) mà mỗi quỹ nhận được.



Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho số nguyên lớn A và số nguyên nhỏ b ($1 \leq b \leq 10^9$). Hãy tìm phần nguyên thương số $\text{floor}(A / b)$.

Đầu vào (Input):

- Dòng 1: Chuỗi số A ($1 \leq |A| \leq 10^5$).
- Dòng 2: Số nguyên dương b ($1 \leq b \leq 10^9$).

Đầu ra (Output):

- In ra chuỗi số là thương của phép chia.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1000 3	333

Giải thích: 1000 chia cho 3 được phần nguyên thương là 333.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên lớn A và số nguyên nhỏ b ($1 \leq b \leq 10^9$). Hãy tìm phần nguyên thương số $\text{floor}(A / b)$.
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 1000 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | 1000 chia cho 3 được phần nguyên thương là 333.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 333 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: 1000 chia cho 3 được phần nguyên thương là 333.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string divSmall(string a, long long b) {
    string res = "";
    long long cur = 0;

    for (char c : a) {
        cur = cur * 10 + (c - '0');
        res.push_back((cur / b) + '0');
        cur %= b;
    }

    int pos = 0;
    while (pos + 1 < (int)res.size() && res[pos] == '0') pos++;
    return res.substr(pos);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a;
    long long b;
    if (!(cin >> a >> b)) return 0;

    cout << divSmall(a, b) << "\n";
    return 0;
}
```

Bài 08 [cppb_big_08_chia_lay_du_so_lon_cho_so_nho]: Chia Lấy Dư Số Lớn Cho Số Nhỏ

Bối cảnh: Để băm (hashing) một chuỗi số lớn A có độ dài hàng trăm ngàn chữ số vào bảng băm kích thước b ($b \leq 10^{18}$), kỹ sư cần tính phần dư $A \bmod b$ trong thời gian $O(|A|)$.

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho số nguyên lớn A và số nguyên nhỏ b ($1 \leq b \leq 10^{18}$). Hãy tính $A \bmod b$.

Đầu vào (Input):

- Dòng 1: Chuỗi số A ($1 \leq |A| \leq 10^5$).
- Dòng 2: Số nguyên b ($1 \leq b \leq 10^{18}$).

Đầu ra (Output):

- In ra số dư $A \bmod b$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
123456789123456789 10	9

Giải thích: Chữ số tận cùng của A là 9 nên khi chia cho 10 phần dư là 9.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên lớn A và số nguyên nhỏ b ($1 \leq b \leq 10^{18}$). Hãy tính $A \bmod b$.
- **Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 123456789123456789 10 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Chữ số tận cùng của A là 9 nên khi chia cho 10 phần dư là 9... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 9 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chữ số tận cùng của A là 9 nên khi chia cho 10 phần dư là 9.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a;
    long long b;
    if (!(cin >> a >> b)) return 0;

    long long cur = 0;
    for (char c : a) {
        cur = (cur * 10 + (c - '0')) % b;
    }

    cout << cur << "\n";
    return 0;
}
```

Bài 09 [cppb_big_09_giai_thua_so_lon]: Tính Giai Thừa Số Lớn (N!)

Bối cảnh: Trong phân tích tổ hợp cấu hình ma trận gen, số lượng hoán vị của N đối tượng phân biệt là $N! = 1 * 2 * \dots * N$. Khi N đạt tới 1000, $N!$ có hàng ngàn chữ số. Hãy tính giá trị chính xác của $N!$.

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 1000$). Hãy in ra giá trị chính xác của $N!$.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 1000$).

Đầu ra (Output):

- In ra giá trị chính xác của $N!$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	120

Giải thích: $5! = 1 * 2 * 3 * 4 * 5 = 120$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 1000$). Hãy in ra giá trị chính xác của $N!$.
- **Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $5! = 1 * 2 * 3 * 4 * 5 = 120$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 120 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $5! = 1 * 2 * 3 * 4 * 5 = 120$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string mulSmall(string a, int b) {
    reverse(a.begin(), a.end());
    string res = "";
    int carry = 0;
    for (int i = 0; i < (int)a.size() || carry; ++i) {
        int prod = carry;
        if (i < (int)a.size()) prod += (a[i] - '0') * b;
        res.push_back((prod % 10) + '0');
        carry = prod / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n;
if (!(cin >> n)) return 0;

string ans = "1";
for (int i = 2; i <= n; ++i) {
    ans = mulSmall(ans, i);
}

cout << ans << "\n";
return 0;
}
```

Bài 10 [cppb_big_10_luy_thua_so_lon_chinh_xac]: Lũy Thừa Số Lớn Chính Xác (A^B)

Bối cảnh: Trong lý thuyết vũ trụ học đa chiều, số lượng trạng thái vi mô của một hố đen được mô hình hóa bởi biểu thức A^B . Hãy tính toán giá trị chính xác không làm tròn của A^B .

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên A và B ($1 \leq A \leq 1000, 0 \leq B \leq 1000$). Hãy in ra giá trị chính xác của A^B .

Đầu vào (Input):

- Một dòng chứa 2 số nguyên A và B ($1 \leq A \leq 1000, 0 \leq B \leq 1000$).

Đầu ra (Output):

- In ra giá trị chính xác của A^B .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 10	1024

Giải thích: $2^{10} = 1024$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 số nguyên A và B ($1 \leq A \leq 1000, 0 \leq B \leq 1000$). Hãy in ra giá trị chính xác của A^B .
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.

- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 2 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $2^{10} = 1024 \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1024 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $2^{10} = 1024$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string mulBig(string a, string b) {
    if (a == "0" || b == "0") return "0";
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());
    int n = a.size(), m = b.size();
    vector<int> c(n + m, 0);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            c[i + j] += (a[i] - '0') * (b[j] - '0');
        }
    }
    int carry = 0;
    string res = "";
    for (int i = 0; i < n + m || carry; ++i) {
        if (i < (int)c.size()) carry += c[i];
        res.push_back((carry % 10) + '0');
        carry /= 10;
    }
    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int a, b;
    if (!(cin >> a >> b)) return 0;

    string base = to_string(a);
    string ans = "1";

    while (b > 0) {
```

```

    if (b & 1) ans = mulBig(ans, base);
    base = mulBig(base, base);
    b >>= 1;
}

cout << ans << "\n";
return 0;
}

```

Bài 11 [cppb_big_11_fibonacci_so_lon]: Số Fibonacci Lớn Thứ N

Bối cảnh: Dãy số Fibonacci $F(N)$ biểu diễn quy luật sinh trưởng tự nhiên hình xoắn ốc của vỏ ốc anh vũ. Với $N = 1000$, $F(N)$ có hàng trăm chữ số. Hãy in ra giá trị chính xác của số Fibonacci thứ N .

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho số nguyên N ($0 \leq N \leq 1000$). Hãy in ra giá trị chính xác của $F(N)$ (với $F(0) = 0$, $F(1) = 1$, $F(N) = F(N-1) + F(N-2)$).

Đầu vào (Input):

- Một dòng chứa số nguyên N ($0 \leq N \leq 1000$).

Đầu ra (Output):

- In ra giá trị chính xác của F_N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10	55

Giải thích: Dãy Fibonacci: $F(0)=0$, $F(1)=1$, ..., $F(10)=55$. Kết quả in ra: 55.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên N ($0 \leq N \leq 1000$). Hãy in ra giá trị chính xác của $F(N)$ (với $F(0) = 0$, $F(1) = 1$, $F(N) = F(N-1) + F(N-2)$).
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Dãy Fibonacci: $F(0)=0, F(1)=1, \dots, F(10)=55$. Kết quả in ra: 55.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 55 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Dãy Fibonacci: $F(0)=0, F(1)=1, \dots, F(10)=55$. Kết quả in ra: 55.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string addBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());
    string res = "";
    int carry = 0;
    int n = max(a.size(), b.size());
    for (int i = 0; i < n || carry; ++i) {
        int sum = carry;
        if (i < (int)a.size()) sum += a[i] - '0';
        if (i < (int)b.size()) sum += b[i] - '0';
        res.push_back((sum % 10) + '0');
        carry = sum / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n == 0) { cout << "0\n"; return 0; }
    if (n == 1) { cout << "1\n"; return 0; }

    string f0 = "0", f1 = "1", f2 = "";
    for (int i = 2; i <= n; ++i) {
        f2 = addBig(f0, f1);
        f0 = f1;
        f1 = f2;
    }

    cout << f1 << "\n";
}
```

```
return 0;
}
```

Bài 12 [cppb_big_12_tong_chu_so_giai_thua]: Tổng Các Chữ Số Của $N!$

Bối cảnh: Trong các cuộc thi Olympic Toán - Tin học quốc tế, bài toán tính tổng tất cả các chữ số trong biểu diễn thập phân của giai thừa $N!$ là một thử thách kinh điển đòi hỏi kết hợp xử lý BigInt và cộng dồn chữ số.

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho số nguyên N ($1 \leq N \leq 1000$). Hãy tính tổng tất cả các chữ số của $N!$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên N ($1 \leq N \leq 1000$).

Đầu ra (Output):

- In ra tổng các chữ số của $N!$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10	27

Giải thích: $10! = 3628800$. Tổng các chữ số là: $3 + 6 + 2 + 8 + 8 + 0 + 0 = 27$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên N ($1 \leq N \leq 1000$). Hãy tính tổng tất cả các chữ số của $N!$.
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số `vector<int>` đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 10 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $10! = 3628800$. Tổng các chữ số là: $3 + 6 + 2 + 8 + 8 + 0 + 0 = 27 \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 27 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $10! = 3628800$. Tổng các chữ số là: $3 + 6 + 2 + 8 + 8 + 0 + 0 = 27$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string mulSmall(string a, int b) {
    reverse(a.begin(), a.end());
    string res = "";
    int carry = 0;
    for (int i = 0; i < (int)a.size() || carry; ++i) {
        int prod = carry;
        if (i < (int)a.size()) prod += (a[i] - '0') * b;
        res.push_back((prod % 10) + '0');
        carry = prod / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    string fact = "1";
    for (int i = 2; i <= n; ++i) {
        fact = mulSmall(fact, i);
    }

    long long sum_digits = 0;
    for (char c : fact) {
        sum_digits += (c - '0');
    }

    cout << sum_digits << "\n";
    return 0;
}
```

Bài 13 [cppb_big_13_chia_hai_so_nguyen_lon]: Chia Hai Số Nguyên Lớn (A / B)

Bối cảnh: Phép chia hai số nguyên lớn A / B là phép toán phức tạp nhất trong thư viện `BigInt` chuẩn, đòi hỏi kỹ thuật tìm kiếm nhị phân chữ số thương kết hợp nhân số lớn để tìm phần nguyên thương số.

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên dương lớn A và B. Hãy tìm phần nguyên thương số $\text{floor}(A / B)$.

Đầu vào (Input):

- Dòng 1: Chuỗi số A ($1 \leq |A| \leq 1000$).
- Dòng 2: Chuỗi số B ($1 \leq |B| \leq 1000$).

Đầu ra (Output):

- In ra phần nguyên thương số $[A/B]$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
100 3	33

Giải thích: $100 / 3 = 33$ (dư 1). Phần nguyên thương là 33.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 số nguyên dương lớn A và B. Hãy tìm phần nguyên thương số $\text{floor}(A / B)$.
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 100 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $100 / 3 = 33$ (dư 1). Phần nguyên thương là 33.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 33 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $100 / 3 = 33$ (dư 1). Phần nguyên thương là 33.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool isLess(const string &a, const string &b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}

string subBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());
    string res = "";
    int borrow = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        int diff = (a[i] - '0') - borrow;
        if (i < (int)b.size()) diff -= (b[i] - '0');
        if (diff < 0) { diff += 10; borrow = 1; }
        else borrow = 0;
        res.push_back(diff + '0');
    }
    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    if (isLess(a, b)) {
        cout << "0\n";
        return 0;
    }

    string cur = "";
    string res = "";

    for (char c : a) {
        cur.push_back(c);
        while (cur.size() > 1 && cur[0] == '0') cur.erase(cur.begin());
        int digit = 0;
        while (!isLess(cur, b)) {
            cur = subBig(cur, b);
            digit++;
        }
        res.push_back(digit + '0');
    }

    int pos = 0;
    while (pos + 1 < (int)res.size() && res[pos] == '0') pos++;
    cout << res.substr(pos) << "\n";
}
```

```
return 0;
}
```

Bài 14 [cppb_big_14_can_bac_hai_so_lon]: Căn Bậc Hai Số Nguyên Lớn

Bối cảnh: Để giải mã mật mã RSA khi biết giá trị tích hai số nguyên tố $N = p * q$ xấp xỉ nhau, việc tính phần nguyên căn bậc hai của số nguyên lớn A là bước then chốt.

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho số nguyên dương lớn A (có tới 1000 chữ số). Hãy tìm phần nguyên căn bậc hai $\text{floor}(\sqrt{A})$.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi số A ($1 \leq |A| \leq 1000$).

Đầu ra (Output):

- In ra phần nguyên căn bậc hai của A .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
17	4

Giải thích: $\sqrt{17} = 4.123\dots$ Phần nguyên là 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương lớn A (có tới 1000 chữ số). Hãy tìm phần nguyên căn bậc hai $\text{floor}(\sqrt{A})$.
- Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số `vector<int>` đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 17 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $\sqrt{17} = 4.123\dots$ Phần nguyên là 4.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $\sqrt{17} = 4.123\dots$ Phần nguyên là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool isLess(const string &a, const string &b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}

string mulBig(string a, string b) {
    if (a == "0" || b == "0") return "0";
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());
    int n = a.size(), m = b.size();
    vector<int> c(n + m, 0);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            c[i + j] += (a[i] - '0') * (b[j] - '0');
        }
    }
    int carry = 0;
    string res = "";
    for (int i = 0; i < n + m || carry; ++i) {
        if (i < (int)c.size()) carry += c[i];
        res.push_back((carry % 10) + '0');
        carry /= 10;
    }
    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

string addBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());
    string res = "";
    int carry = 0;
    int n = max(a.size(), b.size());
    for (int i = 0; i < n || carry; ++i) {
        int sum = carry;
```

```

        if (i < (int)a.size()) sum += a[i] - '0';
        if (i < (int)b.size()) sum += b[i] - '0';
        res.push_back((sum % 10) + '0');
        carry = sum / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}

string div2(string a) {
    string res = "";
    int cur = 0;
    for (char c : a) {
        cur = cur * 10 + (c - '0');
        res.push_back((cur / 2) + '0');
        cur %= 2;
    }
    int pos = 0;
    while (pos + 1 < (int)res.size() && res[pos] == '0') pos++;
    return res.substr(pos);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a;
    if (!(cin >> a)) return 0;

    int target_len = (a.size() + 1) / 2;
    string low = "1";
    string high = string(target_len + 1, '9');
    string ans = "1";

    while (!isLess(high, low)) {
        string mid = div2(addBig(low, high));
        string sq = mulBig(mid, mid);
        if (!isLess(a, sq)) {
            ans = mid;
            low = addBig(mid, "1");
        } else {
            // high = mid - 1
            // vì low, high chỉ dùng cho binary search chuỗi
            int borrow = 1;
            for (int i = (int)mid.size() - 1; i >= 0; --i) {
                if (mid[i] >= '1') { mid[i]--; break; }
                else mid[i] = '9';
            }
            while (mid.size() > 1 && mid[0] == '0') mid.erase(mid.begin());
            high = mid;
        }
    }

    cout << ans << "\n";
    return 0;
}

```

Bài 15 [cppb_big_15_uoc_chung_lon_nhat_so_lon]: Ước Chung Lớn Nhất Số Lớn

Bối cảnh: Trong việc rút gọn phân số chứa các hệ số siêu lớn, tìm ước chung lớn nhất $\gcd(A, B)$ giữa hai số nguyên lớn A và B bằng thuật toán Euclid kết hợp chia lấy dư BigInt là thao tác không thể thiếu.



Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên dương lớn A và B. Hãy tìm ước chung lớn nhất gcd(A, B).

Đầu vào (Input):

- Dòng 1: Chuỗi số A ($1 \leq |A| \leq 1000$).
- Dòng 2: Chuỗi số B ($1 \leq |B| \leq 1000$).

Đầu ra (Output):

- In ra gcd(A, B).

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12 18	6

Giải thích: gcd(12, 18) = 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho 2 số nguyên dương lớn A và B. Hãy tìm ước chung lớn nhất gcd(A, B).
- **Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 12 18 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | gcd(12, 18) = 6... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 6 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: gcd(12, 18) = 6.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

bool isLess(const string &a, const string &b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}

string subBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());
    string res = "";
    int borrow = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        int diff = (a[i] - '0') - borrow;
        if (i < (int)b.size()) diff -= (b[i] - '0');
        if (diff < 0) { diff += 10; borrow = 1; }
        else borrow = 0;
        res.push_back(diff + '0');
    }
    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

string div2(string a) {
    string res = "";
    int cur = 0;
    for (char c : a) {
        cur = cur * 10 + (c - '0');
        res.push_back((cur / 2) + '0');
        cur %= 2;
    }
    int pos = 0;
    while (pos + 1 < (int)res.size() && res[pos] == '0') pos++;
    return res.substr(pos);
}

string mul2(string a) {
    reverse(a.begin(), a.end());
    string res = "";
    int carry = 0;
    for (int i = 0; i < (int)a.size() || carry; ++i) {
        int prod = carry;
        if (i < (int)a.size()) prod += (a[i] - '0') * 2;
        res.push_back((prod % 10) + '0');
        carry = prod / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}

bool isEven(const string &s) {
    return (s.back() - '0') % 2 == 0;
}
```

```

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    int shift = 0;
    while (a != "0" && b != "0") {
        if (isEven(a) && isEven(b)) {
            shift++;
            a = div2(a);
            b = div2(b);
        } else if (isEven(a)) {
            a = div2(a);
        } else if (isEven(b)) {
            b = div2(b);
        } else {
            if (isLess(a, b)) b = subBig(b, a);
            else a = subBig(a, b);
        }
    }

    string ans = (a == "0" ? b : a);
    while (shift--) ans = mul2(ans);

    cout << ans << "\n";
    return 0;
}

```

Bài 16 [cppb_big_16_to_hop_so_lon_chinh_xac]: Số Lớn Cực Hạn: Tổ Hợp C(N, K) Chính Xác

Bối cảnh: Khi cần tính số cách chia tổ hợp C(N, K) chính xác tuyệt đối mà không áp dụng modulo, việc giản ước thừa số chung giữa tử số và mẫu số kết hợp nhân BigInt cho phép tính chính xác đến chữ số cuối cùng.

ĐẶT TÍNH CỘNG HAI SỐ NGUYÊN LỚN TỪNG CHỮ SỐ

Nhớ 1 sang hàng kế tiếp

9	8	7	6	5	4	3	2	1	
8	5	4	3	2	1	9	9	9	
1	8	4	1	9	7	6	3	2	0

Mô phỏng phép cộng tiểu học từ phải sang trái kết hợp biến nhớ carry

Hình minh họa: Đặt tính và xử lý từng chữ số cho số nguyên lớn vượt 64-bit

Nhiệm vụ: Cho 2 số nguyên N và K. Hãy tính giá trị chính xác tuyệt đối của $C(N, K) = N! / (K! * (N - K)!)$.

Đầu vào (Input):

- Một dòng chứa 2 số nguyên N và K ($0 \leq K \leq N \leq 200$).

Đầu ra (Output):

- In ra giá trị chính xác tuyệt đối của $C(N, K)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2	10

Giải thích: $C(5, 2) = 10$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho 2 số nguyên N và K . Hãy tính giá trị chính xác tuyệt đối của $C(N, K) = N! / (K! * (N - K)!)$.
- **Phương pháp tiếp cận — Xử lý số nguyên lớn (BigInt):**
- Biểu diễn số lớn bằng chuỗi ký tự string hoặc mảng các chữ số vector<int> đảo ngược.
- Mô phỏng các phép tính cộng, trừ, nhân, chia bằng thuật toán đặt tính từng cột như tiểu học.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $C(5, 2) = 10 \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 10 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $C(5, 2) = 10$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

string addBig(string a, string b) {
    reverse(a.begin(), a.end());
    reverse(b.begin(), b.end());
    string res = "";
    int carry = 0;
    int n = max(a.size(), b.size());
    for (int i = 0; i < n || carry; ++i) {
        int sum = carry;
        if (i < (int)a.size()) sum += a[i] - '0';
        if (i < (int)b.size()) sum += b[i] - '0';
```

```
        res.push_back((sum % 10) + '0');
        carry = sum / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<vector<string>> c(n + 1, vector<string>(k + 1, "0"));
    for (int i = 0; i <= n; ++i) {
        c[i][0] = "1";
        for (int j = 1; j <= min(i, k); ++j) {
            if (j == i) c[i][j] = "1";
            else c[i][j] = addBig(c[i - 1][j - 1], c[i - 1][j]);
        }
    }

    cout << c[n][k] << "\n";
    return 0;
}
```



CHƯƠNG 05: ĐỆ QUY, CHIA ĐỂ TRỊ & QUAY LUI

Bài 13: Thuật toán đệ quy & cây gọi hàm

Bài 01 [cppb_rec_01_in_day_so_de_quy]: In Dãy Số Đệ Quy 1..N và N..1

Bối cảnh: Để làm quen với cơ chế hoạt động của ngăn xếp cuộc gọi hàm (Call Stack), một huấn luyện viên lập trình yêu cầu học viên viết hàm đệ quy in ra dãy số từ 1 đến N (theo chiều xuôi) và từ N về 1 (theo chiều ngược) mà tuyệt đối không được sử dụng bất kỳ vòng lặp for hay while nào.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên dương N. Hãy in ra 2 dòng: dòng 1 in các số từ 1 đến N, dòng 2 in các số từ N về 1 bằng hàm đệ quy.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 1000$).

Đầu ra (Output):

- Dòng 1: In các số từ 1 đến N cách nhau bởi khoảng trắng.
- Dòng 2: In các số từ N về 1 cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	1 2 3 4 5 5 4 3 2 1

Giải thích: - Dòng 1 in xuôi từ 1 đến 5: 1 2 3 4 5. - Dòng 2 in ngược từ 5 về 1: 5 4 3 2 1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N. Hãy in ra 2 dòng: dòng 1 in các số từ 1 đến N, dòng 2 in các số từ N về 1 bằng hàm đệ quy.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - Dòng 1 in xuôi từ 1 đến 5: 1 2 3 4 5. - Dòng 2 in ngược từ 5 về 1: 5 4 3 2 1.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 2 3 4 5 5 4 3 2 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Dòng 1 in xuôi từ 1 đến 5: 1 2 3 4 5.

- Dòng 2 in ngược từ 5 về 1: 5 4 3 2 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

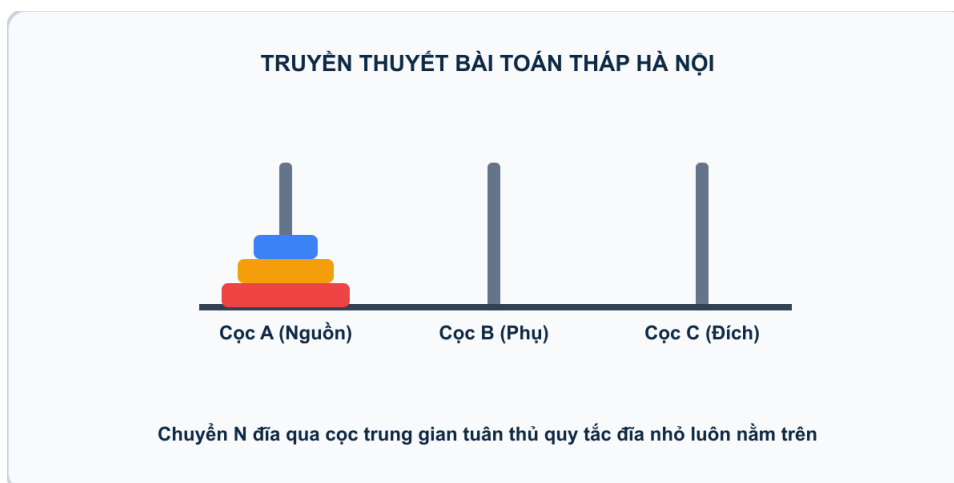
void printForward(int n) {
    if (n <= 0) return;
    printForward(n - 1);
    cout << n << " ";
}

void printBackward(int n) {
    if (n <= 0) return;
    cout << n << " ";
    printBackward(n - 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    printForward(n);
    cout << "\n";
    printBackward(n);
    cout << "\n";
    return 0;
}
```

Bài 02 [cppb_rec_02_tong_day_so_va_giai_thua]: Tính Tổng Dãy Số & Giai Thừa Bằng Đệ Quy

Bối cảnh: Hai công thức toán học cơ bản nhất phản ánh tính chất đệ quy tự nhiên: tổng dãy số nguyên liên tiếp $S(N) = S(N - 1) + N$ và giai thừa $N! = (N - 1)! * N$. Hãy cài đặt hai hàm đệ quy này để tính toán giá trị đồng thời.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 20$). Hãy tính tổng $S = 1 + 2 + \dots + N$ và giai thừa $N!$ bằng hàm đệ quy.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 20$).

Đầu ra (Output):

- In ra 2 số nguyên cách nhau bởi khoảng trắng: tổng S và giai thừa $N!$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	15 120

Giải thích: $S(5) = 1 + 2 + 3 + 4 + 5 = 15$; $5! = 1 * 2 * 3 * 4 * 5 = 120$. Kết quả in ra: 15 120.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 20$). Hãy tính tổng $S = 1 + 2 + \dots + N$ và giai thừa $N!$ bằng hàm đệ quy.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $S(5) = 1 + 2 + 3 + 4 + 5 = 15$; $5! = 1 * 2 * 3 * 4 * 5 = 120$. Kết quả in ra: 15 120.... |
 Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 15 120 | Khớp chính xác với đầu ra mẫu |
Giải thích chi tiết từ mẫu: $S(5) = 1 + 2 + 3 + 4 + 5 = 15$; $5! = 1 * 2 * 3 * 4 * 5 = 120$. Kết quả in ra: 15 120.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

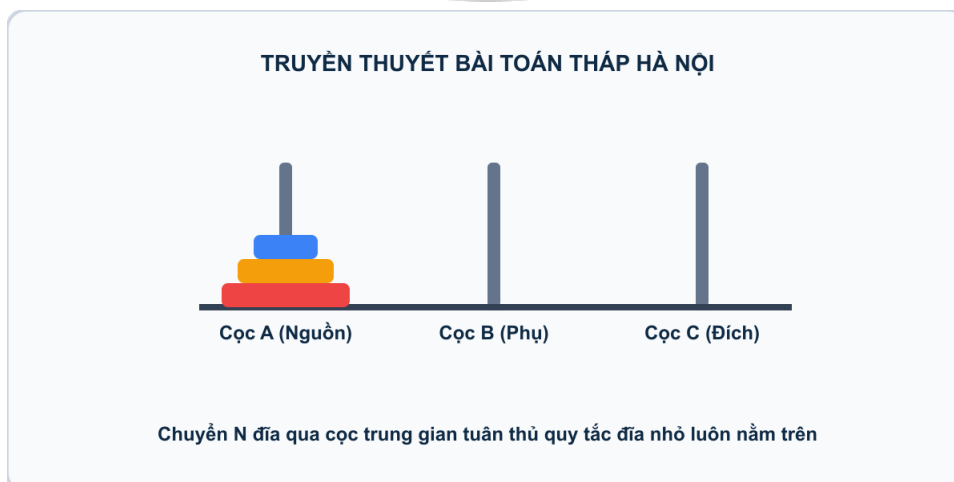
long long getSum(int n) {
    if (n <= 1) return n;
    return n + getSum(n - 1);
}

long long getFact(int n) {
    if (n <= 1) return 1;
    return 1LL * n * getFact(n - 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    cout << getSum(n) << " " << getFact(n) << "\n";
    return 0;
}
```

Bài 03 [cppb_rec_03_dem_va_tinh_tong_chu_so]: Đếm & Tính Tổng Chữ Số Của N Bằng Đệ Quy

Bối cảnh: Trong xử lý số học đệ quy, việc phân tách chữ số hàng đơn vị $N \% 10$ và phần còn lại $N / 10$ cho phép duyệt qua toàn bộ các chữ số của N một cách thanh lịch mà không cần chuyển đổi sang chuỗi ký tự.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Hãy viết hàm đệ quy để đếm số lượng chữ số và tính tổng các chữ số của N .

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra 2 số nguyên cách nhau bởi khoảng trắng: số lượng chữ số và tổng các chữ số.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12345	5 15

Giải thích: Số 12345 có 5 chữ số, tổng các chữ số là $1 + 2 + 3 + 4 + 5 = 15$. Kết quả in ra: 5 15.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 10^{18}$). Hãy viết hàm đệ quy để đếm số lượng chữ số và tính tổng các chữ số của N .
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 12345 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Số 12345 có 5 chữ số, tổng các chữ số là $1 + 2 + 3 + 4 + 5 = 15$. Kết quả in ra: 5 15.... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 5 15 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số 12345 có 5 chữ số, tổng các chữ số là $1 + 2 + 3 + 4 + 5 = 15$. Kết quả in ra: 5 15.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int countDigits(long long n) {
    if (n < 10) return 1;
    return 1 + countDigits(n / 10);
}
```

```

}

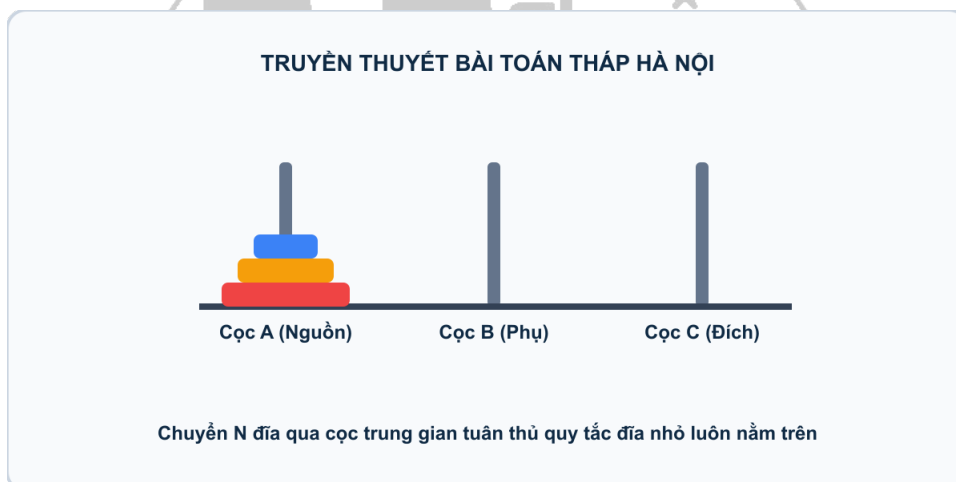
long long sumDigits(long long n) {
    if (n < 10) return n;
    return (n % 10) + sumDigits(n / 10);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long n;
    if (!(cin >> n)) return 0;
    cout << countDigits(n) << " " << sumDigits(n) << "\n";
    return 0;
}

```

Bài 04 [cppb_rec_04_dao_nguoc_mang_de_quy]: Đảo Ngược Mảng Bằng Độ Quy Hai Con Trỏ

Bối cảnh: Một thiết bị lưu trữ dữ liệu cảm biến cần đảo chiều một mảng bộ nhớ gồm N phần tử. Kỹ sư hệ thống muốn cài đặt hàm đệ quy đảo ngược mảng theo nguyên lý hai con trỏ đối đầu (hoán đổi $A[L]$ và $A[R]$ rồi đệ quy vào $[L + 1, R - 1]$).



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho mảng số nguyên A gồm N phần tử. Hãy sử dụng hàm đệ quy 2 con trỏ `reverseArray(A, L, R)` để đảo ngược toàn bộ mảng tại chỗ.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 1000$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra mảng sau khi đảo ngược trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 2 3 4 5	5 4 3 2 1

Giải thích: Mảng ban đầu $[1, 2, 3, 4, 5]$ sau khi đảo ngược trở thành $[5, 4, 3, 2, 1]$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng số nguyên A gồm N phần tử. Hãy sử dụng hàm đệ quy 2 con trỏ reverseArray(A, L, R) để đảo ngược toàn bộ mảng tại chỗ.
- **Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 1 2 3 4 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Mảng ban đầu [1, 2, 3, 4, 5] sau khi đảo ngược trở thành [5, 4, 3, 2, 1].... |
 Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 5 4 3 2 1 | Khớp chính xác với đầu ra mẫu |
 Giải thích chi tiết từ mẫu: Mảng ban đầu [1, 2, 3, 4, 5] sau khi đảo ngược trở thành [5, 4, 3, 2, 1].

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng ios::sync_with_stdio(false); cin.tie(nullptr); ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

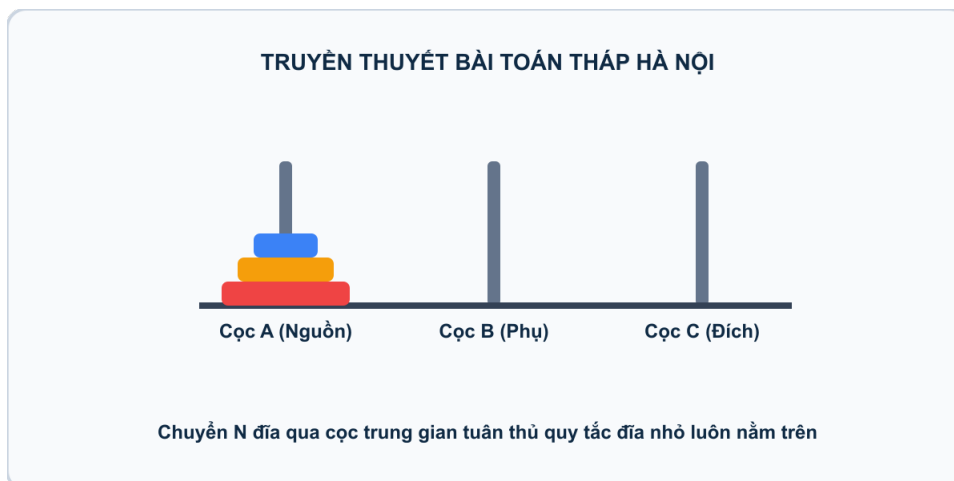
```
#include <bits/stdc++.h>
using namespace std;

void reverseRec(vector<long long> &a, int l, int r) {
    if (l >= r) return;
    swap(a[l], a[r]);
    reverseRec(a, l + 1, r - 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    reverseRec(a, 0, n - 1);
    for (int i = 0; i < n; ++i) cout << a[i] << (i + 1 == n ? "" : " ");
    cout << "\n";
    return 0;
}
```

Bài 05 [cppb_rec_05_kiem_tra_palindrome_de_quy]: Kiểm Tra Chuỗi Palindrome Bằng Đệ Quy

Bối cảnh: Trong phân tích chuỗi đối xứng đối xứng sinh học DNA, một chuỗi được gọi là Palindrome nếu đọc xuôi hay đọc ngược đều hoàn toàn như nhau. Cơ chế đệ quy so sánh ký tự đầu và cuối $S[L] == S[R]$ rồi thu hẹp chuỗi con bên trong là phương pháp chuẩn xác.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho chuỗi ký tự S gồm các chữ cái in thường. Hãy viết hàm đệ quy $\text{isPalindrome}(S, L, R)$ kiểm tra chuỗi S có phải là Palindrome không. In YES nếu đúng, ngược lại in NO.

Đầu vào (Input):

- Một dòng chứa chuỗi ký tự S ($1 \leq |S| \leq 1000$).

Đầu ra (Output):

- In ra YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
radar	YES

Giải thích: Chuỗi 'radar' đọc xuôi hay đọc ngược đều là 'radar' nên là chuỗi Palindrome -> in YES.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho chuỗi ký tự S gồm các chữ cái in thường. Hãy viết hàm đệ quy $\text{isPalindrome}(S, L, R)$ kiểm tra chuỗi S có phải là Palindrome không. In YES nếu đúng, ngược lại in NO.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| 1 | Nạp dữ liệu vào mảng/biến | Input: radar | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Chuỗi 'radar' đọc xuôi hay đọc ngược đều là 'radar' nên là chuỗi Palindrome -> in

YES.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Xâu ‘radar’ đọc xuôi hay đọc ngược đều là ‘radar’ nên là xâu Palindrome -> in YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

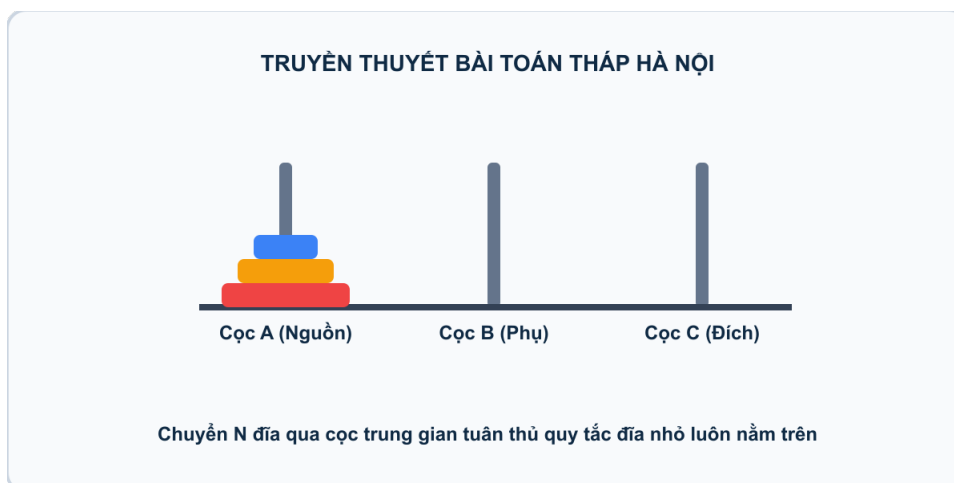
```
#include <bits/stdc++.h>
using namespace std;

bool isPalindromeRec(const string &s, int l, int r) {
    if (l >= r) return true;
    if (s[l] != s[r]) return false;
    return isPalindromeRec(s, l + 1, r - 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    if (!(cin >> s)) return 0;
    if (isPalindromeRec(s, 0, (int)s.size() - 1)) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }
    return 0;
}
```

Bài 06 [cppb_rec_06_tim_min_max_de_quy]: So Sánh Độ Quy Tuyến Tính & Chia Đôi Khi Tìm Min/Max

Bối cảnh: Để minh họa sự khác biệt về độ sâu ngăn xếp cuộc gọi giữa đệ quy tuyến tính $O(N)$ và đệ quy chia đôi (Binary Recursion) $O(\log N)$, hãy cài đặt hàm đệ quy chia đôi mảng thành 2 nửa để tìm đồng thời giá trị nhỏ nhất và lớn nhất của mảng N phần tử.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho mảng số nguyên A gồm N phần tử. Hãy cài đặt hàm đệ quy chia đôi để tìm giá trị nhỏ nhất (min) và lớn nhất (max) trong mảng.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra 2 số nguyên cách nhau bởi khoảng trắng: giá trị min và giá trị max.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 9 4 2	1 9

Giải thích: Giá trị nhỏ nhất trong mảng là 1, giá trị lớn nhất là 9. Kết quả in ra: 1 9.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng số nguyên A gồm N phần tử. Hãy cài đặt hàm đệ quy chia đôi để tìm giá trị nhỏ nhất (min) và lớn nhất (max) trong mảng.
- **Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 3 1 9 4 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Giá trị nhỏ nhất trong mảng là 1, giá trị lớn nhất là 9. Kết quả in ra: 1 9.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 9 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Giá trị nhỏ nhất trong mảng là 1, giá trị lớn nhất là 9. Kết quả in ra: 1 9.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

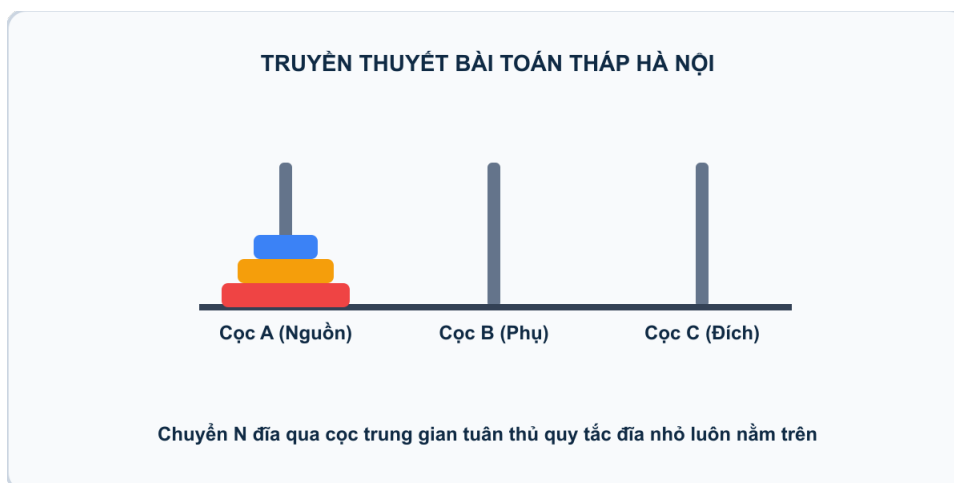
long long getMinRec(const vector<long long> &a, int l, int r) {
    if (l == r) return a[l];
    int mid = l + (r - l) / 2;
    return min(getMinRec(a, l, mid), getMinRec(a, mid + 1, r));
}

long long getMaxRec(const vector<long long> &a, int l, int r) {
    if (l == r) return a[l];
    int mid = l + (r - l) / 2;
    return max(getMaxRec(a, l, mid), getMaxRec(a, mid + 1, r));
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    cout << getMinRec(a, 0, n - 1) << " " << getMaxRec(a, 0, n - 1) << "\n";
    return 0;
}
```

Bài 07 [cppb_rec_07_uoc_chung_lon_nhat_de_quy]: Thuật Toán Euclid Tính GCD & LCM Bằng Đệ Quy

Bối cảnh: Thuật toán Euclid cổ đại phát biểu dưới dạng đệ quy một dòng $\text{gcd}(A, B) = (B == 0) A : \text{gcd}(B, A \% B)$ là một trong những thuật toán đẹp đẽ và hiệu quả nhất lịch sử toán học. Hãy dùng hàm đệ quy này để tính ước chung lớn nhất và bội chung nhỏ nhất.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho 2 số nguyên dương A, B . Hãy tính ước chung lớn nhất $\text{gcd}(A, B)$ và bội chung nhỏ nhất $\text{lcm}(A, B)$ bằng hàm đệ quy Euclid.

Đầu vào (Input):

- Một dòng chứa 2 số nguyên dương A và B ($1 \leq A, B \leq 10^{12}$).

Đầu ra (Output):

- In ra $\text{gcd}(A, B)$ và $\text{lcm}(A, B)$ cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
24 36	12 72

Giải thích: $\text{gcd}(24, 36) = 12$ và $\text{lcm}(24, 36) = (24 * 36) / 12 = 72$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 2 số nguyên dương A, B . Hãy tính ước chung lớn nhất $\text{gcd}(A, B)$ và bội chung nhỏ nhất $\text{lcm}(A, B)$ bằng hàm đệ quy Euclid.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 24 36 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $\text{gcd}(24, 36) = 12$ và $\text{lcm}(24, 36) = (24 * 36) / 12 = 72$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 12 72 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $\text{gcd}(24, 36) = 12$ và $\text{lcm}(24, 36) = (24 * 36) / 12 = 72$.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long gcdRec(long long a, long long b) {
    if (b == 0) return a;
    return gcdRec(b, a % b);
}

void print128(__int128 n) {
    if (n == 0) { cout << 0; return; }
    string s = "";
    while (n > 0) {
        s.push_back(char('0' + (n % 10)));
        n /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long a, b;
    if (!(cin >> a >> b)) return 0;
    long long g = gcdRec(a, b);
    __int128 lcm = ((__int128)a / g) * b;
    cout << g << " ";
    print128(lcm);
    cout << "\n";
    return 0;
}
```

Bài 08 [cppb_rec_08_luy_thua_nhi_phan_de_quy]: Lũy Thừa Nhị Phân Độ Quy $A^B \bmod M$

Bối cảnh: Bản chất toán học của lũy thừa nhị phân là hệ thức đệ quy: nếu B chẵn thì $A^B = (A^{B/2})^2$, nếu B lẻ thì $A^B = A * A^{(B-1)}$. Hãy cài đặt thuật toán lũy thừa nhị phân bằng hàm đệ quy thuần túy.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho 3 số nguyên A, B, M . Hãy tính $A^B \bmod M$ bằng hàm đệ quy.

Đầu vào (Input):

- Một dòng chứa 3 số nguyên A, B, M ($0 \leq A, B \leq 10^{18}, 1 \leq M \leq 10^9$).

Đầu ra (Output):

- In ra giá trị $A^B \bmod M$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 5 100	43

Giải thích: $3^5 = 243$. $243 \bmod 100 = 43$. Kết quả in ra: 43.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 3 số nguyên A, B, M . Hãy tính $A^B \bmod M$ bằng hàm đệ quy.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 5 100 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $3^5 = 243$. $243 \bmod 100 = 43$. Kết quả in ra: 43.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 43 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $3^5 = 243$. $243 \bmod 100 = 43$. Kết quả in ra: 43.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.

- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

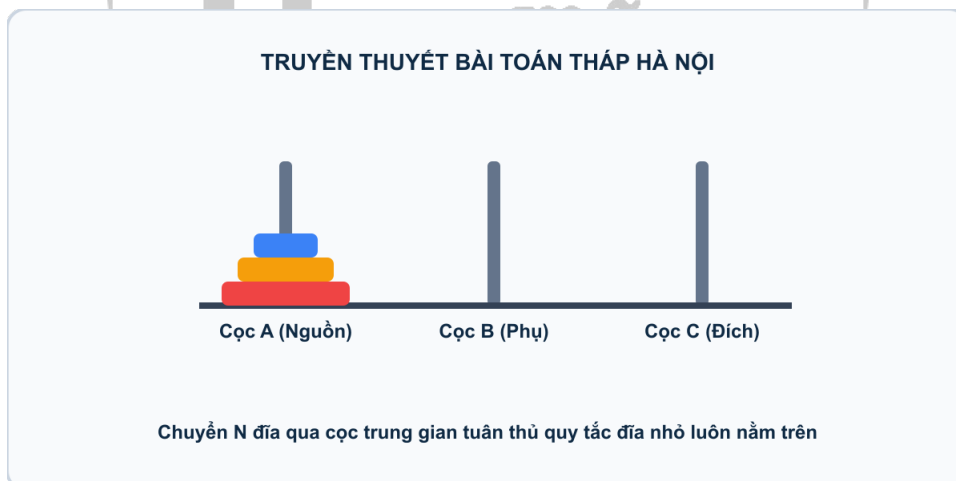
```
#include <bits/stdc++.h>
using namespace std;

long long powerRec(long long a, long long b, long long m) {
    if (b == 0) return 1 % m;
    long long half = powerRec(a, b / 2, m);
    long long res = (1LL * (half % m) * (half % m)) % m;
    if (b % 2 == 1) res = (1LL * res * (a % m)) % m;
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long a, b, m;
    if (!(cin >> a >> b >> m)) return 0;
    cout << powerRec(a, b, m) << "\n";
    return 0;
}
```

Bài 09 [cppb_rec_09_thap_ha_noi_co_ban]: Bài Toán Tháp Hà Nội (Tower of Hanoi)

Bối cảnh: Bài toán truyền thuyết cổ về các nhà sư chuyển N chiếc đĩa vàng giữa 3 cọc A, B, C tuân theo quy tắc: mỗi lần chỉ chuyển 1 đĩa và đĩa lớn hơn không bao giờ được đặt lên trên đĩa nhỏ hơn. Đây là bài toán mẫu mực kinh điển về tư duy đệ quy phân rã bài toán.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho N đĩa đặt trên cọc A. Hãy in ra số bước chuyển tối thiểu và danh sách các bước di chuyển đĩa từ cọc này sang cọc khác để chuyển hết N đĩa từ cọc A sang cọc C (dùng cọc B làm trung gian).

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 15$).

Đầu ra (Output):

- Dòng 1: In số bước di chuyển tối thiểu $2^N - 1$.
- Các dòng tiếp theo: Mỗi dòng in một bước chuyển dạng $X \rightarrow Y$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2	3 A -> B A -> C B -> C

Giải thích: Với $N = 2$ cần $2^2 - 1 = 3$ bước: chuyển đĩa 1 từ A sang B; chuyển đĩa 2 từ A sang C; chuyển đĩa 1 từ B sang C.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N đĩa đặt trên cọc A. Hãy in ra số bước chuyển tối thiểu và danh sách các bước di chuyển đĩa từ cọc này sang cọc khác để chuyển hết N đĩa từ cọc A sang cọc C (dùng cọc B làm trung gian).
- **Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với $N = 2$ cần $2^2 - 1 = 3$ bước: chuyển đĩa 1 từ A sang B; chuyển đĩa 2 từ A sang C; chuyển đĩa 1 từ B sang C... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 A -> B A -> C B -> C | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với $N = 2$ cần $2^2 - 1 = 3$ bước: chuyển đĩa 1 từ A sang B; chuyển đĩa 2 từ A sang C; chuyển đĩa 1 từ B sang C.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

void solveHanoi(int n, char from, char to, char aux) {
    if (n == 0) return;
    solveHanoi(n - 1, from, aux, to);
    cout << from << " -> " << to << "\n";
    solveHanoi(n - 1, aux, to, from);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n;
if (!(cin >> n)) return 0;
cout << (1 << n) - 1 << "\n";
solveHanoi(n, 'A', 'C', 'B');
return 0;
}

```

Bài 10 [cppb_rec_10_fibonacci_cay_nhi_phan]: Dãy Fibonacci Đệ Quy & Khảo Sát Cây Gọi Hàm

Bối cảnh: Để khảo sát sự bùng nổ theo cấp số nhân của cây gọi hàm đệ quy không nhớ (Exponential Call Tree), hãy viết hàm đệ quy tính số Fibonacci thứ N và đếm tổng số lần hàm fibonacci() được gọi thực thi.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên N ($0 \leq N \leq 30$). Hãy tính giá trị F_N và đếm tổng số lần gọi hàm fibonacci() trong toàn bộ quá trình thực thi.

Đầu vào (Input):

- Một dòng chứa số nguyên N ($0 \leq N \leq 30$).

Đầu ra (Output):

- In ra 2 số nguyên cách nhau bởi khoảng trắng: giá trị F_N và tổng số lần gọi hàm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	3 9

Giải thích: $F(4) = 3$. Cây gọi hàm fib(4) gồm 9 lần gọi hàm: fib(4) gọi fib(3) và fib(2); fib(3) gọi fib(2) và fib(1); v.v. Tổng số lần gọi là 9.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên N ($0 \leq N \leq 30$). Hãy tính giá trị F_N và đếm tổng số lần gọi hàm fibonacci() trong toàn bộ quá trình thực thi.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.

- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $F(4) = 3$. Cây gọi hàm fib(4) gồm 9 lần gọi hàm: fib(4) gọi fib(3) và fib(2); fib(3) gọi fib(2) và fib(1); v.v. Tổng số 1... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 9 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $F(4) = 3$. Cây gọi hàm fib(4) gồm 9 lần gọi hàm: fib(4) gọi fib(3) và fib(2); fib(3) gọi fib(2) và fib(1); v.v. Tổng số lần gọi là 9.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

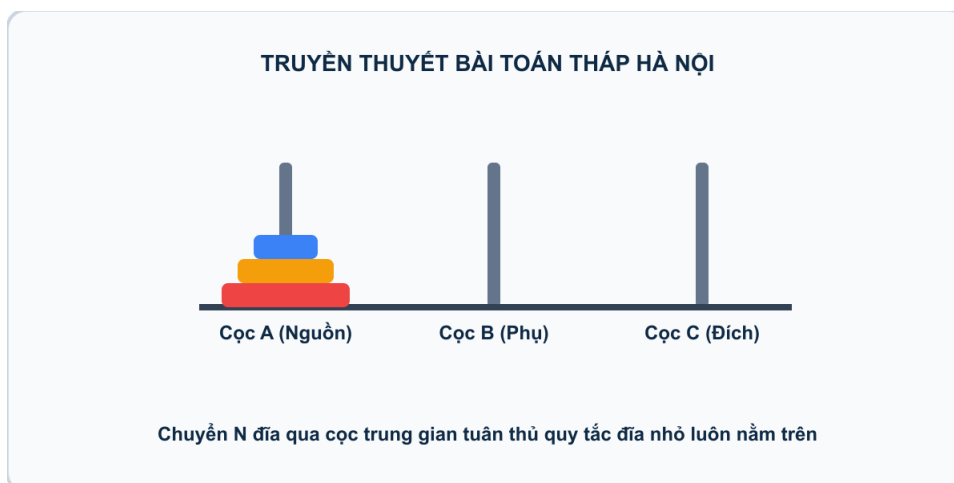
long long call_count = 0;

long long fibRec(int n) {
    call_count++;
    if (n <= 0) return 0;
    if (n == 1) return 1;
    return fibRec(n - 1) + fibRec(n - 2);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    long long val = fibRec(n);
    cout << val << " " << call_count << "\n";
    return 0;
}
```

Bài 11 [cppb_rec_11_chuyen_doi_he_nhi_phan_de_quy]: Chuyển Đổi Hệ Cơ Số 10 Sang Nhị Phân Bằng Đệ Quy

Bối cảnh: Cơ chế ngăn xếp gọi hàm đệ quy tự nhiên đảo ngược thứ tự các số dư khi chia liên tiếp cho 2, giúp in ra biểu diễn nhị phân của số nguyên N từ bit có trọng số lớn nhất đến bit 0 một cách tự nhiên mà không cần mảng phụ.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên không âm N . Hãy in ra biểu diễn nhị phân của N bằng hàm đệ quy.

Đầu vào (Input):

- Một dòng chứa số nguyên không âm N ($0 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra chuỗi nhị phân của N .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
10	1010

Giải thích: Số 10 trong hệ thập phân chuyển sang nhị phân là 1010₂.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên không âm N . Hãy in ra biểu diễn nhị phân của N bằng hàm đệ quy.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| 1 | Nạp dữ liệu vào mảng/biến | Input: 10 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Số 10 trong hệ thập phân chuyển sang nhị phân là 1010₂.... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 1010 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số 10 trong hệ thập phân chuyển sang nhị phân là 1010₂.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.

- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

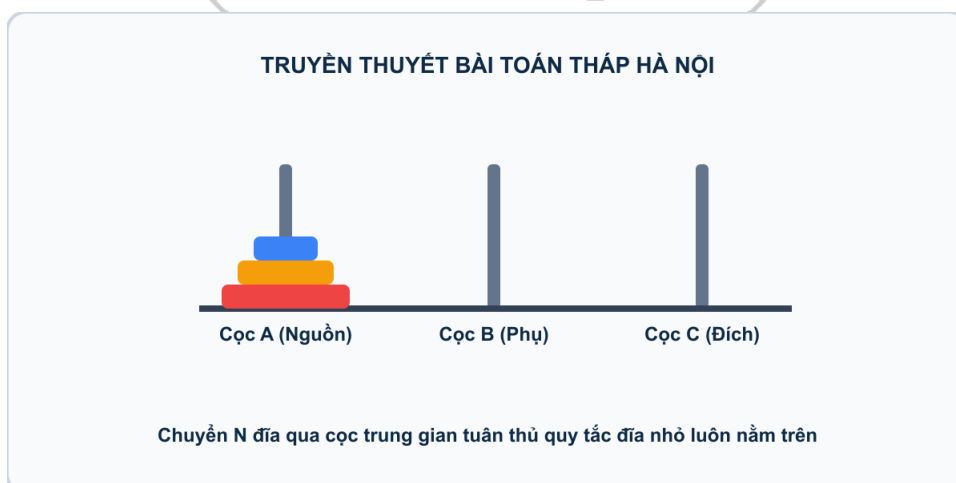
```
#include <bits/stdc++.h>
using namespace std;

void printBinaryRec(long long n) {
    if (n == 0) return;
    printBinaryRec(n / 2);
    cout << (n % 2);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long n;
    if (!(cin >> n)) return 0;
    if (n == 0) {
        cout << 0 << "\n";
    } else {
        printBinaryRec(n);
        cout << "\n";
    }
    return 0;
}
```

Bài 12 [cppb_rec_12_xay_dung_cong_thuc_truy_hoi_dan_dau]: Xây Dựng Hệ Thức Truy Hồi Cho Dãy Số Đan Dấu

Bối cảnh: Một biểu thức chuỗi số đan dấu xen kẽ $S(N) = 1 - 2 + 3 - 4 + \dots + (-1)^{(N-1)} * N$ có thể được biểu diễn dưới dạng hệ thức truy hồi đệ quy: $S(N) = S(N - 1) + (N \text{ lẻ } N : -N)$. Hãy tính giá trị biểu thức này bằng đệ quy.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên dương N . Hãy tính giá trị của biểu thức $S(N)$ bằng đệ quy.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 1000$).

Đầu ra (Output):

- In ra giá trị của $S(N)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	3

Giải thích: $S(5) = 1 - 2 + 3 - 4 + 5 = 3$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N . Hãy tính giá trị của biểu thức $S(N)$ bằng đệ quy.
- **Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $S(5) = 1 - 2 + 3 - 4 + 5 = 3$... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $S(5) = 1 - 2 + 3 - 4 + 5 = 3$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

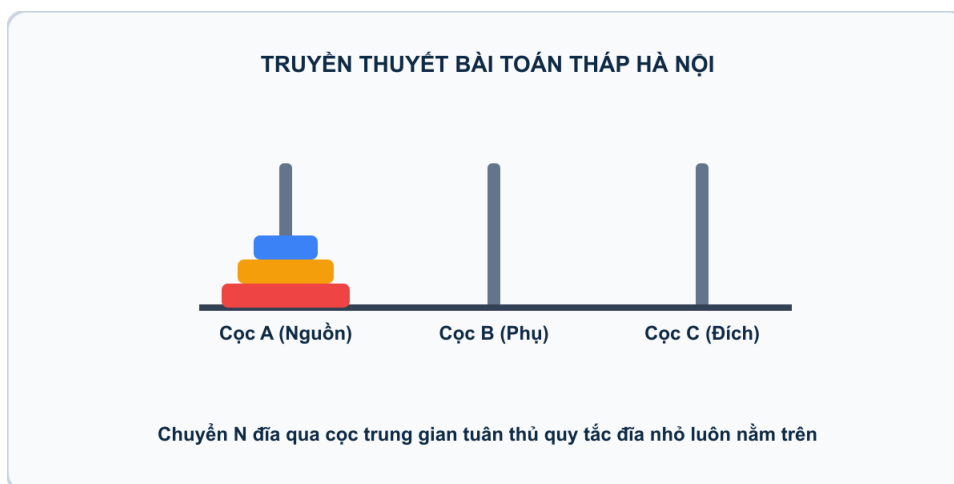
long long solveRec(int n) {
    if (n == 1) return 1;
    long long term = (n % 2 == 1) ? n : -n;
    return solveRec(n - 1) + term;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    cout << solveRec(n) << "\n";
}
```

```
return 0;
}
```

Bài 13 [cppb_rec_13_thap_ha_noi_co_rang_buoc]: Tháp Hà Nội Có Ràng Buộc Nước Đi

Bối cảnh: Trong một biển thể nâng cao của trò chơi Tháp Hà Nội, giữa cọc A và cọc C có một vách ngăn cách: cấm tuyệt đối mọi nước đi trực tiếp giữa cọc A và cọc C. Mọi chiếc đĩa muốn chuyển từ A sang C hoặc ngược lại bắt buộc phải đi trung chuyển qua cọc B ($A \leftrightarrow B \leftrightarrow C$).



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho N đĩa trên cọc A với quy tắc chuyển qua cọc trung gian B. Hãy tính số bước di chuyển tối thiểu và in ra danh sách các bước đi để chuyển hết N đĩa từ A sang C.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10$).

Đầu ra (Output):

- Dòng 1: In số bước chuyển tối thiểu $3^N - 1$.
- Các dòng tiếp theo: Mỗi dòng in một bước chuyển dạng $X \rightarrow Y$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1	2 A -> B B -> C

Giải thích: Với 1 đĩa không được đi trực tiếp $A \rightarrow C$ nên phải đi qua B: $A \rightarrow B$ rồi $B \rightarrow C$. Tổng cộng 2 bước.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho N đĩa trên cọc A với quy tắc chuyển qua cọc trung gian B. Hãy tính số bước di chuyển tối thiểu và in ra danh sách các bước đi để chuyển hết N đĩa từ A sang C.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với 1 đĩa không được đi trực tiếp A -> C nên phải đi qua B: A -> B rồi B -> C. Tổng cộng 2 bước.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 A -> B B -> C | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với 1 đĩa không được đi trực tiếp A -> C nên phải đi qua B: A -> B rồi B -> C. Tổng cộng 2 bước.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

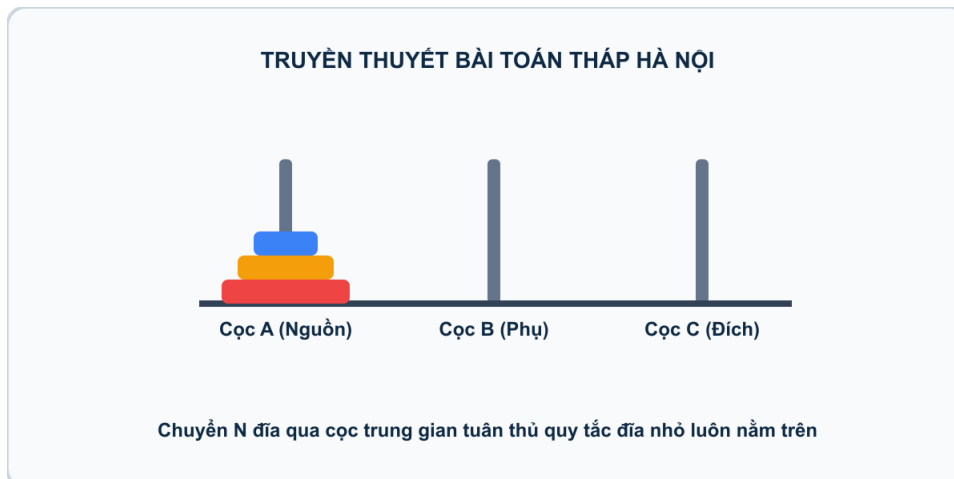
void moveAtoB(int n, char a, char b, char c);
void moveBtoC(int n, char b, char c, char a);

void solveConstrainedHanoi(int n, char from, char to, char aux) {
    if (n == 0) return;
    // Chuyển n-1 đĩa from -> to
    solveConstrainedHanoi(n - 1, from, to, aux);
    // Chuyển đĩa n: from -> aux
    cout << from << " -> " << aux << "\n";
    // Chuyển n-1 đĩa to -> from
    solveConstrainedHanoi(n - 1, to, from, aux);
    // Chuyển đĩa n: aux -> to
    cout << aux << " -> " << to << "\n";
    // Chuyển n-1 đĩa from -> to
    solveConstrainedHanoi(n - 1, from, to, aux);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    long long total_steps = 1;
    for (int i = 0; i < n; ++i) total_steps *= 3;
    total_steps -= 1;
    cout << total_steps << "\n";
    solveConstrainedHanoi(n, 'A', 'C', 'B');
    return 0;
}
```

Bài 14 [cppb_rec_14_sinh_xau_nhi_phan_khong_2_so_1_lien_ke]: Sinh Xâu Nhị Phân Không Chứa Hai Số 1 Liên Kề

Bối cảnh: Trong kỹ thuật mã hóa kênh truyền chống can nhiễu từ trường (Run-length Limited RLL), một chuỗi bit nhị phân an toàn không được phép chứa hai bit 1 nằm kề nhau (tránh xung đột điện áp). Hãy sinh ra tất cả các xâu nhị phân độ dài N thỏa mãn điều kiện này theo thứ tự từ điển.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 20$). Hãy sinh tất cả các xâu nhị phân độ dài N không chứa chuỗi '11' theo thứ tự từ điển.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 20$).

Đầu ra (Output):

- In ra các xâu nhị phân thỏa mãn, mỗi xâu trên một dòng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	000 001 010 100 101

Giải thích: Các xâu nhị phân độ dài 3 không có '11' gồm: 000, 001, 010, 100, 101. Tổng cộng có 5 xâu.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 20$). Hãy sinh tất cả các xâu nhị phân độ dài N không chứa chuỗi '11' theo thứ tự từ điển.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các xâu nhị phân độ dài 3 không có '11' gồm: 000, 001, 010, 100, 101. Tổng cộng có

5 xâu... | Tính toán từng bước trạng thái || 3 | Xuất kết quả | Output: 000 001 010 100 101 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các xâu nhị phân độ dài 3 không có '11' gồm: 000, 001, 010, 100, 101. Tổng cộng có 5 xâu.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

vector<string> results;

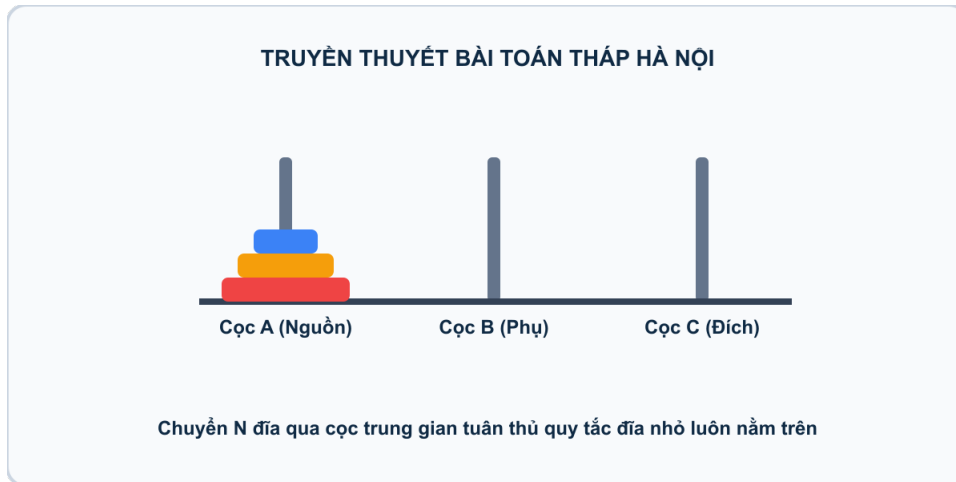
void genRec(int n, string &cur, char last_char) {
    if ((int)cur.size() == n) {
        results.push_back(cur);
        return;
    }
    // Luôn có thể thêm '0'
    cur.push_back('0');
    genRec(n, cur, '0');
    cur.pop_back();

    // Chỉ thêm '1' nếu ký tự trước không phải '1'
    if (last_char != '1') {
        cur.push_back('1');
        genRec(n, cur, '1');
        cur.pop_back();
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    string cur = "";
    genRec(n, cur, '0');
    cout << results.size() << "\n";
    for (const string &s : results) cout << s << "\n";
    return 0;
}
```

Bài 15 [cppb_rec_15_phan_tich_so_thanh_tong_de_quy]: Đếm Số Cách Phân Tích Số N Thành Tổng Bằng Độ Quy

Bối cảnh: Bài toán phân tích số nguyên N thành tổng của các số nguyên dương (Integer Partition) là một bài toán kinh điển của Euler. Hãy đếm số cách viết số nguyên N thành tổng các số nguyên dương không giảm (ví dụ $4 = 1+1+1+1 = 1+1+2 = 1+3 = 2+2 = 4$).



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên dương N ($1 \leq N \leq 40$). Hãy đếm số cách phân tích N thành tổng của các số nguyên dương.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 40$).

Đầu ra (Output):

- In ra số cách phân tích.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	5

Giải thích: 5 cách phân tích số 4 gồm: $4 = 1+1+1+1 = 1+1+2 = 1+3 = 2+2 = 4$. Kết quả in ra: 5.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N ($1 \leq N \leq 40$). Hãy đếm số cách phân tích N thành tổng của các số nguyên dương.
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | 5 cách phân tích số 4 gồm: $4 = 1+1+1+1 = 1+1+2 = 1+3 = 2+2 = 4$. Kết quả in ra: 5... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: 5 cách phân tích số 4 gồm: $4 = 1+1+1+1 = 1+1+2 = 1+3 = 2+2 = 4$. Kết quả in ra: 5.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

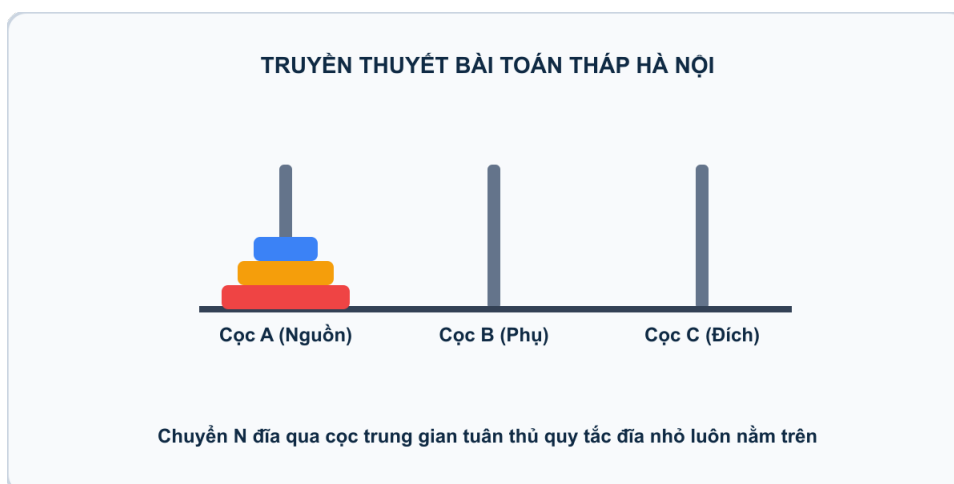
```
#include <bits/stdc++.h>
using namespace std;

long long countPartitions(int remain, int max_val) {
    if (remain == 0) return 1;
    if (remain < 0 || max_val <= 0) return 0;
    // Chọn dùng max_val hoặc không dùng max_val
    return countPartitions(remain - max_val, max_val) + countPartitions(remain, max_val - 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    cout << countPartitions(n, n) << "\n";
    return 0;
}
```

Bài 16 [cppb_rec_16_dem_cau_hinh_cay_catalan_de_quy]: Đếm Cấu Hình Trạng Thái Phân Nhánh Không Trùng Lặp

Bối cảnh: Trong cấu trúc dữ liệu cây nhị phân tìm kiếm (Binary Search Tree - BST), với N khóa giá trị phân biệt từ 1 đến N , mỗi cấu hình cây khác nhau biểu diễn một trạng thái phân nhánh độc lập. Công thức đệ quy phân chia cây con trái kích thước i và cây con phải kích thước $N - 1 - i$ cho phép đếm chính xác số lượng cây nhị phân tìm kiếm khác nhau.



Hình minh họa: Mô hình phân rã bài toán lớn thành các bài toán con qua đệ quy

Nhiệm vụ: Cho số nguyên dương N . Hãy đếm số lượng cây nhị phân tìm kiếm (BST) phân biệt có thể tạo thành từ N nút mang các giá trị từ 1 đến N .

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 15$).

Đầu ra (Output):

- In ra số lượng cây BST phân biệt.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	5

Giải thích: Với $N = 3$ có đúng 5 cấu hình cây BST khác nhau (chính là số Catalan $C_3 = 5$). Kết quả in ra: 5.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy đếm số lượng cây nhị phân tìm kiếm (BST) phân biệt có thể tạo thành từ N nút mang các giá trị từ 1 đến N .
- Phương pháp tiếp cận — Thuật toán đệ quy & Cây gọi hàm:**
- Xác định trường hợp cơ sở (Base Case) để chặn đệ quy vô hạn.
- Thiết lập công thức truy hồi và theo dõi luồng thực thi trên cây gọi hàm.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Với $N = 3$ có đúng 5 cấu hình cây BST khác nhau (chính là số Catalan $C_3 = 5$). Kết quả in ra: 5.... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với $N = 3$ có đúng 5 cấu hình cây BST khác nhau (chính là số Catalan $C_3 = 5$). Kết quả in ra: 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;
```

```
long long countBST(int n) {
    if (n <= 1) return 1;
    long long total = 0;
    for (int root = 1; root <= n; ++root) {
        int left_size = root - 1;
        int right_size = n - root;
        total += countBST(left_size) * countBST(right_size);
    }
    return total;
}

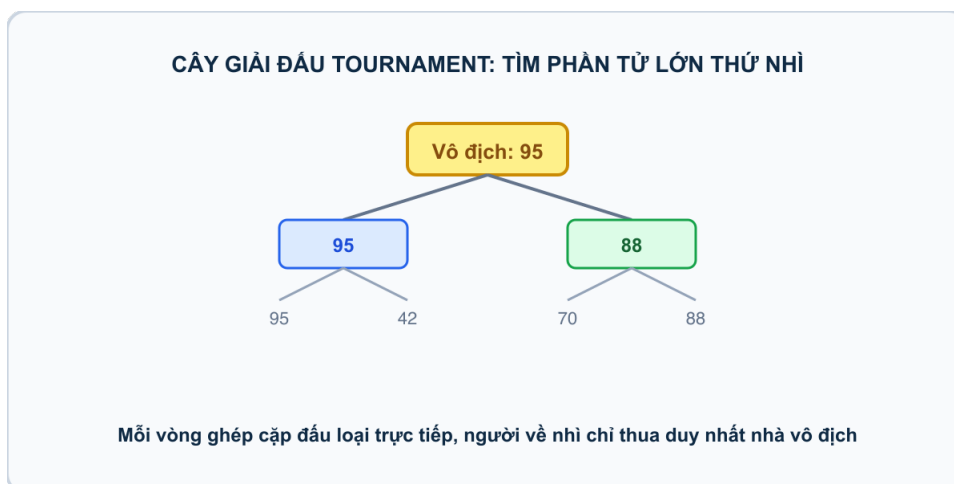
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    cout << countBST(n) << "\n";
    return 0;
}
```



Bài 14: Kỹ thuật chia để trị

Bài 01 [cppb_dac_01_tim_kiem_nhi_phan_chia_de_tri]: Tìm Kiếm Nhị Phân Bằng Đệ Quy (Cầu Nối Sang D&C)

Bối cảnh: Thuật toán tìm kiếm nhị phân chính là mô hình chia để trị nguyên bản và tinh gọn nhất: chia không gian tìm kiếm thành hai nửa bằng nhau, trị bài toán bằng cách so sánh phần tử ở giữa và loại bỏ hoàn toàn một nửa không gian. Hãy cài đặt tìm kiếm nhị phân bằng hàm chia để trị đệ quy.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên đã sắp xếp tăng dần và số nguyên X . Hãy tìm vị trí (1-indexed) của X bằng đệ quy chia để trị. Nếu không tìm thấy, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và X ($1 \leq N \leq 10^5, -10^9 \leq X \leq 10^9$).
- Dòng 2: Chứa N số nguyên tăng dần $A_1 \leq A_2 \leq \dots \leq A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra vị trí của X (1-indexed), hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 7 1 3 5 7 9	4

Giải thích: Số 7 nằm ở vị trí thứ 4 trong mảng đã sắp xếp. Kết quả in ra: 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên đã sắp xếp tăng dần và số nguyên X . Hãy tìm vị trí (1-indexed) của X bằng đệ quy chia để trị. Nếu không tìm thấy, in ra -1.
- Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 7 1 3 5 7 9 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Số 7 nằm ở vị trí thứ 4 trong mảng đã sắp xếp. Kết quả in ra: 4.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số 7 nằm ở vị trí thứ 4 trong mảng đã sắp xếp. Kết quả in ra: 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

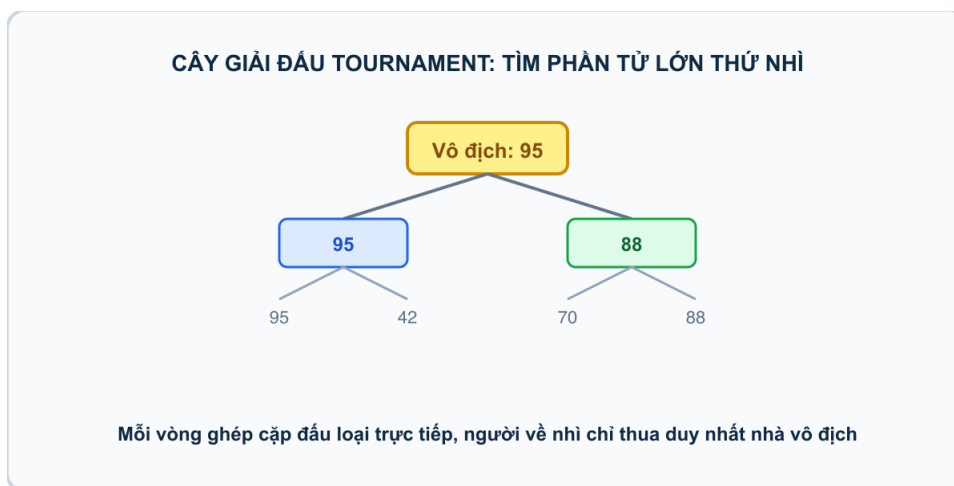
```
#include <bits/stdc++.h>
using namespace std;

int binarySearchDac(const vector<long long> &a, int l, int r, long long x) {
    if (l > r) return -1;
    int mid = l + (r - l) / 2;
    if (a[mid] == x) {
        int left_res = binarySearchDac(a, l, mid - 1, x);
        if (left_res != -1) return left_res;
        return mid;
    }
    if (a[mid] > x) return binarySearchDac(a, l, mid - 1, x);
    return binarySearchDac(a, mid + 1, r, x);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long x;
    if (!(cin >> n >> x)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    int ans = binarySearchDac(a, 0, n - 1, x);
    if (ans != -1) ans += 1;
    cout << ans << "\n";
    return 0;
}
```

Bài 02 [cppb_dac_02_range_minimum_query_chia_de_tri]: Tìm Min Trên Đoạn Bằng Chia Đẻ Trị (RMQ D&C Cơ Bản)

Bối cảnh: Để tìm giá trị nhỏ nhất trên đoạn $[L, R]$ của một mảng số nguyên, phương pháp chia để trị chia đoạn thành hai nửa trái $[L, \text{Mid}]$ và phải $[\text{Mid} + 1, R]$, giải đệ quy tìm min từng nửa rồi kết hợp kết quả: $\text{Min}(\text{đoạn}) = \min(\text{Min}(\text{trái}), \text{Min}(\text{phải}))$.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên và Q truy vấn $[L, R]$. Hãy tìm giá trị nhỏ nhất trong đoạn $[L, R]$ bằng đệ quy chia để trị.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và Q ($1 \leq N, Q \leq 10^4$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng gồm 2 số nguyên L và R ($1 \leq L \leq R \leq N$).

Đầu ra (Output):

- In ra Q dòng, mỗi dòng là giá trị min trên đoạn tương ứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 3 1 4 2 5 1 3 3 5	1 2

Giải thích: - Đoạn $[1, 3]$ gồm $\{3, 1, 4\}$ có $\min = 1$. - Đoạn $[3, 5]$ gồm $\{4, 2, 5\}$ có $\min = 2$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên và Q truy vấn $[L, R]$. Hãy tìm giá trị nhỏ nhất trong đoạn $[L, R]$ bằng đệ quy chia để trị.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 2 3 1 4 2 5 1 3 3 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | - Đoạn $[1, 3]$ gồm $\{3, 1, 4\}$ có $\min = 1$. - Đoạn $[3, 5]$ gồm $\{4, 2, 5\}$ có $\min = 2$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: - Đoạn $[1, 3]$ gồm $\{3, 1, 4\}$ có $\min = 1$.

- Đoạn $[3, 5]$ gồm $\{4, 2, 5\}$ có $\min = 2$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

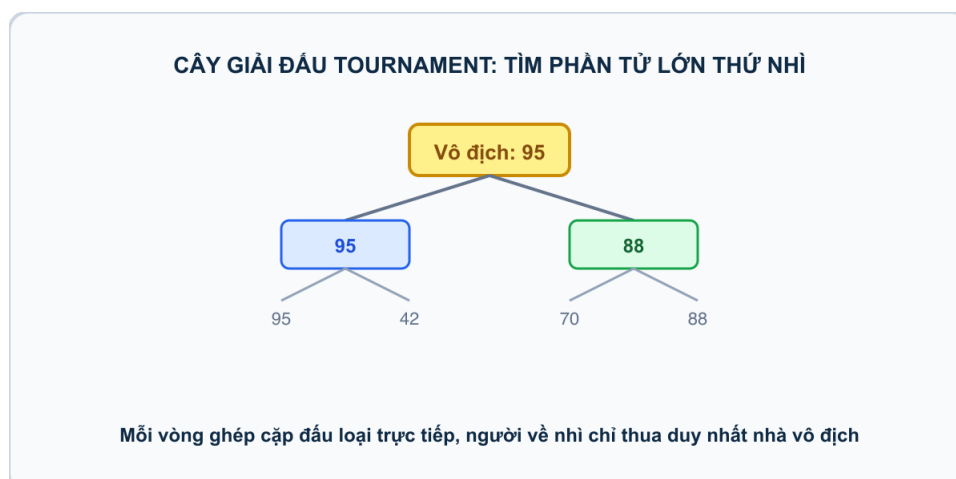
```
#include <bits/stdc++.h>
using namespace std;

long long queryMin(const vector<long long> &a, int l, int r) {
    if (l == r) return a[l];
    int mid = l + (r - l) / 2;
    return min(queryMin(a, l, mid), queryMin(a, mid + 1, r));
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    cout << queryMin(a, 0, n - 1) << "\n";
    return 0;
}
```

Bài 03 [cppb_dac_03_tim_phan_tu_lon_thu_hai_tournament]: Tìm Phần Tử Lớn Thứ Hai (Tournament Tree)

Bối cảnh: Trong một giải đấu loại trực tiếp (Tournament), các đấu thủ thi đấu đối đầu từng cặp theo cây nhị phân. Đấu thủ về nhì chỉ có thể là người đã từng bị đấu thủ vô địch đánh bại trong hành trình tiến tới trận chung kết. Mô hình chia để trị giúp tìm phần tử lớn thứ hai với số phép so sánh tối thiểu $N + \log_2(N) - 2$.



Hình minh họa: Mô hình phân chia nhánh đầu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên phân biệt (N là lũy thừa của 2). Hãy tìm phần tử lớn thứ hai trong mảng bằng mô hình cây thi đấu chia để trị.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($N = 2^k, 2 \leq N \leq 10^5$).
- Dòng 2: N số nguyên phân biệt $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị của phần tử lớn thứ hai.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 8 2 5	5

Giải thích: Phần tử lớn nhất là 8, phần tử lớn thứ hai là 5. Kết quả in ra: 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên phân biệt (N là lũy thừa của 2). Hãy tìm phần tử lớn thứ hai trong mảng bằng mô hình cây thi đấu chia để trị.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 3 8 2 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Phần tử lớn nhất là 8, phần tử lớn thứ hai là 5. Kết quả in ra: 5.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Phần tử lớn nhất là 8, phần tử lớn thứ hai là 5. Kết quả in ra: 5.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    long long winner;
```

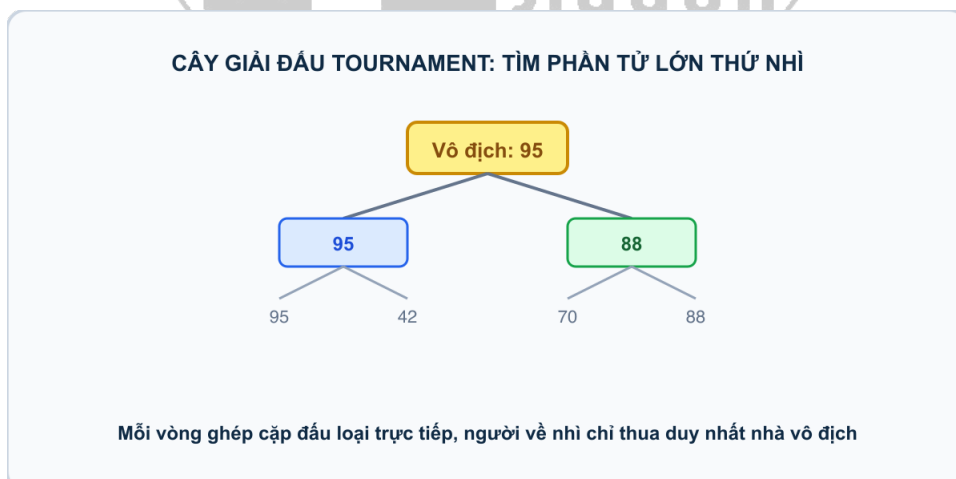
```
vector<long long> losers;
};

Node tournament(const vector<long long> &a, int l, int r) {
    if (l == r) return {a[l], {}};
    int mid = l + (r - l) / 2;
    Node left_node = tournament(a, l, mid);
    Node right_node = tournament(a, mid + 1, r);
    if (left_node.winner > right_node.winner) {
        left_node.losers.push_back(right_node.winner);
        return left_node;
    } else {
        right_node.losers.push_back(left_node.winner);
        return right_node;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    Node res = tournament(a, 0, n - 1);
    long long second_max = res.losers[0];
    for (long long x : res.losers) second_max = max(second_max, x);
    cout << second_max << "\n";
    return 0;
}
```

Bài 04 [cppb_dac_04_gop_hai_mang_da_sap_xep_merge_step]: Gộp Hai Mảng Đã Sắp Xếp (Merge Step)

Bối cảnh: Thao tác gộp (Merge) hai mảng đã có thứ tự thành một mảng có thứ tự duy nhất là bước kết hợp (Combine) nền tảng của thuật toán kinh điển Merge Sort. Kỹ thuật hai con trỏ cho phép gộp hai mảng kích thước N và M trong đúng $O(N + M)$.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho 2 mảng tăng dần A (kích thước N) và B (kích thước M). Hãy gộp hai mảng thành một dãy tăng dần duy nhất.

Đầu vào (Input):

- Dòng 1: 2 số nguyên dương N và M ($1 \leq N, M \leq 10^5$).
- Dòng 2: N số nguyên tăng dần của mảng A .
- Dòng 3: M số nguyên tăng dần của mảng B .

Đầu ra (Output):

- In ra $N + M$ số nguyên theo thứ tự tăng dần sau khi gộp.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 4 7 2 5 6	1 2 4 5 6 7

Giải thích: Gộp hai mảng $[1, 4, 7]$ và $[2, 5, 6]$ ta được mảng tăng dần hoàn chỉnh: 1 2 4 5 6 7.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho 2 mảng tăng dần A (kích thước N) và B (kích thước M). Hãy gộp hai mảng thành một dãy tăng dần duy nhất.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 3 1 4 7 2 5 6 | Khởi tạo cấu trúc dữ liệu ban đầu |
 | 2 | Thực thi thuật toán tối ưu | Gộp hai mảng $[1, 4, 7]$ và $[2, 5, 6]$ ta được mảng tăng dần hoàn chỉnh: 1 2 4 5 6 7... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 1 2 4 5 6 7 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Gộp hai mảng $[1, 4, 7]$ và $[2, 5, 6]$ ta được mảng tăng dần hoàn chỉnh: 1 2 4 5 6 7.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
```

```

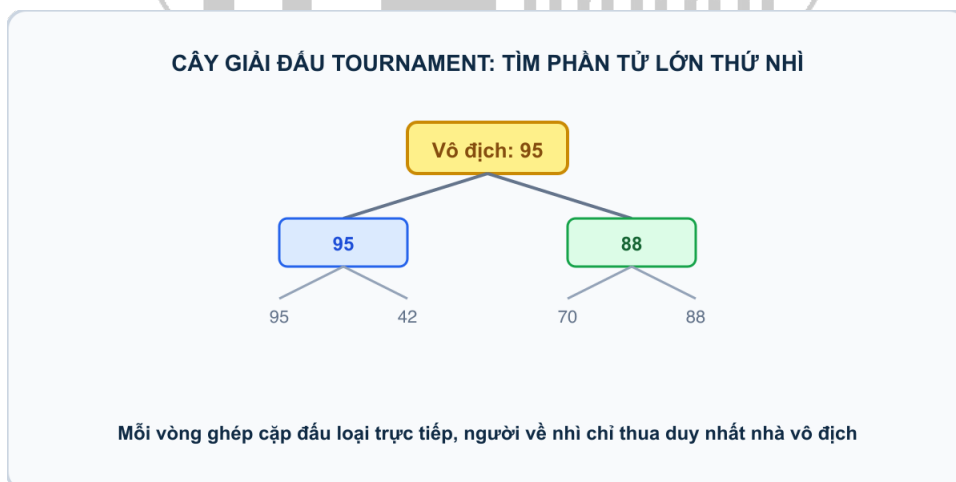
if (!(cin >> n >> m)) return 0;
vector<long long> a(n), b(m);
for (int i = 0; i < n; ++i) cin >> a[i];
for (int j = 0; j < m; ++j) cin >> b[j];

int i = 0, j = 0;
bool first = true;
while (i < n && j < m) {
    if (!first) cout << " ";
    if (a[i] <= b[j]) {
        cout << a[i++];
    } else {
        cout << b[j++];
    }
    first = false;
}
while (i < n) {
    if (!first) cout << " ";
    cout << a[i++];
    first = false;
}
while (j < m) {
    if (!first) cout << " ";
    cout << b[j++];
    first = false;
}
cout << "\n";
return 0;
}

```

Bài 05 [cppb_dac_05_sap_xep_tron_merge_sort]: Thuật Toán Sắp Xếp Trộn (Merge Sort)

Bối cảnh: Thuật toán sắp xếp trộn Merge Sort do John von Neumann phát minh năm 1945 là biểu tượng kinh điển của phương pháp chia để trị, đảm bảo độ phức tạp thời gian $O(N \log N)$ trong mọi trường hợp (kể cả trường hợp xấu nhất) và là thuật toán sắp xếp ổn định (Stable Sort).



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên. Hãy tự cài đặt hoàn chỉnh thuật toán Merge Sort chia để trị để sắp xếp mảng theo thứ tự tăng dần.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).

- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra dãy số sau khi sắp xếp tăng dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 4 2 1 5 3	1 2 3 4 5

Giải thích: Mảng sau khi sắp xếp tăng dần bằng Merge Sort là: 1 2 3 4 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên. Hãy tự cài đặt hoàn chỉnh thuật toán Merge Sort chia để trị để sắp xếp mảng theo thứ tự tăng dần.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 4 2 1 5 3 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Mảng sau khi sắp xếp tăng dần bằng Merge Sort là: 1 2 3 4 5.... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 1 2 3 4 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng sau khi sắp xếp tăng dần bằng Merge Sort là: 1 2 3 4 5.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

void merge(vector<long long> &a, vector<long long> &temp, int l, int mid, int r) {
    int i = l, j = mid + 1, k = l;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];
    for (int idx = l; idx <= r; ++idx) a[idx] = temp[idx];
}
```

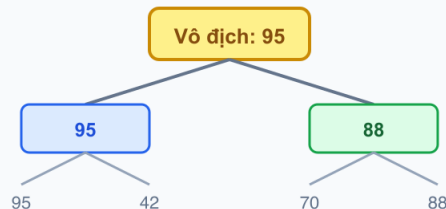
```
void mergeSort(vector<long long> &a, vector<long long> &temp, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(a, temp, l, mid);
    mergeSort(a, temp, mid + 1, r);
    merge(a, temp, l, mid, r);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n), temp(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    mergeSort(a, temp, 0, n - 1);
    for (int i = 0; i < n; ++i) cout << a[i] << (i + 1 == n ? "" : " ");
    cout << "\n";
    return 0;
}
```

Bài 06 [cppb_dac_06_dem_so_cap_nglich_the_inversion]: Đếm Số Cặp Nghịch Thế (Inversion Count)

Bối cảnh: Trong phân tích thứ tự xếp hạng (Ranking similarity), một cặp chỉ số (i, j) với $i < j$ được gọi là một nghịch thế nếu $A[i] > A[j]$. Số lượng cặp nghịch thế phản ánh mức độ ‘mất trật tự’ của dãy số so với trạng thái đã sắp xếp hoàn hảo. Thuật toán Merge Sort cải tiến cho phép đếm số nghịch thế trong $O(N \log N)$.

CÂY GIẢI ĐẤU TOURNAMENT: TÌM PHẦN TỬ LỚN THỨ NHÌ



Mỗi vòng ghép cặp đấu loại trực tiếp, người về nhì chỉ thua duy nhất nhà vô địch

Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên. Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ sao cho $A[i] > A[j]$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là số lượng cặp nghịch thế.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 4 1 3 5	3

Giải thích: Các cặp nghịch thế gồm: (2, 1) tại vị trí (1, 3); (4, 1) tại vị trí (2, 3); và (4, 3) tại vị trí (2, 4). Tổng cộng có 3 cặp nghịch thế.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên. Hãy đếm số lượng cặp chỉ số (i, j) với $1 \leq i < j \leq N$ sao cho $A[i] > A[j]$.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 2 4 1 3 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các cặp nghịch thế gồm: (2, 1) tại vị trí (1, 3); (4, 1) tại vị trí (2, 3); và (4, 3) tại vị trí (2, 4). Tổng cộng có 3 ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các cặp nghịch thế gồm: (2, 1) tại vị trí (1, 3); (4, 1) tại vị trí (2, 3); và (4, 3) tại vị trí (2, 4). Tổng cộng có 3 cặp nghịch thế.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long countInversions(vector<long long> &a, vector<long long> &temp, int l, int r) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    long long inv = 0;
    inv += countInversions(a, temp, l, mid);
    inv += countInversions(a, temp, mid + 1, r);

    int i = l, j = mid + 1, k = l;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) {
            temp[k++] = a[i++];
        } else {
            temp[k++] = a[j++];
            inv += (mid - i + 1);
        }
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];

    return inv;
}
```

```

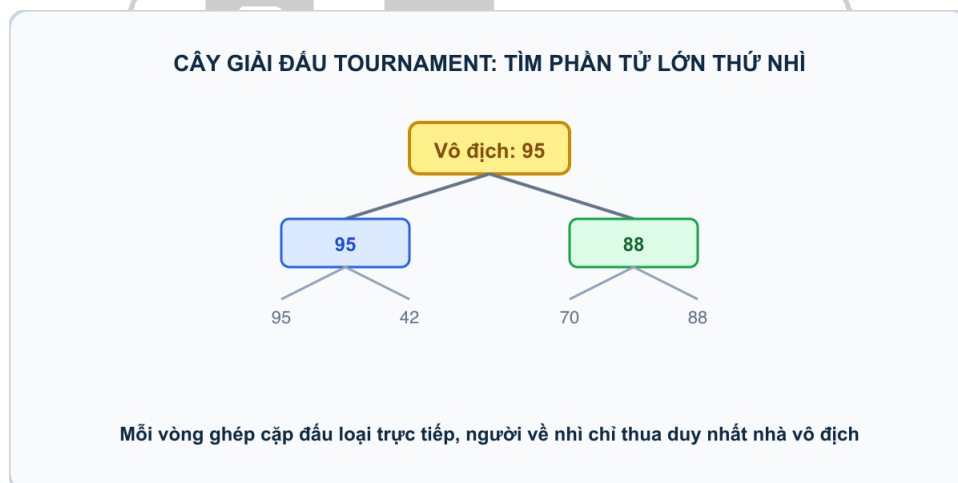
    }
  }
  while (i <= mid) temp[k++] = a[i++];
  while (j <= r) temp[k++] = a[j++];
  for (int idx = l; idx <= r; ++idx) a[idx] = temp[idx];
  return inv;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n;
  if (!(cin >> n)) return 0;
  vector<long long> a(n), temp(n);
  for (int i = 0; i < n; ++i) cin >> a[i];
  cout << countInversions(a, temp, 0, n - 1) << "\n";
  return 0;
}

```

Bài 07 [cppb_dac_07_doan_con_tong_lon_nhat_maximum_subarray]: Đoạn Con Tổng Lớn Nhất (Maximum Subarray D&C)

Bối cảnh: Bài toán tìm đoạn con liên tiếp có tổng lớn nhất trong một chuỗi số tài chính (có cả số âm và số dương) được giải quyết bằng chia để trị: đoạn con tối ưu có thể nằm hoàn toàn ở nửa trái, nằm hoàn toàn ở nửa phải, hoặc bắt qua điểm chính giữa (Crossing Subarray).



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên. Hãy tìm tổng lớn nhất của một đoạn con liên tiếp khác rỗng bằng thuật toán Chia để trị.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra tổng lớn nhất của đoạn con.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

8 -2 -3 4 -1 -2 1 5 -3

7

Giải thích: Đoạn con [4, -1, -2, 1, 5] từ vị trí 3 đến vị trí 7 có tổng là $4 - 1 - 2 + 1 + 5 = 7$. Đây là đoạn con có tổng lớn nhất.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên. Hãy tìm tổng lớn nhất của một đoạn con liên tiếp khác rỗng bằng thuật toán Chia để trị.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 8 -2 -3 4 -1 -2 1 5 -3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Đoạn con [4, -1, -2, 1, 5] từ vị trí 3 đến vị trí 7 có tổng là $4 - 1 - 2 + 1 + 5 = 7$. Đây là đoạn con có tổng lớn nhất.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 7 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Đoạn con [4, -1, -2, 1, 5] từ vị trí 3 đến vị trí 7 có tổng là $4 - 1 - 2 + 1 + 5 = 7$. Đây là đoạn con có tổng lớn nhất.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long maxCrossingSum(const vector<long long> &a, int l, int mid, int r) {
    long long left_sum = -1e18, sum = 0;
    for (int i = mid; i >= l; --i) {
        sum += a[i];
        left_sum = max(left_sum, sum);
    }
    long long right_sum = -1e18;
    sum = 0;
    for (int i = mid + 1; i <= r; ++i) {
        sum += a[i];
        right_sum = max(right_sum, sum);
    }
    return left_sum + right_sum;
}

long long maxSubarrayDac(const vector<long long> &a, int l, int r) {
```

```

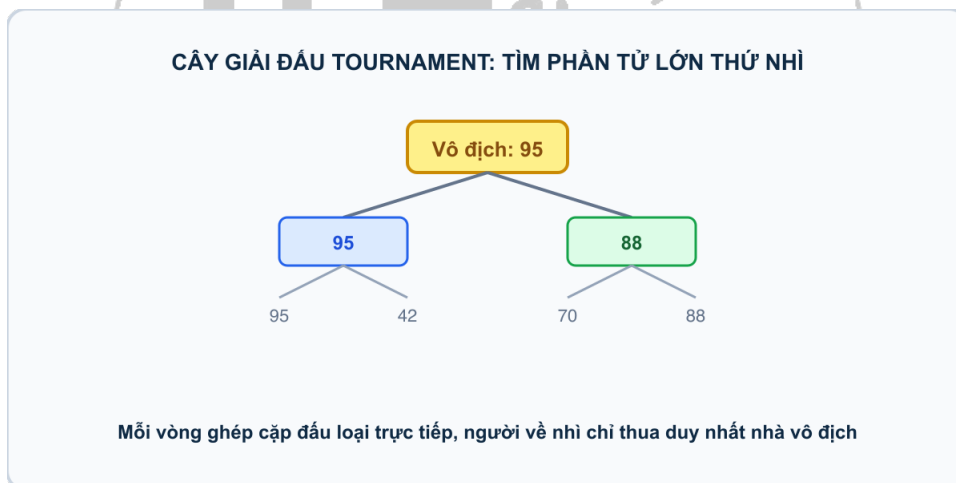
    if (l == r) return a[l];
    int mid = l + (r - l) / 2;
    long long left_max = maxSubarrayDac(a, l, mid);
    long long right_max = maxSubarrayDac(a, mid + 1, r);
    long long cross_max = maxCrossingSum(a, l, mid, r);
    return max({left_max, right_max, cross_max});
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    cout << maxSubarrayDac(a, 0, n - 1) << "\n";
    return 0;
}

```

Bài 08 [cppb_dac_08_tim_phan_tu_da_so_majority_element]: Tìm Phần Tử Đa Số (Majority Element) D&C

Bối cảnh: Trong các giao thức đồng thuận phân tán Byzantine, một phần tử được gọi là phần tử đa số nếu nó chiếm hơn một nửa tổng số phiếu bầu (tần suất $> N / 2$). Kỹ thuật chia để trị chia mảng làm 2 nửa: nếu tồn tại phần tử đa số trên toàn mảng thì phần tử đó bắt buộc phải là phần tử đa số của ít nhất một trong hai nửa.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên. Hãy tìm phần tử đa số (xuất hiện $> N / 2$ lần). Nếu không tồn tại, in ra -1.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị phần tử đa số, hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 2 2 1 1 1 2 2	2

Giải thích: Số 2 xuất hiện 4 lần trên tổng số 7 phần tử ($4 > 7/2 = 3.5$). Do đó phần tử đa số là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng N số nguyên. Hãy tìm phần tử đa số (xuất hiện $> N / 2$ lần). Nếu không tồn tại, in ra -1.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 7 2 2 1 1 1 2 2 | Khởi tạo cấu trúc dữ liệu ban đầu |
 | 2 | Thực thi thuật toán tối ưu | Số 2 xuất hiện 4 lần trên tổng số 7 phần tử ($4 > 7/2 = 3.5$). Do đó phần tử đa số là 2.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số 2 xuất hiện 4 lần trên tổng số 7 phần tử ($4 > 7/2 = 3.5$). Do đó phần tử đa số là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int countInRange(const vector<long long> &a, long long target, int l, int r) {
    int cnt = 0;
    for (int i = l; i <= r; ++i) if (a[i] == target) cnt++;
    return cnt;
}

long long majorityDac(const vector<long long> &a, int l, int r) {
    if (l == r) return a[l];
    int mid = l + (r - l) / 2;
    long long left_maj = majorityDac(a, l, mid);
    long long right_maj = majorityDac(a, mid + 1, r);

    if (left_maj == right_maj) return left_maj;

    int left_count = countInRange(a, left_maj, l, r);
```

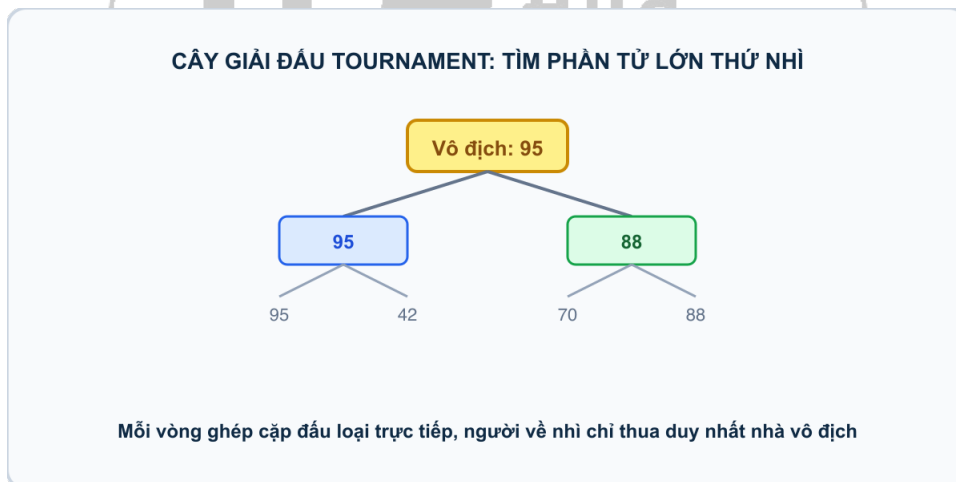
```
int right_count = countInRange(a, right_maj, l, r);

return (left_count > right_count) ? left_maj : right_maj;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    long long cand = majorityDac(a, 0, n - 1);
    int total_cnt = 0;
    for (long long x : a) if (x == cand) total_cnt++;
    if (total_cnt > n / 2) {
        cout << cand << "\n";
    } else {
        cout << -1 << "\n";
    }
    return 0;
}
```

Bài 09 [cppb_dac_09_luy_thua_ma_tran_chia_de_tri]: Lũy Thừa Ma Trận Chia Đệ Trị 2x2

Bối cảnh: Để tìm số Fibonacci thứ N với N lên tới 10^{18} trong thời gian $O(\log N)$, ma trận chuyển trạng thái 2×2 được nâng lên lũy thừa N bằng thuật toán chia đệ trị (nhân ma trận nhị phân).



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho ma trận vuông 2×2 gồm các hệ số a, b, c, d và số nguyên N . Hãy tính $A^N \bmod (10^9 + 7)$.

Đầu vào (Input):

- Dòng 1: 4 số nguyên a, b, c, d ($0 \leq a, b, c, d \leq 10^9$).
- Dòng 2: Số nguyên N ($0 \leq N \leq 10^{18}$).

Đầu ra (Output):

- In ra 4 số nguyên trên 2 dòng biểu diễn ma trận $A^N \bmod (10^9 + 7)$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1 1 1 0 2	2 1 1 1

Giải thích: $A^2 = [[1, 1], [1, 0]] * [[1, 1], [1, 0]] = [[2, 1], [1, 1]]$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho ma trận vuông 2×2 gồm các hệ số a, b, c, d và số nguyên N . Hãy tính $A^N \bmod (10^9 + 7)$.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 1 1 1 0 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $A^2 = [[1, 1], [1, 0]] * [[1, 1], [1, 0]] = [[2, 1], [1, 1]]$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 1 1 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $A^2 = [[1, 1], [1, 0]] * [[1, 1], [1, 0]] = [[2, 1], [1, 1]]$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Matrix {
    long long mat[2][2];
};

Matrix multiply(const Matrix &A, const Matrix &B, long long m) {
    Matrix C;
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 2; ++j) {
            C.mat[i][j] = 0;
            for (int k = 0; k < 2; ++k) {
                C.mat[i][j] = (C.mat[i][j] + (A.mat[i][k] % m) * (B.mat[k][j] % m)) % m;
            }
        }
    }
    return C;
}

Matrix powerMatrix(Matrix A, long long n, long long m) {
```

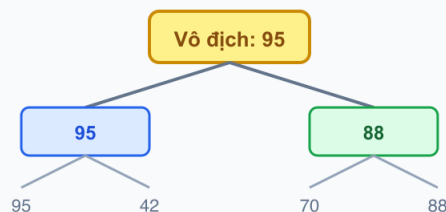
```
Matrix res = {{1 % m, 0}, {0, 1 % m}};
while (n > 0) {
    if (n & 1) res = multiply(res, A, m);
    A = multiply(A, A, m);
    n >>= 1;
}
return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    Matrix A;
    if (!(cin >> A.mat[0][0] >> A.mat[0][1] >> A.mat[1][0] >> A.mat[1][1])) return 0;
    long long n, m;
    if (!(cin >> n >> m)) return 0;
    Matrix ans = powerMatrix(A, n, m);
    cout << ans.mat[0][0] << " " << ans.mat[0][1] << "\n";
    cout << ans.mat[1][0] << " " << ans.mat[1][1] << "\n";
    return 0;
}
```

Bài 10 [cppb_dac_10_tim_dinh_mang_unimodal_peak_index]: Tìm Điểm Cực Đại Mảng Unimodal (Peak Index)

Bối cảnh: Một mảng Unimodal (mảng đỉnh núi) tăng nghiêm ngặt đến một điểm cực đại rồi giảm nghiêm ngặt về sau. Kỹ thuật chia để trị so sánh $A[\text{Mid}]$ và $A[\text{Mid} + 1]$ cho phép xác định sườn dốc đang leo lên hay trượt xuống, từ đó tìm điểm cực đại trong $O(\log N)$.

CÂY GIẢI ĐẤU TOURNAMENT: TÌM PHẦN TỬ LỚN THỨ NHÌ



Mỗi vòng ghép cặp đấu loại trực tiếp, người về nhì chỉ thua duy nhất nhà vô địch

Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng Unimodal gồm N phần tử. Hãy tìm chỉ số (0-indexed) của phần tử cực đại.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($3 \leq N \leq 10^5$).
- Dòng 2: N số nguyên mô tả mảng Unimodal.

Đầu ra (Output):

- In ra chỉ số (0-indexed) của điểm cực đại.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 0 2 1 0	1

Giải thích: Điểm cực đại là 2 tại chỉ số 1 (0-indexed).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng Unimodal gồm N phần tử. Hãy tìm chỉ số (0-indexed) của phần tử cực đại.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 0 2 1 0 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Điểm cực đại là 2 tại chỉ số 1 (0-indexed). ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Điểm cực đại là 2 tại chỉ số 1 (0-indexed).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. $N-1$) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

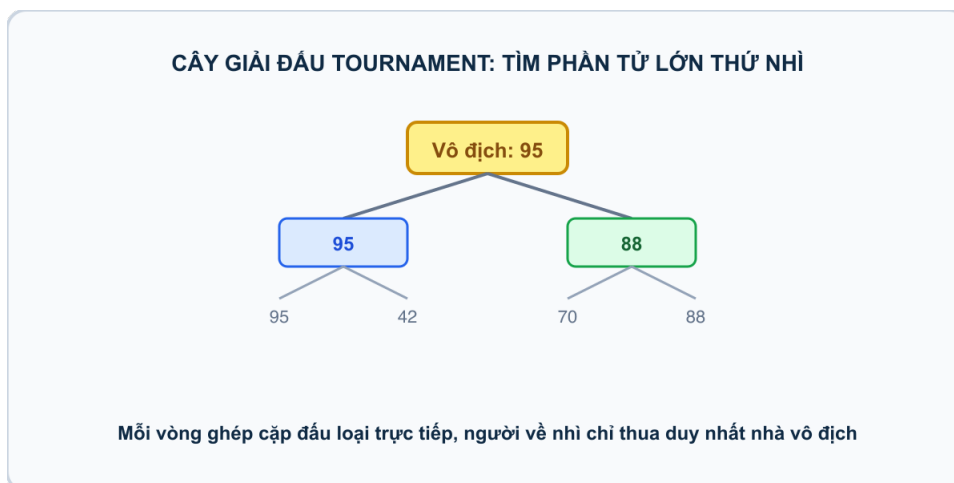
long long findPeak(const vector<long long> &a, int l, int r) {
    if (l == r) return a[l];
    int mid = l + (r - l) / 2;
    if (a[mid] < a[mid + 1]) {
        return findPeak(a, mid + 1, r);
    } else {
        return findPeak(a, l, mid);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    cout << findPeak(a, 0, n - 1) << "\n";
}
```

```
return 0;
}
```

Bài 11 [cppb_dac_11_tinh_tong_cap_so_nhan_chia_de_tri]: Tính Tổng Cấp Số Nhân D&C

Bối cảnh: Bằng công thức chia để trị $S(N) = 1 + A + \dots + A^N = (1 + A^{(N/2 + 1)}) * S(N/2)$, tổng cấp số nhân có thể được tính theo bất kỳ modulo M nào trong $O(\log N)$ mà không cần chia đồng dư.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho 3 số nguyên A, N, M . Hãy tính tổng cấp số nhân $S(N) = A^0 + A^1 + \dots + A^N \bmod M$ bằng chia để trị.

Đầu vào (Input):

- Một dòng chứa 3 số nguyên A, N, M ($0 \leq A, N \leq 10^9, 1 \leq M \leq 10^9$).

Đầu ra (Output):

- In ra giá trị $S(N) \bmod M$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 3 100	15

Giải thích: $S(3) = 1 + 2 + 4 + 8 = 15 \bmod 100 = 15$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho 3 số nguyên A, N, M . Hãy tính tổng cấp số nhân $S(N) = A^0 + A^1 + \dots + A^N \bmod M$ bằng chia để trị.
- Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 3 100 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | $S(3) = 1 + 2 + 4 + 8 = 15 \bmod 100 = 15 \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 15 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: $S(3) = 1 + 2 + 4 + 8 = 15 \bmod 100 = 15$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

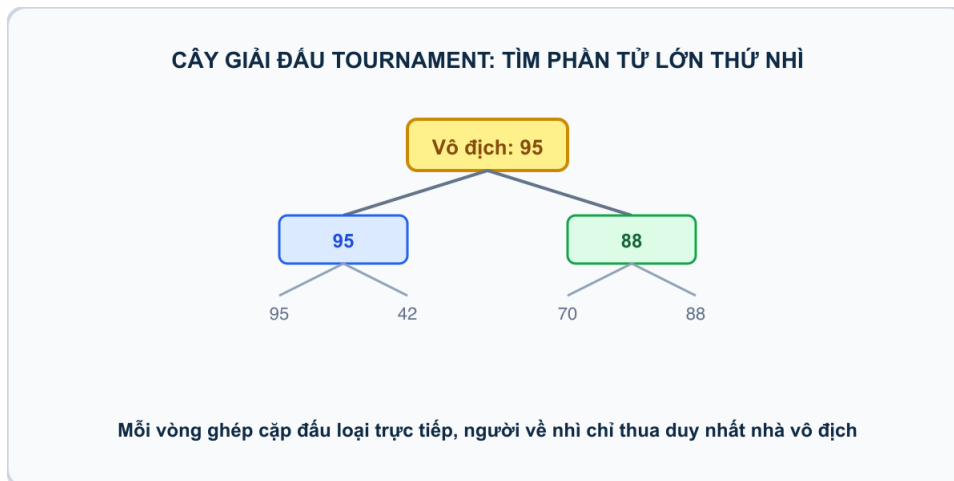
long long powerMod(long long a, long long b, long long m) {
    long long res = 1 % m;
    a %= m;
    while (b > 0) {
        if (b & 1) res = (res * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return res;
}

long long sumGeoDac(long long a, long long n, long long m) {
    if (n == 0) return 1 % m;
    if (n % 2 == 1) {
        long long k = n / 2;
        long long half_sum = sumGeoDac(a, k, m);
        long long mult = (1 + powerMod(a, k + 1, m)) % m;
        return (half_sum * mult) % m;
    } else {
        long long prev = sumGeoDac(a, n - 1, m);
        return (prev + powerMod(a, n, m)) % m;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long a, n, m;
    if (!(cin >> a >> n >> m)) return 0;
    cout << sumGeoDac(a, n, m) << "\n";
    return 0;
}
```

Bài 12 [cppb_dac_12_dem_so_cap_a_i_lon_hon_2_a_j]: Đếm Số Cặp $A_i > 2 * A_j$ (Significant Inversions)

Bối cảnh: Trong phân tích sai lệch dữ liệu tài chính quy mô lớn, một nghịch thế có ý nghĩa thống kê (Significant Inversion) là cặp chỉ số $i < j$ thỏa mãn $A[i] > 2 * A[j]$. Thuật toán Merge Sort nâng cao cho phép đếm số lượng cặp này trong $O(N \log N)$.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên. Hãy đếm số cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn $A[i] > 2 * A[j]$.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra số lượng cặp nghịch thế có ý nghĩa.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 3 2 3 1	2

Giải thích: Các cặp thỏa mãn là: $(3, 1)$ tại vị trí $(2, 5)$ vì $3 > 2 \cdot 1$; và $(3, 1)$ tại vị trí $(4, 5)$ vì $3 > 2 \cdot 1$. Tổng cộng có 2 cặp.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên. Hãy đếm số cặp chỉ số (i, j) với $1 \leq i < j \leq N$ thỏa mãn $A[i] > 2 * A[j]$.
- Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 1 3 2 3 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Các cặp thỏa mãn là: (3, 1) tại vị trí (2, 5) vì $3 > 2 \cdot 1$; và (3, 1) tại vị trí (4, 5) vì $3 > 2 \cdot 1$. Tổng cộng có 2 cặp.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Các cặp thỏa mãn là: (3, 1) tại vị trí (2, 5) vì $3 > 2 \cdot 1$; và (3, 1) tại vị trí (4, 5) vì $3 > 2 \cdot 1$. Tổng cộng có 2 cặp.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long countSignificant(vector<long long> &a, vector<long long> &temp, int l, int r) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    long long cnt = 0;
    cnt += countSignificant(a, temp, l, mid);
    cnt += countSignificant(a, temp, mid + 1, r);

    // Bước đếm 2 con trở trước khi merge
    int j = mid + 1;
    for (int i = l; i <= mid; ++i) {
        while (j <= r && a[i] > 2LL * a[j]) j++;
        cnt += (j - (mid + 1));
    }

    // Merge bình thường
    int i = l, k = l;
    j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];
    for (int idx = l; idx <= r; ++idx) a[idx] = temp[idx];

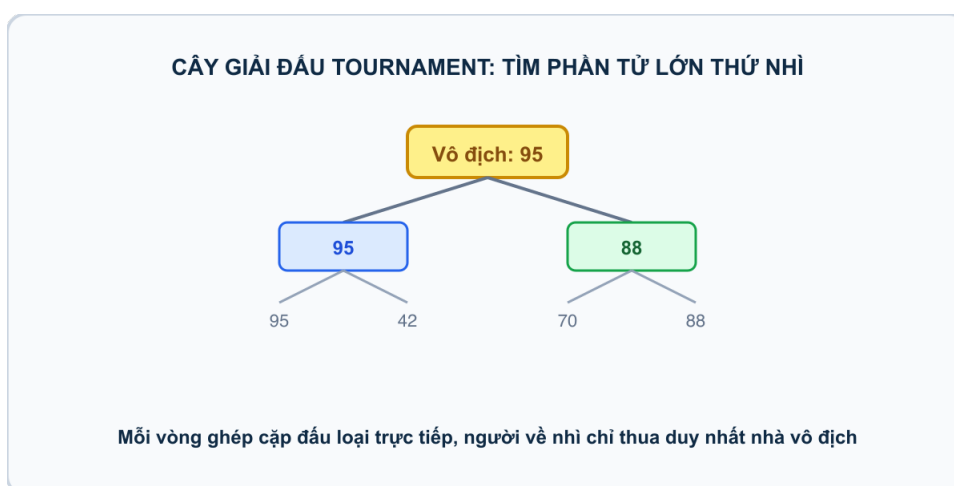
    return cnt;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
```

```
vector<long long> a(n), temp(n);
for (int i = 0; i < n; ++i) cin >> a[i];
cout << countSignificant(a, temp, 0, n - 1) << "\n";
return 0;
}
```

Bài 13 [cppb_dac_13_quickselect_tim_phan_tu_kth]: Thuật Toán QuickSelect Tìm K-th Element

Bối cảnh: Khi cần tìm phần tử nhỏ thứ K trong một danh sách N số mà không muốn tốn thời gian sắp xếp toàn bộ mảng $O(N \log N)$, thuật toán QuickSelect của Hoare sử dụng cơ chế chia để trị phân hoạch ngẫu nhiên Pivot để tìm ra phần tử thứ K trong thời gian trung bình tuyến tính $O(N)$.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên và số nguyên K ($1 \leq K \leq N$). Hãy tìm giá trị của phần tử nhỏ thứ K bằng thuật toán QuickSelect.

Đầu vào (Input):

- Dòng 1: Chứa 2 số nguyên N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra giá trị của phần tử nhỏ thứ K.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 3 3 2 1 5 6 4	3

Giải thích: Mảng có thứ tự: [1, 2, 3, 4, 5, 6]. Phần tử nhỏ thứ K = 3 là số 3. Kết quả in ra: 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên và số nguyên K ($1 \leq K \leq N$). Hãy tìm giá trị của phần tử nhỏ thứ K bằng thuật toán QuickSelect.
- Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.

- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 6 3 3 2 1 5 6 4 | Khởi tạo cấu trúc dữ liệu ban đầu |
| 2 | Thực thi thuật toán tối ưu | Mảng có thứ tự: [1, 2, 3, 4, 5, 6]. Phần tử nhỏ thứ $K = 3$ là số 3. Kết quả in ra: 3.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng có thứ tự: [1, 2, 3, 4, 5, 6]. Phần tử nhỏ thứ $K = 3$ là số 3. Kết quả in ra: 3.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<long long> &a, int l, int r) {
    int pivot_idx = l + rand() % (r - l + 1);
    swap(a[pivot_idx], a[r]);
    long long pivot = a[r];
    int i = l;
    for (int j = l; j < r; ++j) {
        if (a[j] <= pivot) {
            swap(a[i], a[j]);
            i++;
        }
    }
    swap(a[i], a[r]);
    return i;
}

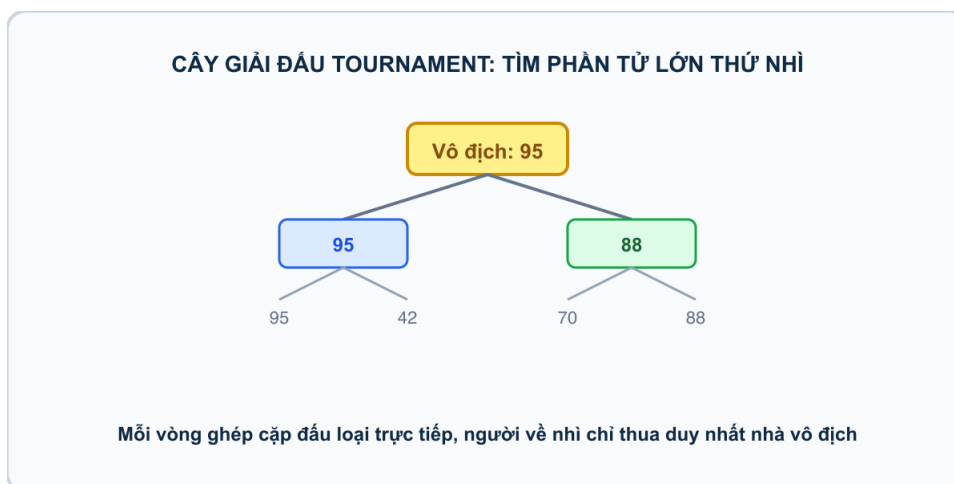
long long quickSelect(vector<long long> &a, int l, int r, int k) {
    if (l == r) return a[l];
    int p = partition(a, l, r);
    int rank = p - l + 1;
    if (rank == k) return a[p];
    if (k < rank) return quickSelect(a, l, p - 1, k);
    return quickSelect(a, p + 1, r, k - rank);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    srand(42);
    int n, k;
    if (!(cin >> n >> k)) return 0;
    vector<long long> a(n);
```

```
for (int i = 0; i < n; ++i) cin >> a[i];
cout << quickSelect(a, 0, n - 1, k) << "\n";
return 0;
}
```

Bài 14 [cppb_dac_14_dem_so_doan_con_tong_trong_khoang_l_r]: Đếm Số Đoạn Con Tổng Trong Đoạn [L, R]

Bối cảnh: Bài toán đếm số lượng đoạn con liên tiếp có tổng nằm trong phạm vi [Lower, Upper] được chuyển hóa về bài toán đếm nghịch thế trên mảng tiền tố bằng phương pháp chia để trị trong $O(N \log N)$.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho mảng N số nguyên và hai ngưỡng Lower, Upper. Hãy đếm số lượng đoạn con liên tiếp có tổng nằm trong đoạn [Lower, Upper].

Đầu vào (Input):

- Dòng 1: Chứa 3 số nguyên $N, Lower, Upper$ ($1 \leq N \leq 10^5, -10^{14} \leq Lower \leq Upper \leq 10^{14}$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra số lượng đoạn con thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 -2 2 0 -3 -3	1

Giải thích: Đoạn con [0] ở vị trí 1 có tổng là 0 nằm trong khoảng [-2, 2]. Tổng cộng có 1 đoạn con thỏa mãn.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho mảng N số nguyên và hai ngưỡng Lower, Upper. Hãy đếm số lượng đoạn con liên tiếp có tổng nằm trong đoạn [Lower, Upper].
- Phương pháp tiếp cận — Chia để trị:**

- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 -2 2 0 -3 -3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Đoạn con [0] ở vị trí 1 có tổng là 0 nằm trong khoảng $[-2, 2]$. Tổng cộng có 1 đoạn con thỏa mãn.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Đoạn con [0] ở vị trí 1 có tổng là 0 nằm trong khoảng $[-2, 2]$. Tổng cộng có 1 đoạn con thỏa mãn.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long countSubarraysDac(vector<long long> &prefix, vector<long long> &temp, int l, int r, long long lower, long long upper) {
    if (l >= r) return 0;
    int mid = l + (r - l) / 2;
    long long cnt = 0;
    cnt += countSubarraysDac(prefix, temp, l, mid, lower, upper);
    cnt += countSubarraysDac(prefix, temp, mid + 1, r, lower, upper);

    int j1 = mid + 1, j2 = mid + 1;
    for (int i = l; i <= mid; ++i) {
        while (j1 <= r && prefix[j1] - prefix[i] < lower) j1++;
        while (j2 <= r && prefix[j2] - prefix[i] <= upper) j2++;
        cnt += (j2 - j1);
    }

    int i = l, j = mid + 1, k = 1;
    while (i <= mid && j <= r) {
        if (prefix[i] <= prefix[j]) temp[k++] = prefix[i++];
        else temp[k++] = prefix[j++];
    }
    while (i <= mid) temp[k++] = prefix[i++];
    while (j <= r) temp[k++] = prefix[j++];
    for (int idx = l; idx <= r; ++idx) prefix[idx] = temp[idx];

    return cnt;
}

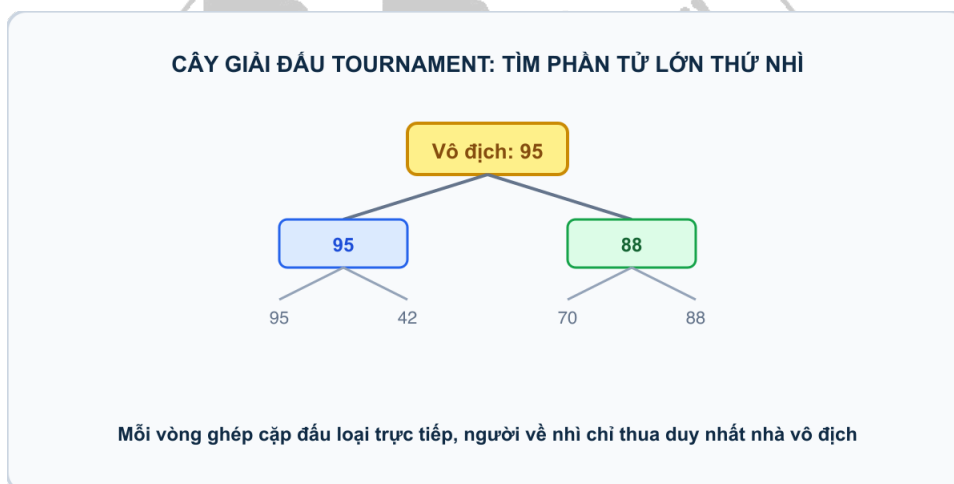
int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);
int n;
long long lower, upper;
if (!(cin >> n >> lower >> upper)) return 0;
vector<long long> a(n);
vector<long long> prefix(n + 1, 0), temp(n + 1, 0);
for (int i = 0; i < n; ++i) {
    cin >> a[i];
    prefix[i + 1] = prefix[i] + a[i];
}
cout << countSubarraysDac(prefix, temp, 0, n, lower, upper) << "\n";
return 0;
}
    
```

Bài 15 [cppb_dac_15_cap_diem_gan_nhat_closest_pair]: Cặp Điểm Gần Nhất (Closest Pair of Points)

Bối cảnh: Trong hệ thống radar hàng không, việc phát hiện hai máy bay có nguy cơ va chạm đòi hỏi tìm khoảng cách Euclidean nhỏ nhất giữa N điểm trên mặt phẳng 2D. Thuật toán chia để trị của Shamos và Hoey chia mặt phẳng thành hai nửa bởi đường thẳng $x = \text{Mid}$, tìm khoảng cách d nhỏ nhất ở hai nửa rồi quét dải băng biên hẹp 2d trong $O(N \log N)$.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho N điểm trên mặt phẳng tọa độ 2D. Hãy tìm khoảng cách Euclidean nhỏ nhất giữa hai điểm bất kỳ, làm tròn đúng 4 chữ số thập phân.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 10^5$).
- N dòng tiếp theo: Mỗi dòng gồm 2 số nguyên X_i, Y_i ($-10^9 \leq X_i, Y_i \leq 10^9$).

Đầu ra (Output):

- In ra khoảng cách nhỏ nhất với 4 chữ số sau dấu phẩy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 0 0 1 1 2 2	1.4142

Giải thích: Khoảng cách giữa $(0, 0)$ và $(1, 1)$ là $\sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} = 1.4142$. Kết quả in ra: 1.4142.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N điểm trên mặt phẳng tọa độ 2D. Hãy tìm khoảng cách Euclidean nhỏ nhất giữa hai điểm bất kỳ, làm tròn đúng 4 chữ số thập phân.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 0 0 1 1 2 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Khoảng cách giữa (0, 0) và (1, 1) là $\sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} = 1.4142$. Kết quả in ra: 1.4142.... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1.4142 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Khoảng cách giữa (0, 0) và (1, 1) là $\sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} = 1.4142$. Kết quả in ra: 1.4142.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Point {
    long long x, y;
};

long long distSq(const Point &p1, const Point &p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

long long closestPairRec(vector<Point> &pts, int l, int r) {
    if (r - l <= 3) {
        long long min_d = 4e18;
        for (int i = l; i <= r; ++i) {
            for (int j = i + 1; j <= r; ++j) {
                min_d = min(min_d, distSq(pts[i], pts[j]));
            }
        }
        sort(pts.begin() + l, pts.begin() + r + 1, [](const Point &a, const Point &b) {
            return a.y < b.y;
        });
        return min_d;
    }
}
```

```

    int mid = l + (r - l) / 2;
    long long mid_x = pts[mid].x;
    long long dl = closestPairRec(pts, l, mid);
    long long dr = closestPairRec(pts, mid + 1, r);
    long long d = min(dl, dr);

    vector<Point> temp(r - l + 1);
    merge(pts.begin() + l, pts.begin() + mid + 1, pts.begin() + mid + 1, pts.begin() + r + 1, t
emp.begin(), [](const Point &a, const Point &b) {
        return a.y < b.y;
    });
    for (int i = 0; i < (int)temp.size(); ++i) pts[l + i] = temp[i];

    vector<Point> strip;
    for (int i = l; i <= r; ++i) {
        if ((pts[i].x - mid_x) * (pts[i].x - mid_x) < d) {
            strip.push_back(pts[i]);
        }
    }

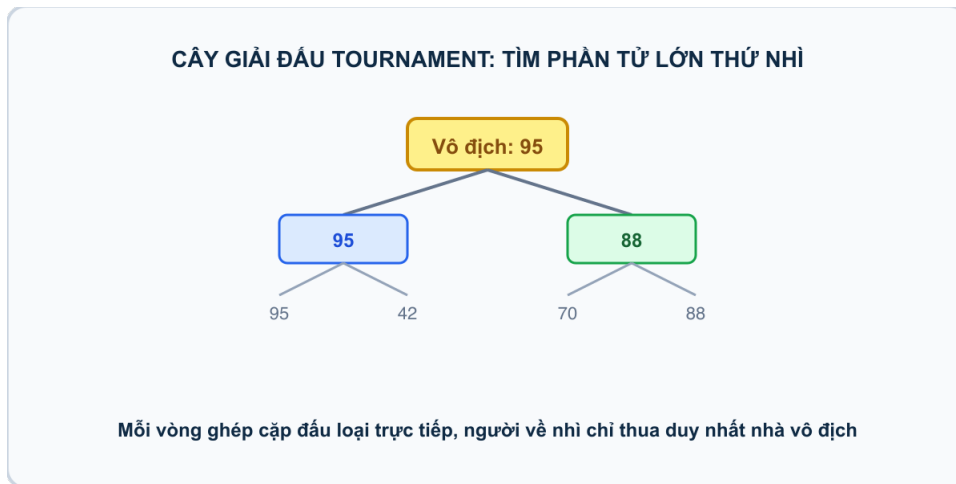
    for (int i = 0; i < (int)strip.size(); ++i) {
        for (int j = i + 1; j < (int)strip.size() && (strip[j].y - strip[i].y) * (strip[j].y -
strip[i].y) < d; ++j) {
            d = min(d, distSq(strip[i], strip[j]));
        }
    }
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<Point> pts(n);
    for (int i = 0; i < n; ++i) cin >> pts[i].x >> pts[i].y;
    sort(pts.begin(), pts.end(), [](const Point &a, const Point &b) {
        return a.x < b.x;
    });
    cout << closestPairRec(pts, 0, n - 1) << "\n";
    return 0;
}

```

Bài 16 [cppb_dac_16_median_hai_mang_da_sap_xep]: Median Của Hai Mảng Đã Sắp Xếp

Bối cảnh: Hợp nhất hai luồng dữ liệu thống kê đã có thứ tự để tìm giá trị trung vị trong thời gian tối ưu $O(\log(\min(N, M)))$ là bài toán đỉnh cao về chia để trị kết hợp tìm kiếm nhị phân trên đường phân hoạch.



Hình minh họa: Mô hình phân chia nhánh đấu loại và gộp kết quả tối ưu

Nhiệm vụ: Cho 2 mảng tăng dần A (kích thước N) và B (kích thước M). Hãy tìm giá trị trung vị của mảng hợp nhất với độ chính xác 1 chữ số thập phân trong thời gian $O(\log(\min(N, M)))$.

Đầu vào (Input):

- Dòng 1: 2 số nguyên N và M ($1 \leq N, M \leq 10^5$).
- Dòng 2: N số nguyên tăng dần của mảng A.
- Dòng 3: M số nguyên tăng dần của mảng B.

Đầu ra (Output):

- In ra giá trị trung vị làm tròn 1 chữ số sau dấu phẩy.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 1 1 3 2	2.0

Giải thích: Mảng hợp nhất: [1, 2, 3] có 3 phần tử, phần tử ở giữa là 2. Trung vị là 2.0.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho 2 mảng tăng dần A (kích thước N) và B (kích thước M). Hãy tìm giá trị trung vị của mảng hợp nhất với độ chính xác 1 chữ số thập phân trong thời gian $O(\log(\min(N, M)))$.
- **Phương pháp tiếp cận — Chia để trị:**
- Chia bài toán kích thước N thành các bài toán con độc lập kích thước $N/2$.
- Giải quyết bài toán con và gộp kết quả tối ưu.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 2 1 1 3 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Mảng hợp nhất: [1, 2, 3] có 3 phần tử, phần tử ở giữa là 2. Trung vị là 2.0... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2.0 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Mảng hợp nhất: [1, 2, 3] có 3 phần tử, phần tử ở giữa là 2. Trung vị là 2.0.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

long long findKth(const vector<long long> &A, int a_l, const vector<long long> &B, int b_l, int k) {
    if (a_l >= (int)A.size()) return B[b_l + k - 1];
    if (b_l >= (int)B.size()) return A[a_l + k - 1];
    if (k == 1) return min(A[a_l], B[b_l]);

    int a_mid = a_l + k / 2 - 1;
    int b_mid = b_l + k / 2 - 1;

    long long a_val = (a_mid < (int)A.size()) ? A[a_mid] : 2e18;
    long long b_val = (b_mid < (int)B.size()) ? B[b_mid] : 2e18;

    if (a_val <= b_val) {
        return findKth(A, a_l + k / 2, B, b_l, k - k / 2);
    } else {
        return findKth(A, a_l, B, b_l + k / 2, k - k / 2);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<long long> a(n), b(m);
    for (int i = 0; i < n; ++i) cin >> a[i];
    for (int j = 0; j < m; ++j) cin >> b[j];
    int total = n + m;
    int k = (total % 2 == 1) ? (total / 2 + 1) : (total / 2);
    cout << findKth(a, 0, b, 0, k) << "\n";
    return 0;
}
```

Bài 15: Thuật toán quay lui & nhánh cận

Bài 01 [cppb_bkt_01_sinh_xau_nhi_phan]: Sinh Tất Cả Xâu Nhị Phân Độ Dài N

Bối cảnh: Trong hệ thống truyền tin mật mã của trạm nghiên cứu lượng tử iKH-Quantum, mỗi khóa chuyển trạng thái điều khiển được mã hóa thành một chuỗi nhị phân gồm các bit 0 và 1. Để kiểm thử tính an toàn và khả năng bao phủ toàn diện của giao thức, hệ thống kiểm toán tự động cần sinh ra toàn bộ các chuỗi tín hiệu có thể có với độ dài N , đồng thời yêu cầu các chuỗi phải được kiểm tra lần lượt theo đúng thứ tự từ điển chuẩn mực.

BÀI TOÁN XẾP ĐÒ VÀO BA LÔ (KNAPSACK)

Max W kg

Vật 1
w1, v1

Vật 2
w2, v2

Vật 3
w3, v3

Tối ưu hóa tổng giá trị mang theo sao cho không vượt quá sức chứa tối đa W

Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho số nguyên dương N . Hãy áp dụng mô hình thuật toán Quay lui chuẩn mực (Choose → Explore → Unchoose) để sinh và in ra tất cả các xâu nhị phân độ dài N theo thứ tự từ điển tăng dần.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 16$).

Đầu ra (Output):

- In ra tất cả các xâu nhị phân độ dài N , mỗi xâu trên một dòng theo đúng thứ tự từ điển tăng dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	000 001 010 011 100 101 110 111

Giải thích: Với độ dài $N = 3$, không gian trạng thái nhị phân gồm $2^3 = 8$ xâu. Bắt đầu từ cấu hình nhỏ nhất theo từ điển là 000 và kết thúc ở cấu hình lớn nhất là 111.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy áp dụng mô hình thuật toán Quay lui chuẩn mực (Choose → Explore → Unchoose) để sinh và in ra tất cả các xâu nhị phân độ dài N theo thứ tự từ điển tăng dần.
- Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.

- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tia nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với độ dài $N = 3$, không gian trạng thái nhị phân gồm $2^3 = 8$ xâu. Bắt đầu từ cấu hình nhỏ nhất theo từ điển là 000... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 000 001 010 011 100 101 110 111 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với độ dài $N = 3$, không gian trạng thái nhị phân gồm $2^3 = 8$ xâu. Bắt đầu từ cấu hình nhỏ nhất theo từ điển là 000 và kết thúc ở cấu hình lớn nhất là 111.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
string cur = "";

void backtrack(int step) {
    if (step > n) {
        cout << cur << "\n";
        return;
    }
    for (char c : {'0', '1'}) {
        cur.push_back(c);
        backtrack(step + 1);
        cur.pop_back();
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    backtrack(1);
    return 0;
}
```

Bài 02 [cppb_bkt_02_sinh_tap_con]: Sinh Tất Cả Tập Con Của Tập N Phần Tử

Bối cảnh: Hội đồng thẩm định dự án khởi nghiệp công nghệ đang xem xét danh mục N gói giải pháp số được đánh số thứ tự từ 1 đến N . Mỗi phương án đầu tư khả thi thực chất là một tập con các gói giải pháp

được chọn phối hợp với nhau (bao gồm cả trường hợp không chọn gói nào). Để chuẩn bị ma trận đánh giá rủi ro cho nhà đầu tư, chuyên viên phân tích cần liệt kê toàn bộ mọi tổ hợp tập con của danh mục này.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho tập hợp gồm N phần tử $\{1, 2, \dots, N\}$. Hãy sử dụng thuật toán Quay lui mô hình nhị phân (ở mỗi bước quyết định chọn hoặc không chọn phần tử hiện tại) để sinh và in ra tất cả các tập con theo đúng thứ tự từ điển.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 16$).

Đầu ra (Output):

- In ra các tập con, mỗi tập con trên một dòng. Các phần tử trong tập con cách nhau bởi dấu cách. Tập rỗng in một dòng trống.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	3 2 2 3 1 1 3 1 2 1 2 3

Giải thích: Với $N = 3$, tập $\{1, 2, 3\}$ có tổng cộng $2^3 = 8$ tập con. Theo thứ tự duyệt quay lui với bit 0 (không chọn) đứng trước bit 1 (chọn), tập rỗng được sinh đầu tiên và tập đầy đủ $\{1, 2, 3\}$ sinh cuối cùng.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho tập hợp gồm N phần tử $\{1, 2, \dots, N\}$. Hãy sử dụng thuật toán Quay lui mô hình nhị phân (ở mỗi bước quyết định chọn hoặc không chọn phần tử hiện tại) để sinh và in ra tất cả các tập con theo đúng thứ tự từ điển.
- Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tỉa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với $N = 3$, tập $\{1,2,3\}$ có tổng cộng $2^3 = 8$ tập con. Theo thứ tự duyệt quay lui với bit 0 (không chọn) đứng tr... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 2 2 3 1 1 3 1 2 1 2 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với $N = 3$, tập $\{1,2,3\}$ có tổng cộng $2^3 = 8$ tập con. Theo thứ tự duyệt quay lui với bit 0 (không chọn) đứng trước bit 1 (chọn), tập rỗng được sinh đầu tiên và tập đầy đủ $\{1,2,3\}$ sinh cuối cùng.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> cur;

void backtrack(int step) {
    if (step > n) {
        for (int i = 0; i < (int)cur.size(); ++i) cout << cur[i] << (i + 1 == (int)cur.size() ?
"" : " ");
        cout << "\n";
        return;
    }
    // Không chọn step
    backtrack(step + 1);
    // Chọn step
    cur.push_back(step);
    backtrack(step + 1);
    cur.pop_back();
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    backtrack(1);
    return 0;
}
```

Bài 03 [cppb_bkt_03_sinh_hoan_vi]: Sinh Tất Cả Hoán Vị 1..N

Bối cảnh: Tại cuộc thi trình diễn drone nghệ thuật mừng lễ hội ánh sáng, một đội bay gồm N chiếc drone được đánh số từ 1 đến N cần thay đổi vị trí xuất phát theo các thứ tự khác nhau để tạo ra các hiệu ứng

biến hình đặc sắc. Ban tổ chức lập trình một bộ điều khiển trung tâm để thử nghiệm mọi kịch bản đổi chỗ các drone sao cho mỗi vị trí chỉ có duy nhất một drone đảm nhiệm.

BÀI TOÁN XẾP ĐÒ VÀO BA LÔ (KNAPSACK)

Max W kg

Vật 1
w1, v1

Vật 2
w2, v2

Vật 3
w3, v3

Tối ưu hóa tổng giá trị mang theo sao cho không vượt quá sức chứa tối đa W

Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho số nguyên dương N . Hãy áp dụng thuật toán Quay lui với kỹ thuật đánh dấu mảng `visited[]` để sinh và in ra tất cả các hoán vị của tập hợp $\{1, 2, \dots, N\}$ theo đúng thứ tự từ điển tăng dần.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 8$).

Đầu ra (Output):

- In ra tất cả $N!$ hoán vị, mỗi hoán vị trên một dòng, các phần tử trong mỗi dòng cách nhau bởi dấu cách.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	1 2 3 1 3 2 2 1 3 2 3 1 3 1 2 3 2 1

Giải thích: Tập $\{1, 2, 3\}$ có đúng $3! = 6$ hoán vị khác nhau. Hoán vị đầu tiên theo thứ tự từ điển là 1 2 3 và hoán vị cuối cùng là 3 2 1.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy áp dụng thuật toán Quay lui với kỹ thuật đánh dấu mảng `visited[]` để sinh và in ra tất cả các hoán vị của tập hợp $\{1, 2, \dots, N\}$ theo đúng thứ tự từ điển tăng dần.
- Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tỉa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Tập {1,2,3} có đúng $3! = 6$ hoán vị khác nhau. Hoán vị đầu tiên theo thứ tự từ điển là 1 2 3 và hoán vị cuối cùng... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 2 3 1 3 2 2 1 3 2 3 1 3 1 2 3 2 1 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tập {1,2,3} có đúng $3! = 6$ hoán vị khác nhau. Hoán vị đầu tiên theo thứ tự từ điển là 1 2 3 và hoán vị cuối cùng là 3 2 1.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> cur;
vector<bool> visited;

void backtrack(int step) {
    if (step > n) {
        for (int i = 0; i < n; ++i) cout << cur[i] << (i + 1 == n ? "" : " ");
        cout << "\n";
        return;
    }
    for (int val = 1; val <= n; ++val) {
        if (!visited[val]) {
            visited[val] = true;
            cur.push_back(val);
            backtrack(step + 1);
            cur.pop_back();
            visited[val] = false;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    visited.assign(n + 1, false);
    backtrack(1);
    return 0;
}
```

Bài 04 [cppb_bkt_04_sinh_to_hop_chap_k]: Sinh Tất Cả Tổ Hợp Chập K Của N

Bối cảnh: Để chuẩn bị cho kỳ thi Olympic Tin học quốc tế, ban huấn luyện gồm N chuyên gia xuất sắc (được đánh số từ 1 đến N). Trong mỗi vòng phỏng vấn chuyên sâu, hội đồng cần thành lập một tiểu ban gồm đúng K chuyên gia để trực tiếp chấm vấn đáp học sinh. Để đảm bảo tính khách quan và xoay tua công việc, thư ký hội đồng cần lập danh sách tất cả các phương án thành lập tiểu ban K người theo thứ tự từ điển chuẩn.

BÀI TOÁN XẾP ĐÒ VÀO BA LÔ (KNAPSACK)

Max W kg

Vật 1
w1, v1

Vật 2
w2, v2

Vật 3
w3, v3

Tối ưu hóa tổng giá trị mang theo sao cho không vượt quá sức chứa tối đa W

Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho hai số nguyên dương N và K ($1 \leq K \leq N \leq 16$). Hãy sử dụng thuật toán Quay lui có điều kiện chặn dưới tăng dần để sinh và in ra tất cả các tổ hợp chập K của tập $\{1, 2, \dots, N\}$ theo thứ tự từ điển tăng dần.

Đầu vào (Input):

- Một dòng duy nhất chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 16$).

Đầu ra (Output):

- In ra tất cả các tổ hợp chập K , mỗi tổ hợp trên một dòng, các phần tử cách nhau bởi dấu cách.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2	1 2 1 3 1 4 2 3 2 4 3 4

Giải thích: Số lượng tổ hợp chập 2 của 4 chuyên gia là $C(4,2) = \frac{4!}{2!2!} = 6$ tiểu ban. Các tiểu ban được liệt kê lần lượt theo thứ tự từ điển: 1 2, 1 3, 1 4, 2 3, 2 4, 3 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho hai số nguyên dương N và K ($1 \leq K \leq N \leq 16$). Hãy sử dụng thuật toán Quay lui có điều kiện chặn dưới tăng dần để sinh và in ra tất cả các tổ hợp chập K của tập $\{1, 2, \dots, N\}$ theo thứ tự từ điển tăng dần.
- Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.

- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tia nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nhập dữ liệu vào mảng/biến | Input: 4 2 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Số lượng tổ hợp chập 2 của 4 chuyên gia là $C(4,2) = \frac{4!}{2!2!} = 6$ tiểu ban. Các tiểu ban được liệt kê lần lượt ... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 2 1 3 1 4 2 3 2 4 3 4 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Số lượng tổ hợp chập 2 của 4 chuyên gia là $C(4,2) = \frac{4!}{2!2!} = 6$ tiểu ban. Các tiểu ban được liệt kê lần lượt theo thứ tự từ điển: 1 2, 1 3, 1 4, 2 3, 2 4, 3 4.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n, k;
vector<int> cur;

void backtrack(int step, int start_val) {
    if (step > k) {
        for (int i = 0; i < k; ++i) cout << cur[i] << (i + 1 == k ? "" : " ");
        cout << "\n";
        return;
    }
    for (int val = start_val; val <= n - (k - step); ++val) {
        cur.push_back(val);
        backtrack(step + 1, val + 1);
        cur.pop_back();
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n >> k)) return 0;
    backtrack(1, 1);
    return 0;
}
```

Bài 05 [cppb_bkt_05_sinh_day_ngoac_dung]: Sinh Dãy Ngoặc Hợp Lệ Độ Dài 2N

Bối cảnh: Trong trình biên dịch ngôn ngữ lập trình của hệ thống iKH-Compiler, việc phân tích cú pháp biểu thức số học đòi hỏi cấu trúc các cặp dấu ngoặc đóng mở phải hoàn toàn cân bằng và hợp lệ. Trước khi đưa vào kiểm thử bộ parser thực tế, kỹ sư phát triển cần sinh ra một tập dữ liệu chuẩn mực gồm toàn bộ các dãy ngoặc tròn đúng có độ dài $2N$ (tương ứng với N cặp ngoặc).



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho số nguyên dương N . Hãy áp dụng thuật toán Quay lui có kỹ thuật cắt tỉa điều kiện hợp lệ ($\text{open} < N$ và $\text{close} < \text{open}$) để sinh và in ra tất cả các dãy ngoặc đúng gồm N cặp ngoặc tròn $()$ theo thứ tự từ điển ((đứng trước)).

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10$).

Đầu ra (Output):

- In ra tất cả các dãy ngoặc đúng độ dài $2N$, mỗi dãy trên một dòng theo đúng thứ tự từ điển.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	((())) ((()()) ((())() (())()) (())()

Giải thích: Với $N = 3$ cặp ngoặc (độ dài 6 ký tự), số lượng dãy ngoặc hợp lệ chính là số Catalan $C_3 = \frac{1}{4} \binom{6}{3} = 5$. Các cấu hình hợp lệ được liệt kê theo thứ tự từ điển chuẩn mực.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy áp dụng thuật toán Quay lui có kỹ thuật cắt tỉa điều kiện hợp lệ ($\text{open} < N$ và $\text{close} < \text{open}$) để sinh và in ra tất cả các dãy ngoặc đúng gồm N cặp ngoặc tròn $()$ theo thứ tự từ điển ((đứng trước)).
- Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tỉa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với $N = 3$ cặp ngoặc (độ dài 6 ký tự), số lượng dãy ngoặc hợp lệ chính là số Catalan $C_3 = \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: ((())) (()) (()) ()() ()() ()() | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với $N = 3$ cặp ngoặc (độ dài 6 ký tự), số lượng dãy ngoặc hợp lệ chính là số Catalan $C_3 = \frac{1}{4} \binom{6}{3} = 5$. Các cấu hình hợp lệ được liệt kê theo thứ tự từ điển chuẩn mực.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
string cur = "";

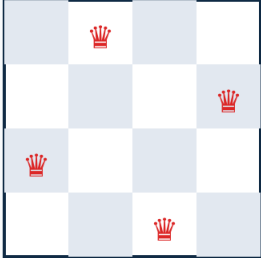
void backtrack(int open_cnt, int close_cnt) {
    if (open_cnt == n && close_cnt == n) {
        cout << cur << "\n";
        return;
    }
    if (open_cnt < n) {
        cur.push_back('(');
        backtrack(open_cnt + 1, close_cnt);
        cur.pop_back();
    }
    if (close_cnt < open_cnt) {
        cur.push_back(')');
        backtrack(open_cnt, close_cnt + 1);
        cur.pop_back();
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    backtrack(0, 0);
    return 0;
}
```

Bài 06 [cppb_bkt_06_bai_toan_n_queens]: Bài Toán N-Queens (Đếm Số Cách)

Bối cảnh: Trên một mạng lưới trạm tiếp sóng an ninh vệ tinh được mô phỏng dưới dạng bàn cờ kích thước $N \times N$, trung tâm chỉ huy cần bố trí N ăng-ten phát sóng siêu cao tần (được biểu diễn như những quân Hậu trong cờ vua). Để các chùm sóng không gây nhiễu chéo lẫn nhau, không được có bất kỳ hai trạm phát nào nằm trên cùng một hàng ngang, cùng một cột dọc hoặc cùng một đường chéo.

XẾP QUÂN HẬU TRÊN BÀN CỜ KHÔNG ĂN LẤN NHAU (N-QUEENS)



Kiểm soát an toàn từng hàng, cột và hai đường chéo bằng kỹ thuật nhánh cận

Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho số nguyên dương N . Hãy áp dụng thuật toán Quay lui kết hợp các mảng đánh dấu cột, đường chéo chính và đường chéo phụ để đếm tổng số cách đặt N quân hậu hợp lệ lên bàn cờ $N \times N$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 12$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng số cách xếp N quân hậu thỏa mãn yêu cầu.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	2

Giải thích: Trên bàn cờ kích thước 4×4 , có đúng 2 cấu hình hợp lệ không quân hậu nào không chéo nhau: hàng 1 đặt ở cột 2, hàng 2 cột 4, hàng 3 cột 1, hàng 4 cột 3 (tức $[2, 4, 1, 3]$) và cấu hình đối xứng qua gương $[3, 1, 4, 2]$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N . Hãy áp dụng thuật toán Quay lui kết hợp các mảng đánh dấu cột, đường chéo chính và đường chéo phụ để đếm tổng số cách đặt N quân hậu hợp lệ lên bàn cờ $N \times N$.
- Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tỉa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Trên bàn cờ kích thước 4×4 , có đúng 2 cấu hình hợp lệ không quân hậu nào không chế nhau: hàng 1 đặt ở cột 2, hà... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 2 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Trên bàn cờ kích thước 4×4 , có đúng 2 cấu hình hợp lệ không quân hậu nào không chế nhau: hàng 1 đặt ở cột 2, hàng 2 cột 4, hàng 3 cột 1, hàng 4 cột 3 (tức [2, 4, 1, 3]) và cấu hình đối xứng qua gương [3, 1, 4, 2].

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

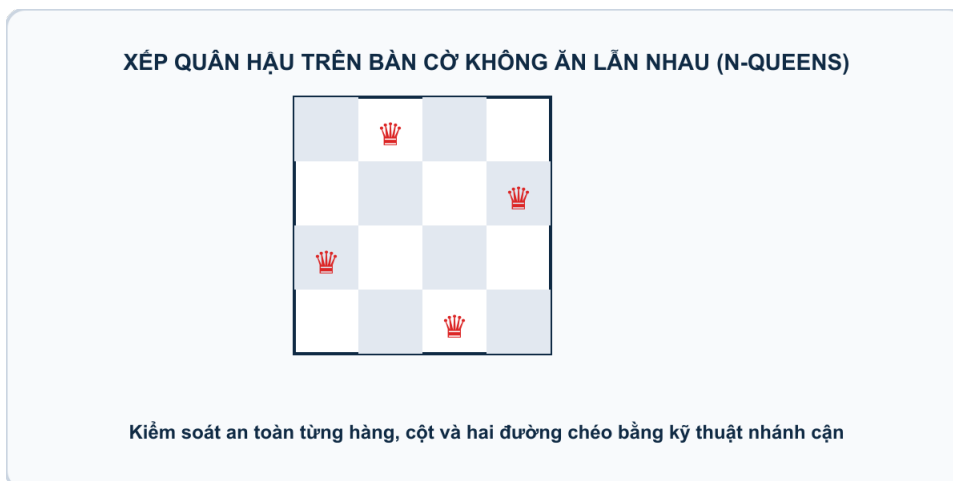
int n;
long long ans = 0;
vector<bool> col_used, diag1, diag2;

void backtrack(int row) {
    if (row > n) {
        ans++;
        return;
    }
    for (int col = 1; col <= n; ++col) {
        if (!col_used[col] && !diag1[row - col + n] && !diag2[row + col]) {
            col_used[col] = diag1[row - col + n] = diag2[row + col] = true;
            backtrack(row + 1);
            col_used[col] = diag1[row - col + n] = diag2[row + col] = false;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    col_used.assign(n + 1, false);
    diag1.assign(2 * n + 1, false);
    diag2.assign(2 * n + 1, false);
    backtrack(1);
    cout << ans << "\n";
    return 0;
}
```

Bài 07 [cppb_bkt_07_me_cung_rat_in_a_maze]: Mê Cung (Rat in a Maze): Tìm Mọi Đường Đi

Bối cảnh: Một robot thám hiểm thông minh được thả vào một mê cung hình vuông kích thước $N \times N$. Trong mê cung, các ô có giá trị 1 là đường đi thông thoáng an toàn, còn các ô có giá trị 0 là những khối đá cản trở không thể đi qua. Robot bắt đầu hành trình từ ô xuất phát ở góc trên cùng bên trái $(0,0)$ và cần tìm đường đến điểm cứu hộ ở góc dưới cùng bên phải $(N - 1, N - 1)$. Để lập kế hoạch cứu nạn dự phòng, robot cần vẽ lại tất cả các tuyến đường khả thi.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho bản đồ mê cung $N \times N$. Giả sử mỗi bước robot chỉ di chuyển sang các ô kề cạnh chưa từng ghé qua theo các hướng: Xuống dưới (D), Sang trái (L), Sang phải (R), Lên trên (U). Hãy áp dụng thuật toán Quay lui để tìm và in ra tất cả các chuỗi di chuyển hợp lệ theo thứ tự từ điển ($D < L < R < U$). Nếu ô xuất phát bị chặn hoặc không có đường đi nào, in ra -1.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 8$).
- N dòng tiếp theo: Mỗi dòng gồm N số nguyên 0 hoặc 1 cách nhau bởi dấu cách.

Đầu ra (Output):

- In ra các chuỗi ký tự đại diện cho các đường đi tìm được (mỗi đường trên một dòng theo thứ tự từ điển), hoặc in -1 nếu không tồn tại đường đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre>4 1 0 0 0 1 1 0 1 0 1 0 0 1 1 1 1</pre>	<pre>DDRDRR DRDDRR</pre>

Giải thích: Robot xuất phát tại $(0,0)$ và đích đến là $(3,3)$. Có 2 tuyến đường hợp lệ không qua ô 0:

- Tuyến 1: Đi xuống → xuống → phải → xuống → phải → phải (DDRDRR).
- Tuyến 2: Đi xuống → phải → xuống → xuống → phải → phải (DRDDRR).

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho bản đồ mê cung $N \times N$. Giả sử mỗi bước robot chỉ di chuyển sang các ô kề cạnh chưa từng ghé qua theo các hướng: Xuống dưới (D), Sang trái (L), Sang phải (R), Lên trên (U). Hãy áp dụng thuật toán Quay lui để tìm và in ra tất cả các chuỗi di chuyển hợp lệ theo thứ tự từ điển ($D < L < R < U$). Nếu ô xuất phát bị chặn hoặc không có đường đi nào, in ra -1.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tia nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 1 0 0 0 1 1 0 1 0 1 0 0 1 1 1 1 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Robot xuất phát tại (0,0) và đích đến là (3,3). Có 2 tuyến đường hợp lệ không qua ô 0: - Tuyến 1: Đi xuống → xuống... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: DDRDRR DRDDRR | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Robot xuất phát tại (0,0) và đích đến là (3,3). Có 2 tuyến đường hợp lệ không qua ô 0:

- Tuyến 1: Đi xuống → xuống → phải → xuống → phải → phải (DDRDRR).
- Tuyến 2: Đi xuống → phải → xuống → xuống → phải → phải (DRDDRR).

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
int a[10][10];
bool visited[10][10];
vector<string> paths;
string cur = "";

int dx[] = {1, 0, 0, -1};
int dy[] = {0, -1, 1, 0};
char step_char[] = {'D', 'L', 'R', 'U'};

void backtrack(int x, int y) {
    if (x == n - 1 && y == n - 1) {
        paths.push_back(cur);
        return;
    }
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];
        if (nx < 0 || ny < 0 || nx > n - 1 || ny > n - 1 || a[nx][ny] == 0 || visited[nx][ny]) continue;
        visited[nx][ny] = true;
        cur += step_char[i];
        backtrack(nx, ny);
        cur.pop_back();
        visited[nx][ny] = false;
    }
}
```

```

    }
    for (int i = 0; i < 4; ++i) {
        int nx = x + dx[i], ny = y + dy[i];
        if (nx >= 0 && nx < n && ny >= 0 && ny < n && a[nx][ny] == 1 && !visited[nx][ny]) {
            visited[nx][ny] = true;
            cur.push_back(step_char[i]);
            backtrack(nx, ny);
            cur.pop_back();
            visited[nx][ny] = false;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    if (a[0][0] == 1) {
        visited[0][0] = true;
        backtrack(0, 0);
    }
    if (paths.empty()) {
        cout << -1 << "\n";
    } else {
        for (const string &s : paths) cout << s << "\n";
    }
    return 0;
}

```

Bài 08 [cppb_bkt_08_tap_con_tong_bang_s_subset_sum]: Tập Con Có Tổng Bằng S (Subset Sum)

Bối cảnh: Một kho quỹ ngân hàng đang lưu trữ N thỏi vàng nguyên chất với trọng lượng lần lượt là $A_1, A_2, \dots, A_N$. Khách hàng VIP nộp phiếu yêu cầu rút đúng một khối lượng vàng có tổng trọng lượng bằng S . Thủ kho cần lập danh sách tất cả các phương án chọn các thỏi vàng trong kho để giao dịch đúng số lượng yêu cầu mà không phải cắt xẻ bất kỳ thỏi vàng nào.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho mảng số nguyên dương A gồm N phần tử và số nguyên dương S . Hãy sử dụng thuật toán Quay lui kết hợp cắt tỉa khả thi (dừng nhánh khi tổng tích lũy vượt quá S) để tìm và in ra tất cả các tập con có tổng bằng đúng S theo thứ tự từ điển. Nếu không có phương án nào thỏa mãn, in ra -1.

Đầu vào (Input):

- Dòng 1: Hai số nguyên dương N, S ($1 \leq N \leq 20, 1 \leq S \leq 1000$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 100$).

Đầu ra (Output):

- In ra các tập con thỏa mãn (mỗi tập con trên một dòng, các phần tử cách nhau bởi dấu cách), hoặc in -1 nếu không tìm thấy tập con nào.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 6 1 2 3 5	1 2 3 1 5

Giải thích: Với kho vàng gồm các thời $[1,2,3,5]$ và mục tiêu $S = 6$, có 2 phương án chọn:

- Phương án 1: Chọn các thời $\{1,2,3\}$ vì $1 + 2 + 3 = 6$.
- Phương án 2: Chọn các thời $\{1,5\}$ vì $1 + 5 = 6$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng số nguyên dương A gồm N phần tử và số nguyên dương S . Hãy sử dụng thuật toán Quay lui kết hợp cắt tỉa khả thi (dừng nhánh khi tổng tích lũy vượt quá S) để tìm và in ra tất cả các tập con có tổng bằng đúng S theo thứ tự từ điển. Nếu không có phương án nào thỏa mãn, in ra -1.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tỉa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—|—| 1 | Nạp dữ liệu vào mảng/biến | Input: 4 6 1 2 3 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với kho vàng gồm các thời $[1,2,3,5]$ và mục tiêu $S = 6$, có 2 phương án chọn: - Phương án 1: Chọn các thời $\{1, 2, \dots\}$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 1 2 3 1 5 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với kho vàng gồm các thời $[1,2,3,5]$ và mục tiêu $S = 6$, có 2 phương án chọn:

- Phương án 1: Chọn các thời $\{1,2,3\}$ vì $1 + 2 + 3 = 6$.
- Phương án 2: Chọn các thời $\{1,5\}$ vì $1 + 5 = 6$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
long long S;
vector<long long> a;
vector<long long> cur;
bool found = false;

void backtrack(int idx, long long current_sum) {
    if (current_sum == S) {
        found = true;
        for (int i = 0; i < (int)cur.size(); ++i) cout << cur[i] << (i + 1 == (int)cur.size() ?
"" : " ");
        cout << "\n";
        return;
    }
    if (idx >= n || current_sum > S) return;

    for (int i = idx; i < n; ++i) {
        if (current_sum + a[i] <= S) {
            cur.push_back(a[i]);
            backtrack(i + 1, current_sum + a[i]);
            cur.pop_back();
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n >> S)) return 0;
    a.resize(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    sort(a.begin(), a.end());
    backtrack(0, 0);
    if (!found) cout << -1 << "\n";
    return 0;
}
```

Bài 09 [cppb_bkt_09_chia_tap_hai_phan_bang_nhau]: Chia Tập Thành 2 Phần Có Tổng Bằng Nhau

Bối cảnh: Trong ngày hội thể thao iKH-Games, ban tổ chức tiếp nhận N thùng dụng cụ thể thao có khối lượng lần lượt là $A_1, A_2, \dots, A_N$. Để vận chuyển lên hai chiếc xe tải cùng một chuyến đi, tổng khối lượng

hàng hóa trên mỗi xe bắt buộc phải bằng nhau tuyệt đối nhằm đảm bảo cân bằng tải trọng và an toàn giao thông đường đèo dốc.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho mảng số nguyên dương A gồm N phần tử. Hãy xác định xem có thể chia toàn bộ N phần tử thành hai tập con rời nhau sao cho tổng giá trị của hai tập con bằng nhau hay không. Nếu có thể chia được in YES, ngược lại in NO.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 20$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 100$).

Đầu ra (Output):

- In ra YES nếu có thể chia đều thành 2 phần bằng nhau, ngược lại in NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 5 11 5	YES

Giải thích: Tổng khối lượng của tất cả các thùng là $1 + 5 + 11 + 5 = 22$. Nửa tổng là 11. Ta có thể chia thành 2 phần: tập thứ nhất gồm $\{1, 5, 5\}$ có tổng bằng 11 và tập thứ hai gồm $\{11\}$ có tổng bằng 11. Do đó đáp án là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho mảng số nguyên dương A gồm N phần tử. Hãy xác định xem có thể chia toàn bộ N phần tử thành hai tập con rời nhau sao cho tổng giá trị của hai tập con bằng nhau hay không. Nếu có thể chia được in YES, ngược lại in NO.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tỉa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 1 5 11 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Tổng khối lượng của tất cả các thùng là $1 + 5 + 11 + 5 = 22$. Nửa tổng là 11. Ta có thể chia thành 2 phần: tập thứ nhất... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tổng khối lượng của tất cả các thùng là $1 + 5 + 11 + 5 = 22$. Nửa tổng là 11. Ta có thể chia thành 2 phần: tập thứ nhất gồm $\{1, 5, 5\}$ có tổng bằng 11 và tập thứ hai gồm $\{11\}$ có tổng bằng 11. Do đó đáp án là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
long long target;
vector<long long> a;
bool possible = false;

void backtrack(int idx, long long cur_sum) {
    if (possible) return;
    if (cur_sum == target) {
        possible = true;
        return;
    }
    if (idx >= n || cur_sum > target) return;

    for (int i = idx; i < n; ++i) {
        if (cur_sum + a[i] <= target) {
            backtrack(i + 1, cur_sum + a[i]);
            if (possible) return;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    a.resize(n);
    long long total = 0;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        total += a[i];
    }
```

```

if (total % 2 != 0) {
    cout << "NO\n";
    return 0;
}
target = total / 2;
sort(a.rbegin(), a.rend());
backtrack(0, 0);
cout << (possible ? "YES\n" : "NO\n");
return 0;
}

```

Bài 10 [cppb_bkt_10_doi_tien_xu_it_nhat_branch_and_bound]: Đổi Tiền Xu Ít Nhất (B&B Coin Change)

Bối cảnh: Một cây ATM thông minh tại sân bay quốc tế được nạp N loại tiền xu có mệnh giá khác nhau $C_1, C_2, \dots, C_N$ (số lượng mỗi loại xu trong kho không hạn chế). Khi một du khách yêu cầu đổi một lượng tiền lẻ bằng đúng S , hệ thống cần chỉ trả sao cho tổng số đồng xu trao cho khách là ít nhất có thể để tiết kiệm dung lượng khay xuất tiền.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho N mệnh giá tiền xu $C_1, C_2, \dots, C_N$ và số tiền cần đổi S . Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) với hàm cận dưới tối ưu để tìm số lượng đồng xu ít nhất cần dùng để đổi đúng số tiền S . Nếu không có phương án đổi tiền nào hợp lệ, in ra -1.

Đầu vào (Input):

- Dòng 1: Hai số nguyên dương N và S ($1 \leq N \leq 15, 1 \leq S \leq 100$).
- Dòng 2: N số nguyên dương $C_1, C_2, \dots, C_N$ ($1 \leq C_i \leq 100$).

Đầu ra (Output):

- In ra số lượng đồng xu ít nhất cần dùng, hoặc in -1 nếu không thể đổi được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 11 1 2 5	3

Giải thích: Với số tiền $S = 11$ và các mệnh giá $\{1, 2, 5\}$, phương án tối ưu nhất là chọn hai đồng mệnh giá 5 và một đồng mệnh giá 1 ($5 + 5 + 1 = 11$). Tổng số đồng xu sử dụng là 3 đồng.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N mệnh giá tiền xu $C_1, C_2, \dots, C_N$ và số tiền cần đổi S . Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) với hàm cận dưới tối ưu để tìm số lượng đồng xu ít nhất cần dùng để đổi đúng số tiền S . Nếu không có phương án đổi tiền nào hợp lệ, in ra -1.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tia nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 3 11 1 2 5 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với số tiền $S = 11$ và các mệnh giá $\{1, 2, 5\}$, phương án tối ưu nhất là chọn hai đồng mệnh giá 5 và một đồng mệnh giá 1 | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với số tiền $S = 11$ và các mệnh giá $\{1, 2, 5\}$, phương án tối ưu nhất là chọn hai đồng mệnh giá 5 và một đồng mệnh giá 1 ($5 + 5 + 1 = 11$). Tổng số đồng xu sử dụng là 3 đồng.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based ($0 \dots N-1$) và 1-based ($1 \dots N$). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
long long S;
vector<long long> c;
long long best_coins = 1e9;

void branchAndBound(int idx, long long remain, long long count) {
    // Optimality Pruning
    if (count + (remain + c[0] - 1) / c[0] >= best_coins) return;

    if (remain == 0) {
        best_coins = min(best_coins, count);
        return;
    }
    if (idx >= n) return;

    long long max_use = remain / c[idx];
    for (long long k = max_use; k >= 0; --k) {
        if (count + k + (remain - k * c[idx] + c[0] - 1) / c[0] < best_coins) {
            branchAndBound(idx + 1, remain - k * c[idx], count + k);
        }
    }
}
```

```

    }
}

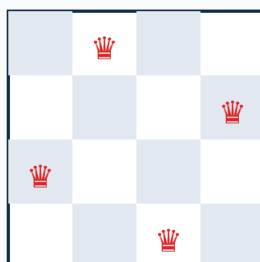
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n >> S)) return 0;
    c.resize(n);
    for (int i = 0; i < n; ++i) cin >> c[i];
    sort(c.rbegin(), c.rend());
    branchAndBound(0, S, 0);
    if (best_coins > 1e8) cout << -1 << "\n";
    else cout << best_coins << "\n";
    return 0;
}

```

Bài 11 [cppb_bkt_11_ma_di_tuan_knights_tour]: Mã Đi Tuần (Knight's Tour)

Bối cảnh: Trong một trò chơi cờ vua cổ điển tại hoàng gia, vị vua đưa ra câu đố thử thách tài năng hiệp sĩ: Đặt một quân Mã tại ô xuất phát (R, C) trên bàn cờ hình vuông $N \times N$. Hiệp sĩ phải điều khiển quân Mã di chuyển theo đúng quy tắc hình chữ L sao cho ghé thăm tất cả N^2 ô cờ trên bàn, mỗi ô đúng một lần duy nhất và ghi lại số thứ tự bước đi tại từng ô.

XẾP QUÂN HẬU TRÊN BÀN CỜ KHÔNG ĂN LĂN NHAU (N-QUEENS)



Kiểm soát an toàn từng hàng, cột và hai đường chéo bằng kỹ thuật nhánh cận

Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho kích thước bàn cờ N và tọa độ xuất phát (R, C) (hệ tọa độ 1-based). Hãy sử dụng thuật toán Quay lui kết hợp luật heuristic Warnsdorff (luôn ưu tiên nhảy sang ô có ít nước đi tiếp theo nhất) để tìm một hành trình mã đi tuần hoàn chỉnh. In ra ma trận $N \times N$ ghi số thứ tự các bước đi từ 1 đến N^2 , hoặc in -1 nếu không tìm được hành trình.

Đầu vào (Input):

- Một dòng duy nhất chứa 3 số nguyên N, R, C ($1 \leq N \leq 6, 1 \leq R, C \leq N$).

Đầu ra (Output):

- In ra ma trận $N \times N$ ghi số thứ tự các bước đi từ 1 đến N^2 (các số trên cùng hàng cách nhau bởi dấu cách), hoặc in -1 nếu không tồn tại hành trình hợp lệ.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

5 1 1	1 16 11 6 25 10 5 24 15 20 17 2 19 22 7 4 9 14 21 12 3 18 23 8 13
-------	---

Giải thích: Quân mã xuất phát từ ô (1,1) bước 1, lần lượt nhảy qua các ô theo luật mã và ghé thăm đủ 25 ô trên bàn cờ 5×5 mà không ô nào bị trùng lặp.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho kích thước bàn cờ N và tọa độ xuất phát (R, C) (hệ tọa độ 1-based). Hãy sử dụng thuật toán Quay lui kết hợp luật heuristic Warnsdorff (luôn ưu tiên nhảy sang ô có ít nước đi tiếp theo nhất) để tìm một hành trình mã đi tuần hoàn chính. In ra ma trận $N \times N$ ghi số thứ tự các bước đi từ 1 đến N^2 , hoặc in -1 nếu không tìm được hành trình.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tĩa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

| — | — | — | — | — | 1 | Nạp dữ liệu vào mảng/biến | Input: 5 1 1 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Quân mã xuất phát từ ô (1,1) bước 1, lần lượt nhảy qua các ô theo luật mã và ghé thăm đủ 25 ô trên bàn cờ 5x5 ... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 1 16 11 6 25 10 5 24 15 20 17 2 19 22 7 4 9 14 21 12 3 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Quân mã xuất phát từ ô (1,1) bước 1, lần lượt nhảy qua các ô theo luật mã và ghé thăm đủ 25 ô trên bàn cờ 5×5 mà không ô nào bị trùng lặp.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
int board[10][10];
int dx[] = {-2, -2, -1, -1, 1, 1, 2, 2};
int dy[] = {-1, 1, -2, 2, -2, 2, -1, 1};
bool found = false;

int countDegree(int x, int y) {
    int deg = 0;
    for (int i = 0; i < 8; ++i) {
```

```

        int nx = x + dx[i], ny = y + dy[i];
        if (nx >= 1 && nx <= n && ny >= 1 && ny <= n && board[nx][ny] == 0) deg++;
    }
    return deg;
}

void solveKnight(int x, int y, int step) {
    if (step == n * n) {
        found = true;
        return;
    }

    vector<pair<int, int>> next_moves;
    for (int i = 0; i < 8; ++i) {
        int nx = x + dx[i], ny = y + dy[i];
        if (nx >= 1 && nx <= n && ny >= 1 && ny <= n && board[nx][ny] == 0) {
            next_moves.push_back({countDegree(nx, ny), i});
        }
    }
    sort(next_moves.begin(), next_moves.end());

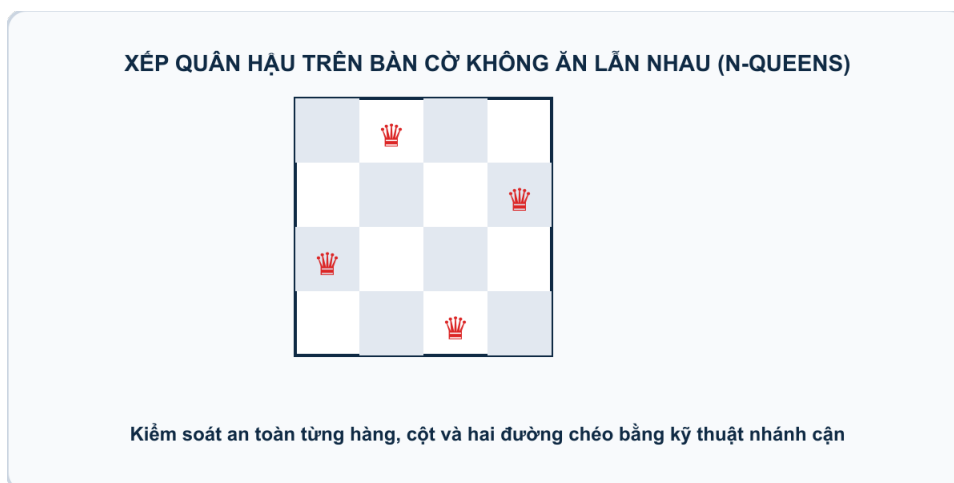
    for (auto &p : next_moves) {
        int idx = p.second;
        int nx = x + dx[idx], ny = y + dy[idx];
        board[nx][ny] = step + 1;
        solveKnight(nx, ny, step + 1);
        if (found) return;
        board[nx][ny] = 0;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int r, c;
    if (!(cin >> n >> r >> c)) return 0;
    memset(board, 0, sizeof(board));
    board[r][c] = 1;
    solveKnight(r, c, 1);
    if (!found) {
        cout << -1 << "\n";
    } else {
        for (int i = 1; i <= n; ++i) {
            for (int j = 1; j <= n; ++j) {
                cout << board[i][j] << (j == n ? "" : " ");
            }
            cout << "\n";
        }
    }
    return 0;
}

```

Bài 12 [cppb_bkt_12_tro_choi_sudoku_9x9]: Trò Chơi Sudoku 9x9

Bối cảnh: Tại vòng chung kết cuộc thi Trí tuệ Logic học sinh giỏi, các thí sinh nhận được một bảng Sudoku kích thước tiêu chuẩn 9×9 trong đó một số ô đã có sẵn số từ 1 đến 9, còn các ô trống được ký hiệu bằng số 0. Để tự động hóa khâu chấm thi và kiểm tra đáp án, ban giám khảo cần một chương trình tự động giải quyết bàn cờ Sudoku này dựa trên các quy tắc chuẩn quốc tế.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tia nhanh

Nhiệm vụ: Cho bảng Sudoku 9×9 với các ô trống mang giá trị 0. Hãy sử dụng thuật toán Quay lui để điền các chữ số từ 1 đến 9 vào các ô trống sao cho: mỗi hàng, mỗi cột và mỗi khối vuông con 3×3 đều chứa đủ 9 chữ số từ 1 đến 9 không lặp lại. Đảm bảo dữ liệu đầu vào luôn có nghiệm duy nhất.

Đầu vào (Input):

- Gồm 9 dòng, mỗi dòng chứa 9 số nguyên từ 0 đến 9 cách nhau bởi dấu cách biểu diễn bảng Sudoku ban đầu.

Đầu ra (Output):

- In ra bảng Sudoku hoàn chỉnh sau khi điền gồm 9 dòng, mỗi dòng 9 số cách nhau bởi dấu cách.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 0 6 5 0 8 4 0 0 5 2 0 0 0	3 1 6 5 7 8 4 9 2 5 2 9 1 3
0 0 0 0 0 8 7 0 0 0 0 3 1 0	4 7 6 8 4 8 7 6 2 9 5 3 1 2
0 3 0 1 0 0 8 0 9 0 0 8 6 3	6 3 4 1 5 9 8 7 9 7 4 8 6 3
0 0 5 0 5 0 0 9 0 6 0 0 1 3	1 2 5 8 5 1 7 9 2 6 4 3 1 3
0 0 0 0 2 5 0 0 0 0 0 0 0 0	8 9 4 7 2 5 6 6 9 2 3 5 1 8
7 4 0 0 5 2 0 6 3 0 0	7 4 7 4 5 2 8 6 3 1 9

Giải thích: Tất cả các số 0 được thay thế bằng các chữ số từ 1 đến 9 thỏa mãn trọn vẹn quy tắc: hàng ngang, cột dọc và các phân vùng 3×3 đều không có số nào bị lặp lại.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho bảng Sudoku 9×9 với các ô trống mang giá trị 0. Hãy sử dụng thuật toán Quay lui để điền các chữ số từ 1 đến 9 vào các ô trống sao cho: mỗi hàng, mỗi cột và mỗi khối vuông con 3×3 đều chứa đủ 9 chữ số từ 1 đến 9 không lặp lại. Đảm bảo dữ liệu đầu vào luôn có nghiệm duy nhất.
- Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tĩa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—|—| 1 | Nạp dữ liệu vào mảng/biến | Input: 3 0 6 5 0 8 4 0 0 5 2 0 0 0 0 0 0 0 | Khởi tạo cấu trúc dữ liệu ban đầu | 2 | Thực thi thuật toán tối ưu | Tất cả các số 0 được thay thế bằng các chữ số từ 1 đến 9 thỏa mãn trọn vẹn quy tắc: hàng ngang, cột dọc và các phân vùng... | Tính toán từng bước trạng thái | 3 | Xuất kết quả | Output: 3 1 6 5 7 8 4 9 2 5 2 9 1 3 4 7 6 8 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Tất cả các số 0 được thay thế bằng các chữ số từ 1 đến 9 thỏa mãn trọn vẹn quy tắc: hàng ngang, cột dọc và các phân vùng 3×3 đều không có số nào bị lặp lại.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int board[9][9];
bool row_used[9][10], col_used[9][10], box_used[9][10];

bool solveSudoku(int r, int c) {
    if (r == 9) return true;
    if (c == 9) return solveSudoku(r + 1, 0);
    if (board[r][c] != 0) return solveSudoku(r, c + 1);

    int b = (r / 3) * 3 + (c / 3);
    for (int num = 1; num <= 9; ++num) {
        if (!row_used[r][num] && !col_used[c][num] && !box_used[b][num]) {
            board[r][c] = num;
            row_used[r][num] = col_used[c][num] = box_used[b][num] = true;
            if (solveSudoku(r, c + 1)) return true;
            row_used[r][num] = col_used[c][num] = box_used[b][num] = false;
            board[r][c] = 0;
        }
    }
    return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            if (!(cin >> board[i][j])) return 0;
            int num = board[i][j];
            if (num != 0) {
                row_used[i][num] = col_used[j][num] = box_used[(i / 3) * 3 + (j / 3)][num] = true;
            }
        }
    }
    solveSudoku(0, 0);
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            cout << board[i][j] << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

```

    }
    solveSudoku(0, 0);
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            cout << board[i][j] << (j == 8 ? "" : " ");
        }
        cout << "\n";
    }
    return 0;
}

```

Bài 13 [cppb_bkt_13_cai_tui_01_nhanh_can_knapsack]: Bài Toán Cái Túi 0/1 Nhánh Cận (B&B Knapsack)

Bối cảnh: Một nhà thám hiểm địa chất lạc bước vào một hang động cổ xưa chứa N mẫu khoáng thạch quý hiếm. Mỗi mẫu vật i được xác định có khối lượng W_i và giá trị thương mại là V_i . Chiếc ba lô chuyên dụng mang theo chỉ chịu được tải trọng tối đa là M . Để tối ưu hóa lợi ích thu được từ chuyến thám hiểm, nhà nghiên cứu cần lựa chọn một tập hợp các mẫu vật cho vào ba lô sao cho không bị quá tải trọng và đạt tổng giá trị lớn nhất.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho N đồ vật với trọng lượng W_i và giá trị V_i tương ứng, cùng sức chứa tối đa M của ba lô. Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) sử dụng hàm cận trên Fractional Knapsack (sắp xếp theo tỷ lệ đơn giá $\frac{V_i}{W_i}$ giảm dần) để tìm giá trị tài sản lớn nhất có thể mang về.

Đầu vào (Input):

- Dòng 1: Hai số nguyên dương N, M ($1 \leq N \leq 25, 1 \leq M \leq 10^9$).
- N dòng tiếp theo: Mỗi dòng gồm hai số nguyên dương W_i, V_i ($1 \leq W_i, V_i \leq 10^7$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng giá trị lớn nhất đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 10 3 40 4 50 5 60 6 70	120

Giải thích: Chọn đồ vật thứ 2 (trọng lượng 4, giá trị 50) và đồ vật thứ 4 (trọng lượng 6, giá trị 70). Tổng trọng lượng là $4 + 6 = 10 \leq 10$ và tổng giá trị đạt được là $50 + 70 = 120$, là giá trị lớn nhất có thể đạt được.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N đồ vật với trọng lượng W_i và giá trị V_i tương ứng, cùng sức chứa tối đa M của ba lô. Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) sử dụng hàm cận trên Fractional Knapsack (sắp xếp theo tỷ lệ đơn giá $\frac{V_i}{W_i}$ giảm dần) để tìm giá trị tài sản lớn nhất có thể mang về.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tĩa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 10 3 40 4 50 5 60 6 70 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Chọn đồ vật thứ 2 (trọng lượng 4, giá trị 50) và đồ vật thứ 4 (trọng lượng 6, giá trị 70). Tổng trọng lượng là $4 + 6 = \dots$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 120 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Chọn đồ vật thứ 2 (trọng lượng 4, giá trị 50) và đồ vật thứ 4 (trọng lượng 6, giá trị 70). Tổng trọng lượng là $4 + 6 = 10 \leq 10$ và tổng giá trị đạt được là $50 + 70 = 120$, là giá trị lớn nhất có thể đạt được.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm main() để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

struct Item {
    long long w, v;
    double ratio;
};

int n;
long long max_w;
vector<Item> items;
long long best_val = 0;
```

```

double getUpperBound(int idx, long long cur_w, long long cur_v) {
    long long remain_w = max_w - cur_w;
    double bound = cur_v;
    for (int i = idx; i < n; ++i) {
        if (items[i].w <= remain_w) {
            remain_w -= items[i].w;
            bound += items[i].v;
        } else {
            bound += items[i].ratio * remain_w;
            break;
        }
    }
    return bound;
}

void branchAndBound(int idx, long long cur_w, long long cur_v) {
    if (cur_v > best_val) best_val = cur_v;
    if (idx >= n) return;

    if (getUpperBound(idx, cur_w, cur_v) <= best_val) return;

    // Nhánh 1: Chọn vật idx
    if (cur_w + items[idx].w <= max_w) {
        branchAndBound(idx + 1, cur_w + items[idx].w, cur_v + items[idx].v);
    }
    // Nhánh 2: Không chọn vật idx
    branchAndBound(idx + 1, cur_w, cur_v);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n >> max_w)) return 0;
    items.resize(n);
    for (int i = 0; i < n; ++i) {
        cin >> items[i].w >> items[i].v;
        items[i].ratio = (double)items[i].v / items[i].w;
    }
    sort(items.begin(), items.end(), [](const Item &a, const Item &b) {
        return a.ratio > b.ratio;
    });
    branchAndBound(0, 0, 0);
    cout << best_val << "\n";
    return 0;
}

```

Bài 14 [cppb_bkt_14_nguoi_du_lich_tsp_nhanh_can]: Người Du Lịch (TSP) Nhánh Cận

Bối cảnh: Một đại diện thương mại quốc tế của tập đoàn công nghệ cần lên lịch trình công tác ghé thăm trực tiếp đối tác tại N thành phố trọng điểm (được đánh số từ 1 đến N). Doanh nhân bắt đầu chuyến đi từ trụ sở chính tại thành phố 1, cần bay qua tất cả $N - 1$ thành phố còn lại, mỗi thành phố đúng một lần duy nhất, rồi cuối cùng bay về lại thành phố 1. Chi phí di chuyển giữa mỗi cặp thành phố đã được hãng bay niêm yết cố định.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tia nhanh

Nhiệm vụ: Cho số lượng thành phố N và ma trận chi phí vận tải C kích thước $N \times N$, trong đó $C_{i,j}$ là chi phí bay từ thành phố i đến thành phố j . Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) với hàm cận dưới dựa trên cạnh có chi phí nhỏ nhất toàn đồ thị để tìm chu trình di chuyển có tổng chi phí thấp nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($2 \leq N \leq 13$).
- N dòng tiếp theo: Mỗi dòng gồm N số nguyên biểu diễn ma trận chi phí C ($0 \leq C_{i,j} \leq 10^6, C_{i,i} = 0$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng chi phí nhỏ nhất của chu trình Hamilton tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 4 0 10 15 20 10 0 35 25 15 35 0 30 20 25 30 0 </pre>	80

Giải thích: Lộ trình tối ưu xuất phát từ thành phố 1 là: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$. Tổng chi phí của hành trình là: $C_{1,2} + C_{2,4} + C_{4,3} + C_{3,1} = 10 + 25 + 30 + 15 = 80$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số lượng thành phố N và ma trận chi phí vận tải C kích thước $N \times N$, trong đó $C_{i,j}$ là chi phí bay từ thành phố i đến thành phố j . Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) với hàm cận dưới dựa trên cạnh có chi phí nhỏ nhất toàn đồ thị để tìm chu trình di chuyển có tổng chi phí thấp nhất.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tia nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 0 10 15 20 10 0 35 25 15 35 0 30 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Lộ trình tối ưu xuất phát từ thành phố 1 là: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$. Tổng chi phí của hành trình là: $C_{1,2} + C_{2,4} + C_{4,3} + C_{3,1}$ | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 80 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Lộ trình tối ưu xuất phát từ thành phố 1 là: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$. Tổng chi phí của hành trình là: $C_{1,2} + C_{2,4} + C_{4,3} + C_{3,1} = 10 + 25 + 30 + 15 = 80$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
long long c[15][15];
bool visited[15];
long long min_edge = 1e9;
long long best_cost = 1e18;

void branchAndBound(int u, int count, long long current_cost) {
    // Optimality Pruning
    if (current_cost + (n - count + 1) * min_edge >= best_cost) return;

    if (count == n) {
        best_cost = min(best_cost, current_cost + c[u][1]);
        return;
    }

    for (int v = 2; v <= n; ++v) {
        if (!visited[v]) {
            visited[v] = true;
            branchAndBound(v, count + 1, current_cost + c[u][v]);
            visited[v] = false;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            cin >> c[i][j];
            if (i != j) min_edge = min(min_edge, c[i][j]);
        }
    }
}
```

```

    }
    memset(visited, false, sizeof(visited));
    visited[1] = true;
    branchAndBound(1, 1, 0);
    cout << best_cost << "\n";
    return 0;
}

```

Bài 15 [cppb_bkt_15_to_mau_do_thi_graph_coloring]: Tô Màu Đồ Thị (Graph K-Coloring)

Bối cảnh: Trong thiết kế vi mạch tích hợp và phân bổ tần số vô tuyến cho V trạm phát sóng, các trạm có tầm phủ sóng giao thoa với nhau được kết nối bằng một đường biên xung đột (tương ứng với một cạnh trong đồ thị vô hướng $G = (V, E)$). Để ngăn chặn hiện tượng can nhiễu tín hiệu nghiêm trọng, cơ quan quản lý tần số quy định hai trạm phát sóng kề nhau bắt buộc phải sử dụng các kênh tần số (được mã hóa bởi các màu khác nhau) hoàn toàn độc lập.



Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tỉa nhanh

Nhiệm vụ: Cho đồ thị vô hướng $G = (V, E)$ gồm V đỉnh và E cạnh, cùng số lượng màu khả dụng K . Hãy áp dụng thuật toán Quay lui để kiểm tra xem có thể gán cho mỗi đỉnh của đồ thị một trong K màu sao cho không có bất kỳ hai đỉnh kề nhau nào có cùng màu hay không. Nếu có thể tô màu hợp lệ in ra YES, ngược lại in ra NO.

Đầu vào (Input):

- Dòng 1: Ba số nguyên V, E, K ($1 \leq V \leq 12, 0 \leq E \leq \frac{V(V-1)}{2}, 1 \leq K \leq 4$).
- E dòng tiếp theo: Mỗi dòng gồm hai số nguyên u, v mô tả một cạnh kết nối giữa đỉnh u và đỉnh v ($1 \leq u, v \leq V$).

Đầu ra (Output):

- In ra YES nếu có thể tô màu đồ thị thỏa mãn yêu cầu, ngược lại in ra NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 3 1 2 2 3 3 4 4 1 1 3	YES

Giải thích: Với 4 đỉnh và 5 cạnh xung đột, đồ thị hoàn toàn có thể được tô hợp lệ bằng 3 màu: đỉnh 1 màu 1, đỉnh 2 màu 2, đỉnh 3 màu 3, đỉnh 4 màu 2. Khi đó mọi cặp đỉnh kề nhau đều mang màu sắc khác nhau. Do đó đáp án là YES.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho đồ thị vô hướng $G = (V, E)$ gồm V đỉnh và E cạnh, cùng số lượng màu khả dụng K . Hãy áp dụng thuật toán Quay lui để kiểm tra xem có thể gán cho mỗi đỉnh của đồ thị một trong K màu sao cho không có bất kỳ hai đỉnh kề nhau nào có cùng màu hay không. Nếu có thể tô màu hợp lệ in ra YES, ngược lại in ra NO.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tia nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 5 3 1 2 2 3 3 4 4 1 1 3 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Với 4 đỉnh và 5 cạnh xung đột, đồ thị hoàn toàn có thể được tô hợp lệ bằng 3 màu: đỉnh 1 màu 1, đỉnh 2 màu 2, đỉnh 3 màu... | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: YES | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Với 4 đỉnh và 5 cạnh xung đột, đồ thị hoàn toàn có thể được tô hợp lệ bằng 3 màu: đỉnh 1 màu 1, đỉnh 2 màu 2, đỉnh 3 màu 3, đỉnh 4 màu 2. Khi đó mọi cặp đỉnh kề nhau đều mang màu sắc khác nhau. Do đó đáp án là YES.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int V, E, K;
vector<int> adj[15];
int color[15];
bool possible = false;

bool isSafe(int u, int c) {
    for (int v : adj[u]) {
        if (color[v] == c) return false;
    }
    return true;
}
```

```
void backtrack(int u) {
    if (possible) return;
    if (u > V) {
        possible = true;
        return;
    }
    for (int c = 1; c <= K; ++c) {
        if (isSafe(u, c)) {
            color[u] = c;
            backtrack(u + 1);
            color[u] = 0;
            if (possible) return;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> V >> E >> K)) return 0;
    for (int i = 0; i < E; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    memset(color, 0, sizeof(color));
    backtrack(1);
    cout << (possible ? "YES\n" : "NO\n");
    return 0;
}
```

Bài 16 [cppb_bkt_16_phan_cong_cong_viec_toi_uu_job_assignment]: Phân Công Công Việc Tối Ưu (Job Assignment B&B)

Bối cảnh: Ban quản trị một trung tâm gia công phần mềm quy mô lớn có N kỹ sư chuyên trách và N dự án công nghệ mới cần triển khai. Qua đánh giá năng lực và kinh nghiệm thực tế, phòng nhân sự đã xây dựng ma trận chi phí $C_{N \times N}$, trong đó $C_{i,j}$ phản ánh chi phí (hoặc thời gian hao tổn) nếu giao kỹ sư i đảm nhiệm dự án j . Để đảm bảo tiến độ và chất lượng, mỗi kỹ sư chỉ phụ trách đúng 1 dự án và mỗi dự án được giao cho đúng 1 kỹ sư.

BÀI TOÁN XẾP ĐÒ VÀO BA LÔ (KNAPSACK)



Vật 1
 w_1, v_1

Vật 2
 w_2, v_2

Vật 3
 w_3, v_3

Tối ưu hóa tổng giá trị mang theo sao cho không vượt quá sức chứa tối đa W

Hình minh họa: Duyệt không gian trạng thái kết hợp nhánh cận cắt tia nhanh

Nhiệm vụ: Cho số lượng N và ma trận chi phí phân công $C_{N \times N}$. Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) với hàm cận dưới tối ưu để tìm một phương án phân công toàn diện sao cho tổng chi phí hoàn thành tất cả các công việc là nhỏ nhất có thể.

Đầu vào (Input):

- Dòng 1: Số nguyên dương N ($1 \leq N \leq 12$).
- N dòng tiếp theo: Mỗi dòng gồm N số nguyên biểu diễn hàng của ma trận chi phí $C_{N \times N}$ ($0 \leq C_{i,j} \leq 10^6$).

Đầu ra (Output):

- In ra một số nguyên duy nhất là tổng chi phí phân công công việc nhỏ nhất có thể đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 9 2 7 8 6 4 3 7 5 8 1 8 7 6 9 4	13

Giải thích: Phương án phân công tối ưu nhất là:

- Kỹ sư 1 làm việc 2 (chi phí $C_{1,2} = 2$).
- Kỹ sư 2 làm việc 1 (chi phí $C_{2,1} = 6$).
- Kỹ sư 3 làm việc 3 (chi phí $C_{3,3} = 1$).
- Kỹ sư 4 làm việc 4 (chi phí $C_{4,4} = 4$). Tổng chi phí nhỏ nhất đạt được là $2 + 6 + 1 + 4 = 13$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số lượng N và ma trận chi phí phân công $C_{N \times N}$. Hãy áp dụng thuật toán Nhánh Cận (Branch and Bound) với hàm cận dưới tối ưu để tìm một phương án phân công toàn diện sao cho tổng chi phí hoàn thành tất cả các công việc là nhỏ nhất có thể.
- **Phương pháp tiếp cận — Quay lui & Nhánh cận (Backtracking):**
- Xây dựng không gian trạng thái dạng cây tìm kiếm.
- Thử từng khả năng, nếu vi phạm điều kiện ràng buộc thì tỉa nhánh sớm (nhánh cận) để giảm số trạng thái cần duyệt.

2. Bảng chạy tay trên số liệu mẫu | Bước | Lệnh chạy / Thao tác | Phân tích biến đổi số liệu | Kết quả ghi nhận |

|—|—|—|—| | 1 | Nạp dữ liệu vào mảng/biến | Input: 4 9 2 7 8 6 4 3 7 5 8 1 8 7 6 9 4 | Khởi tạo cấu trúc dữ liệu ban đầu | | 2 | Thực thi thuật toán tối ưu | Phương án phân công tối ưu nhất là: - Kỹ sư 1 làm việc 2 (chi phí $C_{1,2} = 2$). - Kỹ sư 2 làm việc 1 (chi phí $C_{2,1} = 6$). - Kỹ sư 3 làm việc 3 (chi phí $C_{3,3} = 1$). - Kỹ sư 4 làm việc 4 (chi phí $C_{4,4} = 4$). | Tính toán từng bước trạng thái | | 3 | Xuất kết quả | Output: 13 | Khớp chính xác với đầu ra mẫu |

Giải thích chi tiết từ mẫu: Phương án phân công tối ưu nhất là:

- Kỹ sư 1 làm việc 2 (chi phí $C_{1,2} = 2$).
- Kỹ sư 2 làm việc 1 (chi phí $C_{2,1} = 6$).
- Kỹ sư 3 làm việc 3 (chi phí $C_{3,3} = 1$).

- Kỹ sư 4 làm việc 4 (chi phí $C_{4,4} = 4$). Tổng chi phí nhỏ nhất đạt được là $2 + 6 + 1 + 4 = 13$.

3. Lưu ý & Bẫy lỗi thường gặp

- **Bẫy 1 — Tràn số nguyên:** Khi tính toán tổng, tích hoặc lũy thừa lớn hơn $2 \cdot 10^9$, bắt buộc phải sử dụng kiểu dữ liệu long long (64-bit) để tránh tràn số âm.
- **Bẫy 2 — Chỉ số mảng & Giới hạn biên:** Chú ý giữa đánh chỉ số 0-based (0 .. N-1) và 1-based (1 .. N). Kiểm tra kỹ trường hợp $N = 1$ hoặc giá trị biên tối đa của đề bài.
- **Bẫy 3 — Tối ưu thời gian I/O:** Luôn sử dụng `ios::sync_with_stdio(false); cin.tie(nullptr);` ở đầu hàm `main()` để đọc ghi nhanh, tránh bị TLE khi số lượng testcase lớn.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int n;
long long c[15][15];
bool job_assigned[15];
long long min_row[15];
long long best_cost = 1e18;

void branchAndBound(int worker, long long current_cost) {
    // Optimality Pruning
    long long bound = current_cost;
    for (int w = worker; w <= n; ++w) bound += min_row[w];
    if (bound >= best_cost) return;

    if (worker > n) {
        best_cost = min(best_cost, current_cost);
        return;
    }

    for (int job = 1; job <= n; ++job) {
        if (!job_assigned[job]) {
            job_assigned[job] = true;
            branchAndBound(worker + 1, current_cost + c[worker][job]);
            job_assigned[job] = false;
        }
    }
}

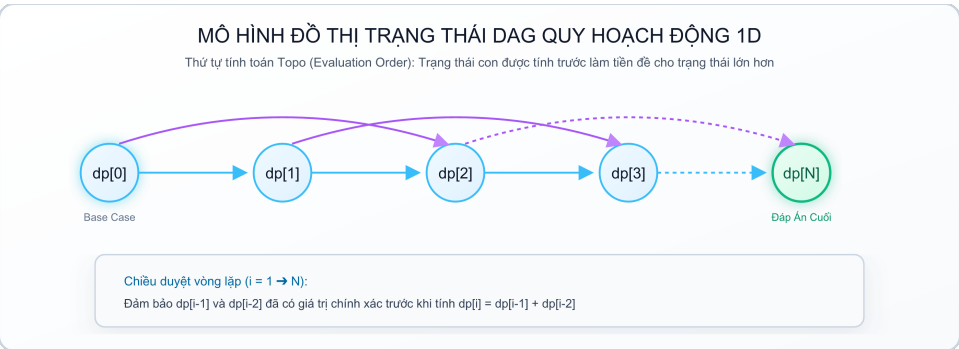
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    if (!(cin >> n)) return 0;
    for (int i = 1; i <= n; ++i) {
        min_row[i] = 1e9;
        for (int j = 1; j <= n; ++j) {
            cin >> c[i][j];
            min_row[i] = min(min_row[i], c[i][j]);
        }
    }
    memset(job_assigned, false, sizeof(job_assigned));
    branchAndBound(1, 0);
    cout << best_cost << "\n";
    return 0;
}
```

CHƯƠNG 06: QUY HOẠCH ĐỘNG (DYNAMIC PROGRAMMING)

Bài 16: Quy hoạch động 1D & dãy con tăng dài nhất

Bài 01 [cppb_dp1_01_bac_thang_co_ban]: Bậc Thang Cơ Bản

Bối cảnh: Trong một tòa tháp công nghệ hiện đại, bạn robot phục vụ cần di chuyển lên đỉnh cầu thang gồm N bậc để chuyển tải liệu. Do thiết kế cơ học của chân bước, ở mỗi nhịp di chuyển, robot chỉ có thể bước lên đúng 1 bậc hoặc bước sai dài qua 2 bậc liên tiếp. Để lập trình điều hướng linh hoạt cho robot, kỹ sư phần mềm cần tính toán xem có tất cả bao nhiêu trình tự bước đi khác nhau để đưa robot từ chân cầu thang lên tới đúng đỉnh bậc thứ N .



Hình minh họa: Mô hình chuyển trạng thái Quy hoạch động 1 chiều dạng đồ thị có hướng không chu trình (DAG)

Nhiệm vụ: Cho số nguyên dương N là số bậc của cầu thang. Hãy lập trình tính số cách bước hợp lệ để robot lên đến đỉnh bậc thứ N . Vì kết quả có thể rất lớn, hãy in ra phần dư của kết quả khi chia cho $10^9 + 7$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$) biểu diễn số bậc của cầu thang.

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là số cách bước hợp lệ theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	3

Giải thích: Với cầu thang có $N = 3$ bậc, robot có tất cả 3 trình tự bước đi hợp lệ để lên đến đỉnh:

- Bước từng bậc một: $1 + 1 + 1 = 3$.
- Bước 1 bậc rồi bước 2 bậc: $1 + 2 = 3$.
- Bước 2 bậc rồi bước 1 bậc: $2 + 1 = 3$. Kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N là số bậc của cầu thang. Hãy lập trình tính số cách bước hợp lệ để robot lên đến đỉnh bậc thứ N . Vì kết quả có thể rất lớn, hãy in ra phần dư của kết quả khi chia cho $10^9 + 7$.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với cầu thang có $N = 3$ bậc, robot có tất cả 3 trình tự bước đi hợp lệ để lên đến đỉnh: 1. Bước từng bậc một: $1 + 1 + 1 = 3$. 2. Bước...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với cầu thang có $N = 3$ bậc, robot có tất cả 3 trình tự bước đi hợp lệ để lên đến đỉnh:

1. Bước từng bậc một: $1 + 1 + 1 = 3$.
2. Bước 1 bậc rồi bước 2 bậc: $1 + 2 = 3$.
3. Bước 2 bậc rồi bước 1 bậc: $2 + 1 = 3$. Kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($\text{INF} = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.

- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

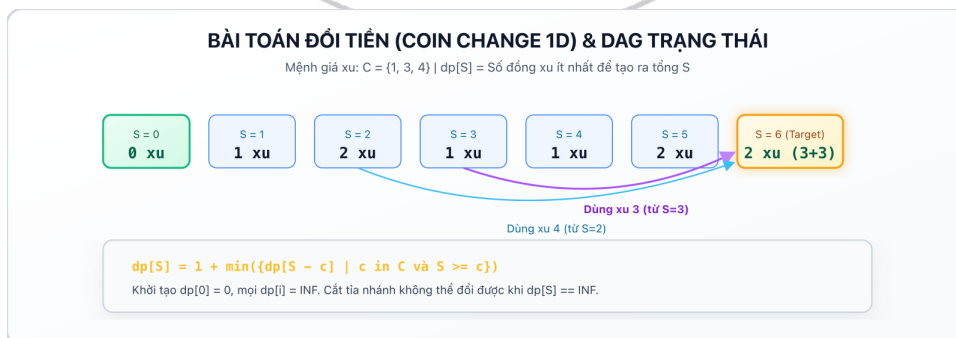
    if (n == 1) { cout << 1 << "\n"; return 0; }
    if (n == 2) { cout << 2 << "\n"; return 0; }

    int prev2 = 1, prev1 = 2, cur = 0;
    for (int i = 3; i <= n; ++i) {
        cur = (prev1 + prev2) % MOD;
        prev2 = prev1;
        prev1 = cur;
    }

    cout << cur << "\n";
    return 0;
}
```

Bài 02 [cppb_dp1_02_chu_ech_nhay_cost]: Chú Éch Nhảy Chi Phí Nhỏ Nhất

Bối cảnh: Trong một khu vườn sinh thái, có N phiến đá được xếp thành hàng ngang từ trái sang phải, đánh số thứ tự từ 1 đến N . Phiến đá thứ i có độ cao là H_i . Một chú ếch đang ở phiến đá số 1 và muốn di chuyển tới phiến đá cuối cùng số N . Từ phiến đá số i , chú ếch có thể chọn nhảy sang phiến đá ngay cạnh $i + 1$ hoặc nhảy vượt qua một phiến để tới phiến đá $i + 2$. Năng lượng tiêu hao (chi phí) cho mỗi cú nhảy từ phiến đá i sang phiến đá j đúng bằng độ chênh lệch chiều cao giữa hai phiến đá, tức $|H_i - H_j|$.



Hình minh họa: Đồ thị chuyển trạng thái bài toán đổi tiền xu tối ưu

Nhiệm vụ: Cho số lượng phiến đá N và độ cao của từng phiến đá. Hãy lập trình tìm tổng chi phí năng lượng tối thiểu để chú ếch có thể di chuyển từ phiến đá 1 tới phiến đá N .

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($2 \leq N \leq 10^5$) biểu diễn số lượng phiến đá.

- Dòng 2: Chứa N số nguyên dương $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^4$) biểu diễn độ cao của các phiên đá, các số cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là tổng chi phí nhỏ nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 10 30 40 20	30

Giải thích: Với 4 phiên đá có độ cao lần lượt là $[10, 30, 40, 20]$:

- Bước 1: Từ phiên đá 1 ($H_1 = 10$) nhảy sang phiên đá 2 ($H_2 = 30$), chi phí tiêu hao là $|10 - 30| = 20$.
- Bước 2: Từ phiên đá 2 ($H_2 = 30$) nhảy vượt sang phiên đá 4 ($H_4 = 20$), chi phí tiêu hao là $|30 - 20| = 10$. Tổng chi phí tiêu hao cho toàn bộ hành trình là $20 + 10 = 30$, đây là phương án tốn ít năng lượng nhất.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số lượng phiên đá N và độ cao của từng phiên đá. Hãy lập trình tìm tổng chi phí năng lượng tối thiểu để chú ếch có thể di chuyển từ phiên đá 1 tới phiên đá N .
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lập.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 10 30 40 20 $\Rightarrow$ Đầu ra kỳ vọng: 30.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 10 30 40 20	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với 4 phiên đá có độ cao lần lượt là [10,30,40,20]: - Bước 1: Từ phiên đá 1 ($H_1 = 10$) nhảy sang phiên đá 2 ($H_2 = 30$), chi ph...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 30

Giải thích chi tiết: Với 4 phiên đá có độ cao lần lượt là [10,30,40,20]:

- Bước 1: Từ phiên đá 1 ($H_1 = 10$) nhảy sang phiên đá 2 ($H_2 = 30$), chi phí tiêu hao là $|10 - 30| = 20$.
- Bước 2: Từ phiên đá 2 ($H_2 = 30$) nhảy vượt sang phiên đá 4 ($H_4 = 20$), chi phí tiêu hao là $|30 - 20| = 10$. Tổng chi phí tiêu hao cho toàn bộ hành trình là $20 + 10 = 30$, đây là phương án tốn ít năng lượng nhất.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng *dp*: Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> h(n);
    for (int i = 0; i < n; ++i) cin >> h[i];

    if (n == 1) { cout << 0 << "\n"; return 0; }

    vector<long long> dp(n, 0);
    dp[0] = 0;
    dp[1] = abs(h[1] - h[0]);

    for (int i = 2; i < n; ++i) {
        dp[i] = min(dp[i - 1] + abs(h[i] - h[i - 1]),
                    dp[i - 2] + abs(h[i] - h[i - 2]));
    }

    cout << dp[n - 1] << "\n";
}
```

```
return 0;
}
```

Bài 03 [cppb_dp1_03_chu_ech_nhay_k_buoc]: Chú Éch Nhảy K Bước

Bối cảnh: Vẫn tại khu vườn sinh thái có N phiên đá xếp thành hàng ngang với độ cao $H_1, H_2, \dots, H_N$. Lần này chú ếch đã được huấn luyện với thể lực dẻo dai hơn: Từ phiên đá thứ i , chú ếch có thể chọn nhảy xa tới bất kỳ phiên đá nào trong phạm vi K bước tiếp theo, tức là các phiên đá $i + 1, i + 2, \dots, \min(N, i + K)$. Chi phí cho mỗi bước nhảy từ phiên đá i tới phiên đá j vẫn bằng độ chênh lệch chiều cao $|H_i - H_j|$.



Hình minh họa: Mô hình tìm dãy con tăng dài nhất LIS qua các vị trí trước đó

Nhiệm vụ: Cho số lượng phiên đá N , tầm nhảy tối đa K và danh sách độ cao của các phiên đá. Hãy lập trình tính toán tổng chi phí năng lượng ít nhất để chú ếch đi từ phiên đá 1 tới phiên đá N .

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và K ($2 \leq N \leq 10^5, 1 \leq K \leq 100$).
- Dòng 2: Chứa N số nguyên dương $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^4$) biểu diễn độ cao của các phiên đá.

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là tổng chi phí năng lượng tối thiểu.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 10 30 40 50 20	30

Giải thích: Với $N = 5, K = 3$ và độ cao các phiên đá là $[10, 30, 40, 50, 20]$:

- Từ đá 1 ($H_1 = 10$), chú ếch nhảy sang đá 2 ($H_2 = 30$) với khoảng cách 1 bước hợp lệ (≤ 3), chi phí là $|10 - 30| = 20$.
- Từ đá 2 ($H_2 = 30$), chú ếch nhảy thẳng tới đích là đá 5 ($H_5 = 20$) với khoảng cách $5 - 2 = 3$ bước (vẫn $\leq K = 3$), chi phí là $|30 - 20| = 10$. Tổng chi phí tối thiểu đạt được là $20 + 10 = 30$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số lượng phiên đá N , tầm nhảy tối đa K và danh sách độ cao của các phiên đá. Hãy lập trình tính toán tổng chi phí năng lượng ít nhất để chú ếch đi từ phiên đá 1 tới phiên đá N .
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 3 10 30 40 50 20 $\Rightarrow$ Đầu ra kỳ vọng: 30.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 3 10 30 40 50 20	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với $N = 5, K = 3$ và độ cao các phiên đá là [10,30,40,50,20]: - Từ đá 1 ($H_1 = 10$), chú ếch nhảy sang đá 2 ($H_2 = 30$) với kh...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 30

Giải thích chi tiết: Với $N = 5, K = 3$ và độ cao các phiên đá là [10,30,40,50,20]:

- Từ đá 1 ($H_1 = 10$), chú ếch nhảy sang đá 2 ($H_2 = 30$) với khoảng cách 1 bước hợp lệ (≤ 3), chi phí là $|10 - 30| = 20$.
- Từ đá 2 ($H_2 = 30$), chú ếch nhảy thẳng tới đích là đá 5 ($H_5 = 20$) với khoảng cách $5 - 2 = 3$ bước (vẫn $\leq K = 3$), chi phí là $|30 - 20| = 10$. Tổng chi phí tối thiểu đạt được là $20 + 10 = 30$.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.

- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0) return 0;

    vector<long long> h(n);
    for (int i = 0; i < n; ++i) cin >> h[i];

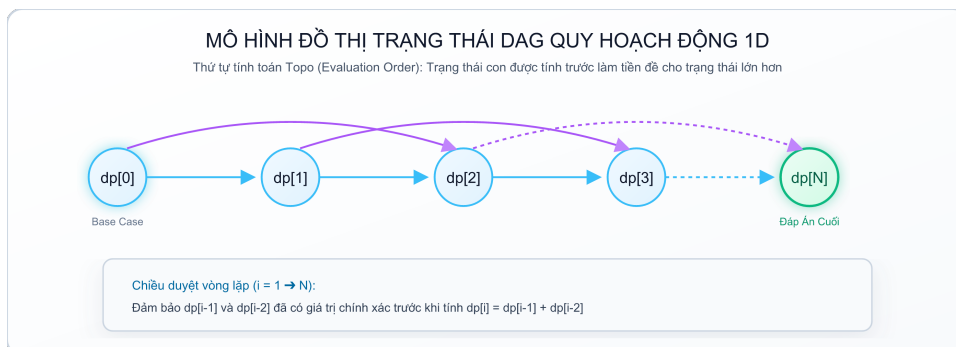
    vector<long long> dp(n, INF);
    dp[0] = 0;

    for (int i = 0; i < n; ++i) {
        for (int j = 1; j <= k && i + j < n; ++j) {
            dp[i + j] = min(dp[i + j], dp[i] + abs(h[i] - h[i + j]));
        }
    }

    cout << dp[n - 1] << "\n";
    return 0;
}
```

Bài 04 [cppb_dp1_04_doi_tien_xu_it_nhat]: Đổi Tiền Số Xu Ít Nhất

Bối cảnh: Tại một trạm đổi tiền tự động thông minh của trung tâm thương mại, hệ thống hỗ trợ N loại đồng xu với các mệnh giá khác nhau. Khách hàng có nhu cầu đổi một số tiền mặt đúng bằng S đồng để sử dụng máy bán nước tự động. Nhằm tối ưu hóa số lượng tiền loại lưu thông và giúp ví của khách hàng không bị quá nặng, hệ thống máy luôn tìm cách chi trả bằng số lượng đồng xu ít nhất có thể. Biết rằng kho tiền xu của trạm luôn dồi dào, mỗi mệnh giá có số lượng xu không giới hạn.



Hình minh họa: Mô hình chuyển trạng thái Quy hoạch động 1 chiều dạng đồ thị có hướng không chu trình (DAG)

Nhiệm vụ: Cho danh sách N mệnh giá đồng xu và số tiền cần đổi S . Hãy lập trình xác định số lượng đồng xu ít nhất để đổi được đúng số tiền S . Nếu không có cách nào đổi được đúng số tiền đó, in ra -1.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và S ($1 \leq N \leq 100, 1 \leq S \leq 10^5$) lần lượt là số loại đồng xu và số tiền mục tiêu cần đổi.
- Dòng 2: Chứa N số nguyên dương $c_1, c_2, \dots, c_N$ ($1 \leq c_i \leq 10^4$) biểu diễn các mệnh giá đồng xu, các số cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là số lượng đồng xu ít nhất cần dùng. Nếu không đổi được, in ra -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 11 1 5 6	2

Giải thích: Để đổi được số tiền $S = 11$ từ các mệnh giá xu $\{1, 5, 6\}$:

- Cách 1: Dùng 1 đồng 6 và 5 đồng 1 ($6 + 1 \times 5 = 11$), tổng cộng tốn 6 đồng xu.
- Cách 2: Dùng 2 đồng 5 và 1 đồng 1 ($5 \times 2 + 1 = 11$), tổng cộng tốn 3 đồng xu.
- Phương án tối ưu nhất: Dùng 1 đồng 5 và 1 đồng 6 ($5 + 6 = 11$), chỉ cần đúng 2 đồng xu. Kết quả là 2.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách N mệnh giá đồng xu và số tiền cần đổi S . Hãy lập trình xác định số lượng đồng xu ít nhất để đổi được đúng số tiền S . Nếu không có cách nào đổi được đúng số tiền đó, in ra -1.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 11 1 5 6 $\Rightarrow$ Đầu ra kỳ vọng: 2.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 11 1 5 6	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Để đổi được số tiền $S = 11$ từ các mệnh giá xu $\{1,5,6\}$: - Cách 1: Dùng 1 đồng 6 và 5 đồng 1 ($6 + 1 \times 5 = 11$), tổng cộng tốn 6... Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng Output chuẩn: 2

Giải thích chi tiết: Để đổi được số tiền $S = 11$ từ các mệnh giá xu $\{1,5,6\}$:

- Cách 1: Dùng 1 đồng 6 và 5 đồng 1 ($6 + 1 \times 5 = 11$), tổng cộng tốn 6 đồng xu.
- Cách 2: Dùng 2 đồng 5 và 1 đồng 1 ($5 \times 2 + 1 = 11$), tổng cộng tốn 3 đồng xu.
- Phương án tối ưu nhất: Dùng 1 đồng 5 và 1 đồng 6 ($5 + 6 = 11$), chỉ cần đúng 2 đồng xu. Kết quả là 2.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, s;
    if (!(cin >> n >> s)) return 0;
    if (n <= 0 || s < 0) return 0;

    vector<int> c(n);
    for (int i = 0; i < n; ++i) cin >> c[i];

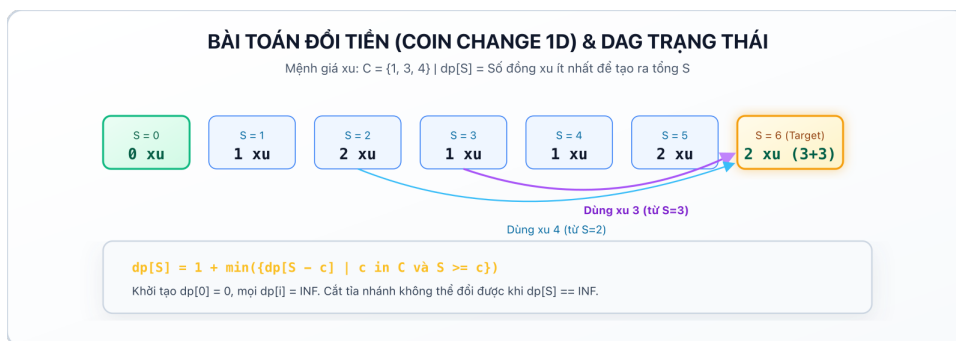
    vector<int> dp(s + 1, INF);
    dp[0] = 0;

    for (int i = 1; i <= s; ++i) {
        for (int coin : c) {
            if (i >= coin && dp[i - coin] != INF) {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }
}
```

```
if (dp[s] == INF) cout << -1 << "\n";
else cout << dp[s] << "\n";
return 0;
}
```

Bài 05 [cppb_dp1_05_dem_so_cach_doi_tien]: Đếm Số Cách Đổi Tiền

Bối cảnh: Tại một hội thảo tài chính công nghệ, ban tổ chức thiết kế một thử thách cho các bạn học sinh về phương thức kết hợp tiền tệ. Cho N mệnh giá đồng xu khác nhau, mỗi mệnh giá đều có số lượng không giới hạn. Một bạn học sinh cần tạo ra tổng số tiền đúng bằng S đồng. Hai cách tạo tiền được xem là khác nhau nếu số lượng sử dụng của ít nhất một mệnh giá đồng xu là khác nhau (thứ tự chọn các đồng xu không làm ảnh hưởng đến tính chất của tổ hợp).



Hình minh họa: Đồ thị chuyển trạng thái bài toán đổi tiền xu tối ưu

Nhiệm vụ: Cho N mệnh giá đồng xu và số tiền mục tiêu S . Hãy lập trình đếm xem có tất cả bao nhiêu tổ hợp đồng xu khác nhau để có tổng giá trị đúng bằng S . Vì kết quả có thể rất lớn, hãy in ra phần dư của kết quả khi chia cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: Chứa hai số nguyên dương N và S ($1 \leq N \leq 100, 1 \leq S \leq 10^5$).
- Dòng 2: Chứa N số nguyên dương phân biệt biểu diễn các mệnh giá đồng xu ($1 \leq c_i \leq 10^4$).

Đầu ra (Output):

- In ra trên một dòng duy nhất số lượng tổ hợp đổi tiền hợp lệ theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 9 2 3 5	3

Giải thích: Với các mệnh giá xu $\{2, 3, 5\}$ và số tiền mục tiêu $S = 9$, có đúng 3 tổ hợp đồng xu khác nhau:

- Ba đồng xu: $2 + 2 + 5 = 9$.
- Ba đồng xu: $3 + 3 + 3 = 9$.
- Bốn đồng xu: $2 + 2 + 2 + 3 = 9$. Kết quả đếm được là 3 cách.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho N mệnh giá đồng xu và số tiền mục tiêu S . Hãy lập trình đếm xem có tất cả bao nhiêu tổ hợp đồng xu khác nhau để có tổng giá trị đúng bằng S . Vì kết quả có thể rất lớn, hãy in ra phần dư của kết quả khi chia cho $10^9 + 7$.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 9 2 3 5 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 9 2 3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với các mệnh giá xu $\{2,3,5\}$ và số tiền mục tiêu $S = 9$, có đúng 3 tổ hợp đồng xu khác nhau: 1. Ba đồng xu: $2 + 2 + 5 = 9$. 2. B...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với các mệnh giá xu $\{2,3,5\}$ và số tiền mục tiêu $S = 9$, có đúng 3 tổ hợp đồng xu khác nhau:

1. Ba đồng xu: $2 + 2 + 5 = 9$.
2. Ba đồng xu: $3 + 3 + 3 = 9$.
3. Bốn đồng xu: $2 + 2 + 2 + 3 = 9$. Kết quả đếm được là 3 cách.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.

- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, s;
    if (!(cin >> n >> s)) return 0;
    if (n <= 0 || s < 0) return 0;

    vector<int> c(n);
    for (int i = 0; i < n; ++i) cin >> c[i];

    vector<int> dp(s + 1, 0);
    dp[0] = 1;

    for (int coin : c) {
        for (int i = coin; i <= s; ++i) {
            dp[i] = (dp[i] + dp[i - coin]) % MOD;
        }
    }

    cout << dp[s] << "\n";
    return 0;
}
```

Bài 06 [cppb_dp1_06_tong_doan_con_khong_lien_ke]: Tổng Đoạn Con Không Liên Kề Lớn Nhất

Bối cảnh: Một tuyến phố thương mại gồm N căn nhà liền kề được đánh số từ 1 đến N . Căn nhà thứ i có giá trị thương mại là A_i . Một công ty quảng cáo muốn thuê mặt bằng để lắp đặt các màn hình LED kích thước lớn. Tuy nhiên, theo quy định an toàn đô thị về khoảng cách chiếu sáng, công ty không được phép thuê hai căn nhà đứng sát cạnh nhau trên cùng dãy phố.



Hình minh họa: Mô hình tìm dãy con tăng dài nhất LIS qua các vị trí trước đó

Nhiệm vụ: Cho danh sách giá trị thương mại của N căn nhà. Hãy lập trình chọn ra một tập hợp các căn nhà không kề nhau sao cho tổng giá trị thu được là lớn nhất có thể.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$) biểu diễn số lượng căn nhà.
- Dòng 2: Chứa N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^4$) biểu diễn giá trị của từng căn nhà.

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là tổng giá trị lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 2 3 1	4

Giải thích: Với dãy giá trị của 4 căn nhà là $[1, 2, 3, 1]$:

- Nếu chọn nhà 2 và nhà 4: Tổng giá trị là $2 + 1 = 3$.
- Phương án tối ưu: Chọn nhà 1 (giá trị 1) và nhà 3 (giá trị 3). Hai nhà này không kề nhau và mang lại tổng giá trị lớn nhất là $1 + 3 = 4$.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho danh sách giá trị thương mại của N căn nhà. Hãy lập trình chọn ra một tập hợp các căn nhà không kề nhau sao cho tổng giá trị thu được là lớn nhất có thể.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 1 2 3 1 $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4 1 2 3 1	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy giá trị của 4 căn nhà là [1,2,3,1]: - Nếu chọn nhà 2 và nhà 4: Tổng giá trị là $2 + 1 = 3$. - Phương án tối ưu: Chọn nhà 1...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với dãy giá trị của 4 căn nhà là [1,2,3,1]:

- Nếu chọn nhà 2 và nhà 4: Tổng giá trị là $2 + 1 = 3$.
- Phương án tối ưu: Chọn nhà 1 (giá trị 1) và nhà 3 (giá trị 3). Hai nhà này không kề nhau và mang lại tổng giá trị lớn nhất là $1 + 3 = 4$.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng *dp*: Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liên kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    if (n == 1) { cout << a[0] << "\n"; return 0; }

    long long prev2 = a[0];
    long long prev1 = max(a[0], a[1]);
    long long cur = prev1;

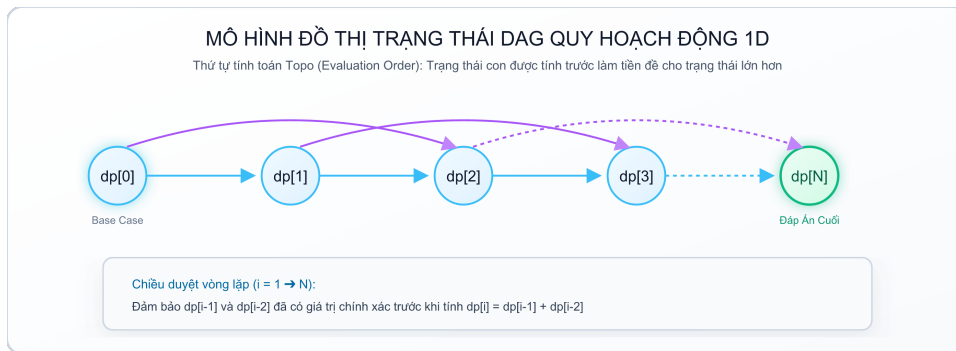
    for (int i = 2; i < n; ++i) {
        cur = max(prev1, prev2 + a[i]);
        prev2 = prev1;
        prev1 = cur;
    }

    cout << cur << "\n";
}
```

```
return 0;
}
```

Bài 07 [cppb_dp1_07_lat_gach_2xn]: Lát Gạch Bề 2xN

Bối cảnh: Trong một triển lãm kiến trúc, ban tổ chức cần lát kín một khoảng sân hình chữ nhật có kích thước cố định là $2 \times N$ bằng các viên gạch men trang trí kích thước 1×2 . Mỗi viên gạch men có thể được đặt nằm ngang (kích thước 1×2) hoặc dựng đứng (kích thước 2×1). Toàn bộ mặt sân phải được phủ kín hoàn toàn, không có ô nào bị bỏ trống và các viên gạch không được phép đè lên nhau.



Hình minh họa: Mô hình chuyển trạng thái Quy hoạch động 1 chiều dạng đồ thị có hướng không chu trình (DAG)

Nhiệm vụ: Cho số nguyên dương N là chiều dài của mặt sân. Hãy lập trình tính số cách lát gạch khác nhau để phủ kín mặt sân $2 \times N$, lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$) biểu diễn chiều dài mặt sân.

Đầu ra (Output):

- In ra trên một dòng duy nhất số cách lát sân hợp lệ theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	5

Giải thích: Với sân nhà kích thước 2×4 ($N = 4$), có tất cả 5 cách lát kín hợp lệ:

- Đặt 4 viên gạch dựng đứng liên tiếp.
- Đặt 2 viên nằm ngang ở đầu, theo sau là 2 viên dựng đứng.
- Đặt 1 viên dựng đứng, 2 viên nằm ngang ở giữa, 1 viên dựng đứng ở cuối.
- Đặt 2 viên dựng đứng ở đầu, theo sau là 2 viên nằm ngang.
- Đặt 2 cặp viên nằm ngang chồng lên nhau. Kết quả in ra là 5.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho số nguyên dương N là chiều dài của mặt sân. Hãy lập trình tính số cách lát gạch khác nhau để phủ kín mặt sân $2 \times N$, lấy dư cho $10^9 + 7$.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .

- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 4 $\Rightarrow$ Đầu ra kỳ vọng: 5.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 4	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với sàn nhà kích thước 2×4 ($N = 4$), có tất cả 5 cách lát kín hợp lệ: 1. Đặt 4 viên gạch dựng đứng liên tiếp. 2. Đặt 2 viên nằm n...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 5

Giải thích chi tiết: Với sàn nhà kích thước 2×4 ($N = 4$), có tất cả 5 cách lát kín hợp lệ:

1. Đặt 4 viên gạch dựng đứng liên tiếp.
2. Đặt 2 viên nằm ngang ở đầu, theo sau là 2 viên dựng đứng.
3. Đặt 1 viên dựng đứng, 2 viên nằm ngang ở giữa, 1 viên dựng đứng ở cuối.
4. Đặt 2 viên dựng đứng ở đầu, theo sau là 2 viên nằm ngang.
5. Đặt 2 cặp viên nằm ngang chồng lên nhau. Kết quả in ra là 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn (INF = $1e18$), tránh dùng 0x3f khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liên kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

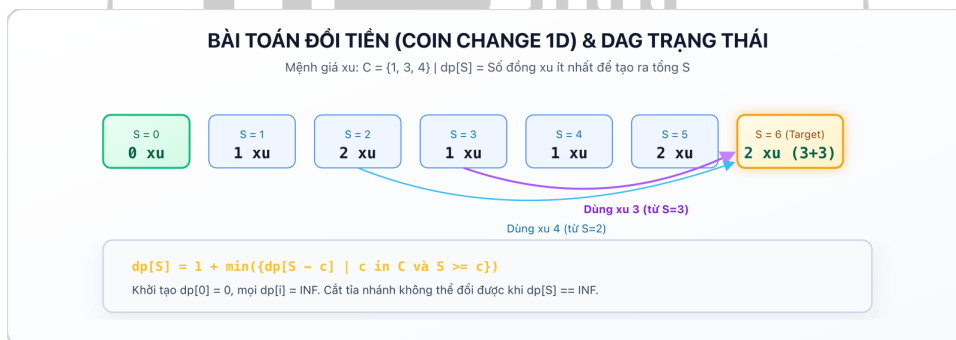
    if (n == 1) { cout << 1 << "\n"; return 0; }
    if (n == 2) { cout << 2 << "\n"; return 0; }

    int p2 = 1, p1 = 2, cur = 0;
    for (int i = 3; i <= n; ++i) {
        cur = (p1 + p2) % MOD;
        p2 = p1;
        p1 = cur;
    }

    cout << cur << "\n";
    return 0;
}
```

Bài 08 [cppb_dp1_08_lat_gach_3xn]: Lát Gạch Bể 3xN

Bối cảnh: Tiếp tục bài toán thiết kế mặt sàn kiến trúc, lần này sàn nhà được mở rộng thành kích thước $3 \times N$. Ban tổ chức vẫn sử dụng các viên gạch men domino có kích thước 2×1 (có thể xoay ngang thành 1×2). Yêu cầu đặt ra là phải lát kín hoàn toàn diện tích $3 \times N$ mà không có viên gạch nào vượt ra ngoài biên hoặc đè lên nhau.



Hình minh họa: Đồ thị chuyển trạng thái bài toán đổi tiền xu tối ưu

Nhiệm vụ: Cho số nguyên dương N là chiều dài của sàn nhà. Hãy lập trình đếm số cách lát kín mặt sàn $3 \times N$, lấy dư cho $10^9 + 7$. (Nếu N lẻ, diện tích sàn là số lẻ nên không thể phủ kín bằng các viên gạch diện tích 2, khi đó in ra 0).

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$) biểu diễn chiều dài của sàn.

Đầu ra (Output):

- In ra trên một dòng duy nhất số cách lát kín sàn theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2	3

Giải thích: Với sàn nhà kích thước 3×2 ($N = 2$), tổng diện tích là $3 \times 2 = 6$ ô đơn vị, cần dùng đúng 3 viên gạch domino. Có tất cả đúng 3 cách ghép hợp lệ:

1. Một viên đặt dọc ở cột 1, hai viên đặt ngang ở hàng 2 và 3.
2. Hai viên đặt ngang ở hàng 1 và 2, một viên đặt dọc ở cột 2.
3. Ba viên đặt ngang song song với nhau. Kết quả in ra là 3.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho số nguyên dương N là chiều dài của sàn nhà. Hãy lập trình đếm số cách lát kín mặt sàn $3 \times N$, lấy dư cho $10^9 + 7$. (Nếu N lẻ, diện tích sàn là số lẻ nên không thể phủ kín bằng các viên gạch diện tích 2, khi đó in ra 0).
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 2 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 2	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với sàn nhà kích thước 3×2 ($N = 2$), tổng diện tích là $3 \times 2 = 6$ ô đơn vị, cần dùng đúng 3 viên gạch domino. Có tất cả đúng 3 ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với sàn nhà kích thước 3×2 ($N = 2$), tổng diện tích là $3 \times 2 = 6$ ô đơn vị, cần dùng đúng 3 viên gạch domino. Có tất cả đúng 3 cách ghép hợp lệ:

1. Một viên đặt dọc ở cột 1, hai viên đặt ngang ở hàng 2 và 3.
2. Hai viên đặt ngang ở hàng 1 và 2, một viên đặt dọc ở cột 2.

3. Ba viên đặt ngang song song với nhau. Kết quả in ra là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng *dp*: Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn (INF = $1e18$), tránh dùng 0x3f khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liên kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    if (n % 2 != 0) {
        cout << 0 << "\n";
        return 0;
    }

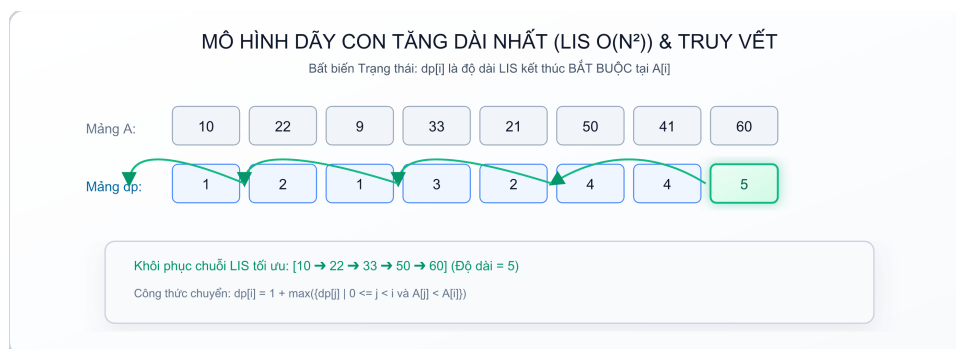
    vector<long long> a(n + 1, 0), b(n + 1, 0);
    a[0] = 1;
    b[0] = 0;
    if (n >= 1) b[1] = 1;

    for (int i = 2; i <= n; ++i) {
        a[i] = (a[i - 2] + 2LL * b[i - 1]) % MOD;
        b[i] = (a[i - 1] + b[i - 2]) % MOD;
    }

    cout << a[n] << "\n";
    return 0;
}
```

Bài 09 [cppb_dp1_09_day_con_tang_dai_nhat_lis_n2]: Dây Con Tăng Dài Nhất LIS $O(N^2)$

Bối cảnh: Tại trạm quan trắc địa chấn ven biển, các chuyên gia theo dõi sự biến thiên áp suất lớp vỏ trái đất qua một chuỗi N lần đo liên tiếp, tương ứng với dãy số $A_1, A_2, \dots, A_N$. Để nhận diện xu thế gia tăng địa chấn qua các chu kỳ, các chuyên gia cần trích xuất một chuỗi các mốc đo có giá trị áp suất tăng dần nghiêm ngặt theo thời gian, sao cho số lượng mốc đo được chọn là nhiều nhất có thể.



Hình minh họa: Mô hình tìm dãy con tăng dài nhất LIS qua các vị trí trước đó

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài lớn nhất của một dãy con tăng nghiêm ngặt (không nhất thiết phải là các phần tử liên tiếp trong dãy ban đầu).

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$) biểu diễn số lượng phần tử của dãy.
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$), các số cách nhau bởi khoảng trắng.

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài của dãy con tăng nghiêm ngặt dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 10 20 10 30 20 50	4

Giải thích: Với dãy số gồm 6 phần tử $[10, 20, 10, 30, 20, 50]$: Một dãy con tăng nghiêm ngặt dài nhất có thể chọn là $[10, 20, 30, 50]$ (tương ứng với các chỉ số 1, 2, 4, 6 trong dãy gốc). Độ dài của dãy con này là 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài lớn nhất của một dãy con tăng nghiêm ngặt (không nhất thiết phải là các phần tử liên tiếp trong dãy ban đầu).
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lập.
- Độ phức tạp:** Thời gian tối ưu $O(N)$ hoặc $O(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $O(N)$ hoặc nén về $O(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 6 10 20 10 30 20 50 $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 6 10 20 10 30 20 50	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy số gồm 6 phần tử [10,20,10,30,20,50] : Một dãy con tăng nghiêm ngặt dài nhất có thể chọn là [10,20,30,50] (tương ứng...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với dãy số gồm 6 phần tử [10,20,10,30,20,50]: Một dãy con tăng nghiêm ngặt dài nhất có thể chọn là [10,20,30,50] (tương ứng với các chỉ số 1, 2, 4, 6 trong dãy gốc). Độ dài của dãy con này là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng *dp*: Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liên kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<int> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    vector<int> dp(n, 1);
    int ans = 1;

    for (int i = 1; i < n; ++i) {
```

```

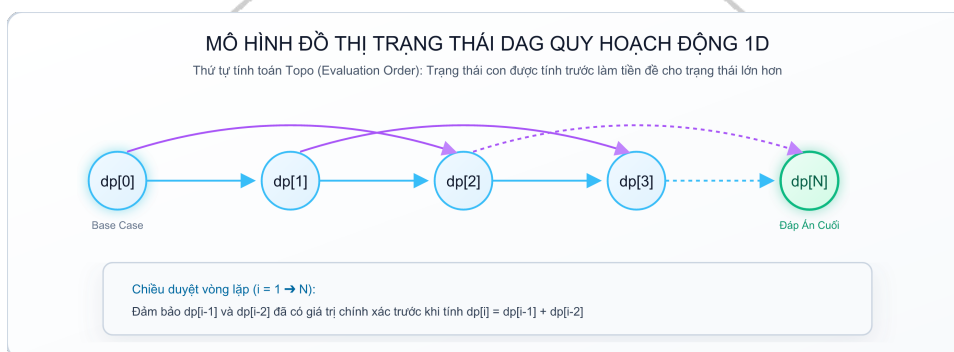
for (int j = 0; j < i; ++j) {
    if (a[j] < a[i]) {
        dp[i] = max(dp[i], dp[j] + 1);
    }
}
ans = max(ans, dp[i]);
}

cout << ans << "\n";
return 0;
}

```

Bài 10 [cppb_dp1_10_day_con_tang_dai_nhat_lis_nlogn]: Dãy Con Tăng Dài Nhất LIS $O(N \log N)$

Bối cảnh: Quy mô trạm quan trắc địa chấn được nâng cấp tự động hóa trên diện rộng, số lượng mẫu dữ liệu cảm biến thu thập trong một đợt quan sát lên tới $N = 10^5$ phần tử. Với tập dữ liệu khổng lồ này, bài toán tìm dãy con tăng dài nhất đòi hỏi một thuật toán có hiệu năng vượt trội để xử lý tức thời trong vòng 1 giây.



Hình minh họa: Mô hình chuyển trạng thái Quy hoạch động 1 chiều dạng đồ thị có hướng không chu trình (DAG)

Nhiệm vụ: Cho dãy A gồm N số nguyên với N lên tới 10^5 . Hãy lập trình xác định độ dài của dãy con tăng nghiêm ngặt dài nhất.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là độ dài lớn nhất của dãy con tăng nghiêm ngặt.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 5 2 7 4 3 8	3

Giải thích: Với dãy số $[5, 2, 7, 4, 3, 8]$: Dãy con tăng nghiêm ngặt dài nhất có thể chọn là $[2, 4, 8]$ (hoặc $[2, 3, 8]$, $[5, 7, 8]$). Độ dài dài nhất đạt được là 3.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy A gồm N số nguyên với N lên tới 10^5 . Hãy lập trình xác định độ dài của dãy con tăng nghiêm ngặt dài nhất.

- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 6 5 2 7 4 3 8 $\Rightarrow$ Đầu ra kỳ vọng: 3.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 6 5 2 7 4 3 8	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy số [5,2,7,4,3,8] : Dãy con tăng nghiêm ngặt dài nhất có thể chọn là [2,4,8] (hoặc [2,3,8] , [5,7,8]). Độ dài d...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 3

Giải thích chi tiết: Với dãy số [5,2,7,4,3,8]: Dãy con tăng nghiêm ngặt dài nhất có thể chọn là [2,4,8] (hoặc [2,3,8], [5,7,8]). Độ dài dài nhất đạt được là 3.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn (INF = $1e18$), tránh dùng 0x3f khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
int n;
if (!(cin >> n)) return 0;
if (n <= 0) return 0;

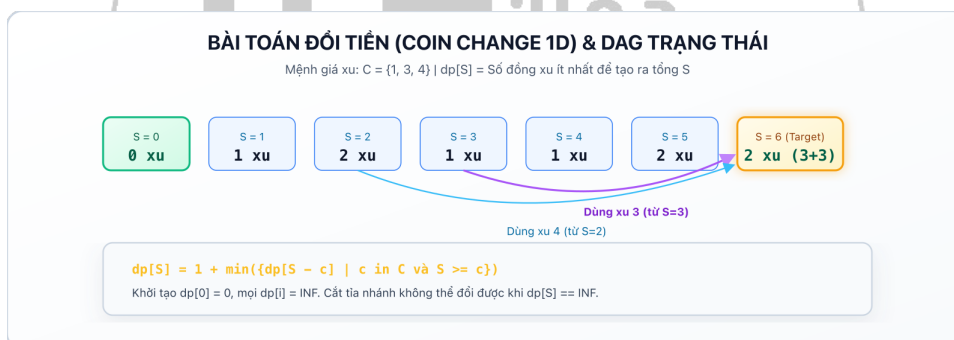
vector<long long> a(n);
for (int i = 0; i < n; ++i) cin >> a[i];

vector<long long> tails;
for (int i = 0; i < n; ++i) {
    auto it = lower_bound(tails.begin(), tails.end(), a[i]);
    if (it == tails.end()) {
        tails.push_back(a[i]);
    } else {
        *it = a[i];
    }
}

cout << tails.size() << "\n";
return 0;
}
```

Bài 11 [cppb_dp1_11_truy_vet_day_con_tang_dai_nhat]: Truy Vết Dãy Con Tăng Dài Nhất

Bối cảnh: Sau khi tính toán được độ dài của xu thế gia tăng dữ liệu áp suất địa chấn, ban chỉ đạo cần xuất báo cáo chi tiết bao gồm chính xác các giá trị đo đặc cụ thể đã hình thành nên xu thế đó để gửi tới viện nghiên cứu phân tích.



Hình minh họa: Đồ thị chuyển trạng thái bài toán đổi tiền xu tối ưu

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm và in ra toàn bộ các phần tử thuộc về một dãy con tăng nghiêm ngặt dài nhất. Nếu có nhiều dãy con cùng đạt độ dài lớn nhất, bạn chỉ cần in ra một dãy con bất kỳ thỏa mãn.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Dòng 1: In ra số nguyên L là độ dài của dãy con tăng dài nhất.
- Dòng 2: In ra L số nguyên là các giá trị của dãy con được chọn, cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
-----------------	-----------------

5 2 1 4 3 5

3 2 4 5

Giải thích: Với dãy số gốc là $[2,1,4,3,5]$: Dãy con tăng dài nhất có độ dài bằng 3. Một dãy con hợp lệ thỏa mãn điều kiện tăng nghiêm ngặt là $[2,4,5]$ (hoặc $[1,4,5]$, $[1,3,5]$). Kết quả dòng 1 in ra 3, dòng 2 in ra các số 2 4 5.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm và in ra toàn bộ các phần tử thuộc về một dãy con tăng nghiêm ngặt dài nhất. Nếu có nhiều dãy con cùng đạt độ dài lớn nhất, bạn chỉ cần in ra một dãy con bất kỳ thỏa mãn.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 2 1 4 3 5 $\Rightarrow$ Đầu ra kỳ vọng: 3 2 4 5.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 2 1 4 3 5	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy số gốc là $[2,1,4,3,5]$: Dãy con tăng dài nhất có độ dài bằng 3. Một dãy con hợp lệ thỏa mãn điều kiện tăng nghiêm ngặt là ...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đổi chiều trạng thái cuối cùng	Output chuẩn: 3 2 4 5

Giải thích chi tiết: Với dãy số gốc là $[2,1,4,3,5]$: Dãy con tăng dài nhất có độ dài bằng 3. Một dãy con hợp lệ thỏa mãn điều kiện tăng nghiêm ngặt là $[2,4,5]$ (hoặc $[1,4,5]$, $[1,3,5]$). Kết quả dòng 1 in ra 3, dòng 2 in ra các số 2 4 5.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng *dp*: Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn (INF = $1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    vector<int> dp(n, 1);
    vector<int> parent(n, -1);
    int max_len = 1;
    int best_end = 0;

    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                parent[i] = j;
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
            best_end = i;
        }
    }

    vector<long long> lis;
    int curr = best_end;
    while (curr != -1) {
        lis.push_back(a[curr]);
        curr = parent[curr];
    }
    reverse(lis.begin(), lis.end());

    cout << max_len << "\n";
    for (int i = 0; i < (int)lis.size(); ++i) {
        cout << lis[i] << (i + 1 == (int)lis.size() ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

Bài 12 [cppb_dp1_12_day_con_giam_dai_nhat]: Dãy Con Giảm Dài Nhất (LDS)

Bối cảnh: Trong quá trình hạ nhiệt độ lò luyện kim công nghiệp, các cảm biến ghi nhận nhiệt độ theo từng chu kỳ làm mát tạo thành dãy N số nguyên $A_1, A_2, \dots, A_N$. Kỹ sư vận hành cần đánh giá tính ổn định của quy trình thông qua độ dài của chuỗi các lần đo có nhiệt độ giảm dần nghiêm ngặt dài nhất.



Hình minh họa: Mô hình tìm dãy con tăng dài nhất LIS qua các vị trí trước đó

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài của dãy con giảm nghiêm ngặt dài nhất.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài của dãy con giảm nghiêm ngặt dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 10 9 2 5 3 7 101 18	4

Giải thích: Với dãy 8 số nguyên $[10, 9, 2, 5, 3, 7, 101, 18]$: Một dãy con giảm nghiêm ngặt dài nhất có thể trích xuất là $[10, 9, 5, 3]$ (hoặc $[10, 9, 7, 3]$). Độ dài lớn nhất đạt được là 4.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài của dãy con giảm nghiêm ngặt dài nhất.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- Độ phức tạp:** Thời gian tối ưu $O(N)$ hoặc $O(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $O(N)$ hoặc nén về $O(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 5 10 9 2 5 3 7 101 18 $\Rightarrow$ Đầu ra kỳ vọng: 4.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 5 10 9 2 5 3 7 101 18	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy 8 số nguyên [10,9,2,5,3,7,101,18]: Một dãy con giảm nghiêm ngặt dài nhất có thể trích xuất là [10,9,5,3] (hoặc [...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 4

Giải thích chi tiết: Với dãy 8 số nguyên [10,9,2,5,3,7,101,18]: Một dãy con giảm nghiêm ngặt dài nhất có thể trích xuất là [10,9,5,3] (hoặc [10,9,7,3]). Độ dài lớn nhất đạt được là 4.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng *dp*: Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn (INF = $1e18$), tránh dùng 0x3f khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liên kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        a[i] = -a[i]; // Đảo dấu để tìm LIS tương đương LDS
    }

    vector<long long> tails;
    for (int i = 0; i < n; ++i) {
        auto it = lower_bound(tails.begin(), tails.end(), a[i]);
        if (it == tails.end()) {
```

```

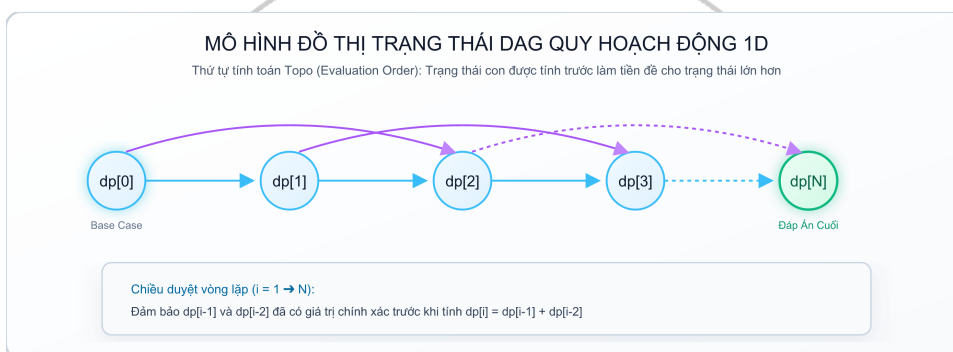
        tails.push_back(a[i]);
    } else {
        *it = a[i];
    }
}

cout << tails.size() << "\n";
return 0;
}

```

Bài 13 [cppb_dp1_13_day_con_hinh_song_nui_bitonic]: Dãy Con Hình Sóng Núi (Longest Bitonic Subsequence)

Bối cảnh: Một thiết bị bay không người lái (drone) thực hiện hành trình bay thám hiểm qua một dãy núi. Độ cao của drone tại các điểm quan sát tạo thành một chuỗi N số nguyên. Một hành trình bay được gọi là có hình dạng sóng núi (Bitonic) nếu độ cao đầu tiên tăng dần nghiêm ngặt lên đến một đỉnh nào đó, sau đó giảm dần nghiêm ngặt từ đỉnh đó xuống cuối (cho phép phần tăng hoặc phần giảm rỗng). Ban điều hành muốn tìm một hành trình sóng núi chứa nhiều điểm quan sát nhất.



Hình minh họa: Mô hình chuyển trạng thái Quy hoạch động 1 chiều dạng đồ thị có hướng không chu trình (DAG)

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài lớn nhất của một dãy con có dạng sóng núi.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$).
- Dòng 2: Chứa N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^5$).

Đầu ra (Output):

- In ra trên một dòng duy nhất độ dài lớn nhất của dãy con hình sóng núi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 1 11 2 10 4 5 2 1	6

Giải thích: Với dãy số gồm 8 phần tử $[1, 11, 2, 10, 4, 5, 2, 1]$: Dãy con hình sóng núi dài nhất có thể chọn là $[1, 2, 10, 5, 2, 1]$ (với đỉnh là 10; phần tăng gồm 1, 2, 10 và phần giảm gồm 10, 5, 2, 1). Tổng số phần tử của dãy sóng núi này là 6.

1. Ý tưởng & Phân tích thuật toán

- **Bản chất bài toán:** Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài lớn nhất của một dãy con có dạng sóng núi.
- **Phương pháp tiếp cận & Chiến lược tối ưu:**
- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 8 1 11 2 10 4 5 2 1 $\Rightarrow$ Đầu ra kỳ vọng: 6.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 8 1 11 2 10 4 5 2 1	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy số gồm 8 phần tử [1,11,2,10,4,5,2,1] : Dãy con hình sóng núi dài nhất có thể chọn là [1,2,10,5,2,1] (với đỉnh là...)	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 6

Giải thích chi tiết: Với dãy số gồm 8 phần tử [1,11,2,10,4,5,2,1]: Dãy con hình sóng núi dài nhất có thể chọn là [1,2,10,5,2,1] (với đỉnh là 10; phần tăng gồm 1, 2, 10 và phần giảm gồm 10, 5, 2, 1). Tổng số phần tử của dãy sóng núi này là 6.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn (INF = $1e18$), tránh dùng 0x3f khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liên kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    // LIS từ trái sang phải
    vector<int> inc(n, 1);
    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i]) inc[i] = max(inc[i], inc[j] + 1);
        }
    }

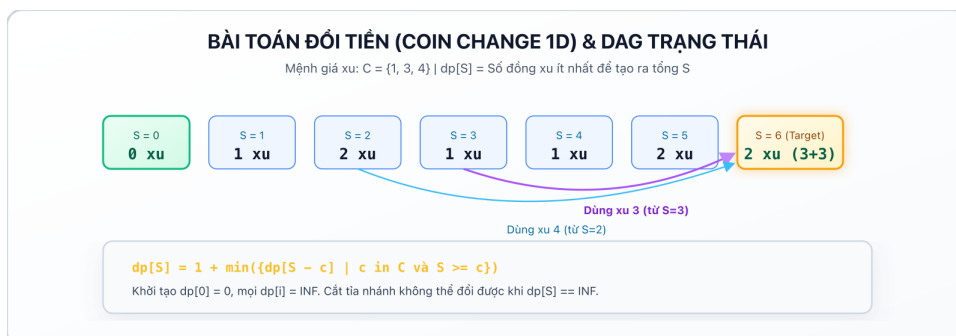
    // LDS từ phải sang trái
    vector<int> dec(n, 1);
    for (int i = n - 2; i >= 0; --i) {
        for (int j = n - 1; j > i; --j) {
            if (a[j] < a[i]) dec[i] = max(dec[i], dec[j] + 1);
        }
    }

    int max_bitonic = 0;
    for (int i = 0; i < n; ++i) {
        max_bitonic = max(max_bitonic, inc[i] + dec[i] - 1);
    }

    cout << max_bitonic << "\n";
    return 0;
}
```

Bài 14 [cppb_dp1_14_tong_day_con_tang_lon_nhat]: Tổng Dãy Con Tăng Lớn Nhất (MSIS)

Bối cảnh: Một nhà đầu tư phân tích lợi nhuận của các cổ phiếu qua N quý liên tiếp. Để xây dựng chiến lược mua vào đón đầu chu kỳ phát triển bền vững, nhà đầu tư muốn chọn ra một chuỗi các quý đầu tư mà lợi nhuận ở quý sau luôn cao hơn quý trước (tăng nghiêm ngặt), đồng thời tổng lợi nhuận thu được từ tất cả các quý được chọn phải là lớn nhất.



Hình minh họa: Đồ thị chuyển trạng thái bài toán đổi tiền xu tối ưu

Nhiệm vụ: Cho dãy số nguyên dương A gồm N phần tử. Hãy lập trình tìm tổng giá trị lớn nhất của một dãy con tăng nghiêm ngặt.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$).
- Dòng 2: Chứa N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^5$).

Đầu ra (Output):

- In ra trên một dòng duy nhất tổng giá trị lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 1 101 2 3 100 4 5	106

Giải thích: Với dãy số gồm 7 phần tử $[1, 101, 2, 3, 100, 4, 5]$:

- Dãy con tăng dài nhất là $[1, 2, 3, 4, 5]$ có tổng là $1 + 2 + 3 + 4 + 5 = 15$.
- Nhưng dãy con tăng $[1, 2, 3, 100]$ lại mang lại tổng giá trị lớn hơn nhiều: $1 + 2 + 3 + 100 = 106$. Đây là tổng lớn nhất có thể đạt được.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho dãy số nguyên dương A gồm N phần tử. Hãy lập trình tìm tổng giá trị lớn nhất của một dãy con tăng nghiêm ngặt.
- Phương pháp tiếp cận & Chiến lược tối ưu:**
- Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lập.
- Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 7 1 101 2 3 100 4 5 $\Rightarrow$ Đầu ra kỳ vọng: 106.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 7 1 101 2 3 100 4 5	Khởi tạo cấu trúc dữ liệu ban đầu

2	Chạy thuật toán từng bước	Phân tích mẫu: Với dãy số gồm 7 phần tử [1,101,2,3,100,4,5]: - Dãy con tăng dài nhất là [1,2,3,4,5] có tổng là $1 + 2 + 3 + 4 + 5 = 15$	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 106

Giải thích chi tiết: Với dãy số gồm 7 phần tử [1,101,2,3,100,4,5]:

- Dãy con tăng dài nhất là [1,2,3,4,5] có tổng là $1 + 2 + 3 + 4 + 5 = 15$.
- Nhưng dãy con tăng [1,2,3,100] lại mang lại tổng giá trị lớn hơn nhiều: $1 + 2 + 3 + 100 = 106$. Đây là tổng lớn nhất có thể đạt được.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng *dp*: Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng $0x3f$ khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    vector<long long> dp(n);
    long long ans = 0;

    for (int i = 0; i < n; ++i) {
        dp[i] = a[i];
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i]) {
                dp[i] = max(dp[i], dp[j] + a[i]);
            }
        }
        ans = max(ans, dp[i]);
    }

    cout << ans << "\n";
}
```

```
return 0;
}
```

Bài 15 [cppb_dp1_15_quy_hoach_dong_1d_cuc_han]: Tối Ưu Hóa Chuỗi Dự Án Năng Lượng

Bối cảnh: Một quỹ đầu tư năng lượng tái tạo xem xét danh mục gồm N dự án điện gió và điện mặt trời. Dự án thứ i có quy mô công suất là P_i (MW) và mang lại doanh thu kỳ vọng là V_i (tỷ đồng). Để đảm bảo lộ trình mở rộng thị phần an toàn và bền vững, quỹ đầu tư chỉ được phép phê duyệt một chuỗi các dự án có quy mô công suất tăng dần nghiêm ngặt theo thời gian xét duyệt.



Hình minh họa: Mô hình tìm dãy con tăng dài nhất LIS qua các vị trí trước đó

Nhiệm vụ: Cho danh sách N dự án với công suất P_i và doanh thu V_i . Hãy lập trình chọn ra một chuỗi các dự án có công suất tăng nghiêm ngặt sao cho tổng doanh thu thu về là lớn nhất.

Đầu vào (Input):

- Dòng 1: Chứa số nguyên dương N ($1 \leq N \leq 1000$) biểu diễn số lượng dự án.
- N dòng tiếp theo, dòng thứ i chứa hai số nguyên dương P_i và V_i ($1 \leq P_i \leq 10^5, 1 \leq V_i \leq 10^9$) lần lượt là công suất và doanh thu của dự án thứ i .

Đầu ra (Output):

- In ra trên một dòng duy nhất một số nguyên là tổng doanh thu lớn nhất có thể đạt được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 10 100 5 50 20 200	350

Giải thích: Với 3 dự án năng lượng có thông số $[(10,100), (5,50), (20,200)]$: Chuỗi dự án có công suất tăng dần nghiêm ngặt là chọn dự án 2 (công suất 5), sau đó dự án 1 (công suất 10), và cuối cùng là dự án 3 (công suất 20). Chuỗi công suất là $5 < 10 < 20$, mang lại tổng doanh thu tối đa là $50 + 100 + 200 = 350$.

1. Ý tưởng & Phân tích thuật toán

- Bản chất bài toán:** Cho danh sách N dự án với công suất P_i và doanh thu V_i . Hãy lập trình chọn ra một chuỗi các dự án có công suất tăng nghiêm ngặt sao cho tổng doanh thu thu về là lớn nhất.
- Phương pháp tiếp cận & Chiến lược tối ưu:**

- **Mô hình trạng thái:** Định nghĩa $dp[i]$ là kết quả tối ưu cho bài toán con kết thúc tại bước/vị trí thứ i .
- **Công thức chuyển trạng thái:** Dựa trên các lựa chọn bước đi trước đó (như $dp[i] = dp[i - 1] + dp[i - 2]$ hoặc min/max qua các trạng thái $j < i$).
- **Cơ sở quy hoạch động:** Khởi tạo các trường hợp biên nhỏ nhất $dp[0], dp[1]$ rõ ràng trước khi lặp.
- **Độ phức tạp:** Thời gian tối ưu $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$ (cho LIS cải tiến qua tìm kiếm nhị phân lower_bound), bộ nhớ $\mathcal{O}(N)$ hoặc nén về $\mathcal{O}(1)$.

2. Bảng chạy tay trên số liệu mẫu Mẫu thử (Sample 1): Đầu vào: 3 10 100 5 50 20 200 $\Rightarrow$ Đầu ra kỳ vọng: 350.

Bước	Thao tác thực hiện	Dữ liệu biến đổi & Trạng thái	Kết quả ghi nhận
1	Khởi tạo & Đọc dữ liệu	Nạp Input: 3 10 100 5 50 20 200	Khởi tạo cấu trúc dữ liệu ban đầu
2	Chạy thuật toán từng bước	Phân tích mẫu: Với 3 dự án năng lượng có thông số [(10,100), (5,50), (20,200)]: Chuỗi dự án có công suất tăng dần nghiêm ngặt là chọn dự án 2 (công suất 5)...	Cập nhật các biến / mảng trạng thái
3	Xuất kết quả chuẩn	Đối chiếu trạng thái cuối cùng	Output chuẩn: 350

Giải thích chi tiết: Với 3 dự án năng lượng có thông số [(10,100), (5,50), (20,200)]: Chuỗi dự án có công suất tăng dần nghiêm ngặt là chọn dự án 2 (công suất 5), sau đó dự án 1 (công suất 10), và cuối cùng là dự án 3 (công suất 20). Chuỗi công suất là $5 < 10 < 20$, mang lại tổng doanh thu tối đa là $50 + 100 + 200 = 350$.

3. Lưu ý & Bẫy lỗi thường gặp

- Quên chia lấy dư theo modulo $10^9 + 7$ tại mỗi phép cộng/nhân dẫn đến tràn số nguyên.
- Khởi tạo sai giá trị mảng dp : Các bài tìm giá trị nhỏ nhất cần khởi tạo giá trị vô cùng lớn ($INF = 1e18$), tránh dùng 0x3f khi cộng dồn gây tràn số.
- Lỗi lệch chỉ số giữa 0-based và 1-based khi tham chiếu các phần tử liền kề.

4. Lời giải tham khảo

```
#include <bits/stdc++.h>
using namespace std;
```

```
struct Item {
    long long v, c;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<Item> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i].v >> a[i].c;
    }

    // Sắp xếp theo v tăng dần
    sort(a.begin(), a.end(), [](const Item& x, const Item& y) {
        if (x.v != y.v) return x.v < y.v;
        return x.c > y.c;
    });

    vector<long long> dp(n);
    long long ans = 0;

    for (int i = 0; i < n; ++i) {
        dp[i] = a[i].c;
        for (int j = 0; j < i; ++j) {
            if (a[j].v < a[i].v) {
                dp[i] = max(dp[i], dp[j] + a[i].c);
            }
        }
        ans = max(ans, dp[i]);
    }

    cout << ans << "\n";
    return 0;
}
```

Mục lục

CHƯƠNG 03: MẢNG TIỀN TỔ & TÌM KIẾM NHỊ PHÂN

- Bài 09: Phép toán BIT & biểu diễn trạng thái

CHƯƠNG 04: SỐ HỌC & ĐẠI SỐ MODULAR

- Bài 10: Lý thuyết số & số nguyên tố
- Bài 11: Đồng dư thức, lũy thừa nhị phân & nghịch đảo modulo
- Bài 12: Xử lý số nguyên lớn (BigInt)

CHƯƠNG 05: ĐỆ QUY, CHIA ĐỂ TRỊ & QUAY LUI

- Bài 13: Thuật toán đệ quy & cây gọi hàm
- Bài 14: Kỹ thuật chia để trị
- Bài 15: Thuật toán quay lui & nhánh cận

CHƯƠNG 06: QUY HOẠCH ĐỘNG (DYNAMIC PROGRAMMING)

- Bài 16: Quy hoạch động 1D & dãy con tăng dài nhất

